02

IT Architecture in Practice

Book 2 · Design evaluation and decision-making

Four design studios with worked dossiers and Azure and AWS mappings

16 chapters · 86 exercises · 20 figures

An eight-week route with experiments and architecture dossiers

Revised edition
October 2026

IT Architecture Series. Revised edition, October 2026.
All organisations, people, workloads, budgets and datasets in this book are fictional teaching examples. Any resemblance to real organisations is coincidental.
Product and company names are trademarks of their respective owners and are used only to identify their products. This book is a study text, not professional, legal or financial advice.
Set in Nimbus Sans and DejaVu Sans Mono.

Contents

How to use the design studio 3
01 Architecture as a defensible argument 6
02 Research experiments and architectural evidence 9
03 Integration studio brief and boundary design 13
04 Integration contracts and competing designs 18
05 Integration evaluation and a changed quota 24
06 Transaction studio brief and business invariants 30
07 Reservations, payments and coordination 35
08 Transaction experiments and failure review 39
09 Data studio brief and semantic contracts 45
10 Pipeline alternatives and versioned publication 48
11 Data quality, security and analytical evaluation 52
12 Modernisation studio brief and discovery 59
13 Data movement, write fencing and recovery 62
14 Cutover rehearsal and modernisation review 69
15 Architecture review, communication and judgement 73
16 Portfolio assessment and professional transfer 76
Exercise answers and assessment guidance 79
Glossary 88
Experiments, templates and research tasks 91
References 94
Index 97

How to use the design studio

This book teaches architecture by doing it. You work through four design studios. Each studio gives you a brief, asks for a first design, then adds measurements and new facts that make you revise it. The aim is a professional habit: state what the design promises, show the mechanism that keeps the promise, and test the promise before anyone relies on it.

The book builds on the foundations of the first book in this series, such as quality scenarios, transactions, messaging and data modelling. You do not need that book open: whenever a chapter relies on an earlier idea, it recaps it first.

The four studios

Studio	Organisation	Central problem	Chapters
Integration	Meridian Service Portal	Hand service requests to three different CRMs without losing or duplicating them	3 to 5
Transactions	Aster Retail	Sell 2,000 units of one product to 500 checkout attempts per second	6 to 8
Data	Beacon Analytics	Publish a daily dashboard that stays explainable when data arrives late or changes	9 to 11
Modernisation	Orion	Move order management off a legacy platform with one write authority at every moment	12 to 14

Chapters 1 and 2 teach the method: how to argue for a design and how to gather evidence for it. Chapter 15 covers design reviews, and Chapter 16 brings your work together in a portfolio.

Meridian, Aster, Beacon and Orion are fictional organisations. Their workloads, budgets, provider contracts, datasets and experiment results are teaching examples, not industry benchmarks. Each studio states its own assumptions, which apply only within that studio.

How a studio works

Every studio follows the same cycle:

1. Read the brief: the business problem, the workload, the constraints and the open questions.
2. Produce a first design before reading the worked analysis, and keep it, so that you can see later what you learned.
3. Study the worked analysis, which compares at least two credible alternatives.
4. Run the experiment: a calculation, a small Python model or a SQL listing.
5. Respond to the change card. A **change card** introduces one new fact part-way through the studio, for example a vendor cutting its rate limit, and asks you to revise the design and record why.
6. Assess your dossier against the rubric in Chapter 16.

The output of a studio is an **architecture dossier**: the connected set of documents that explains a design and the reasons for it. Figure 1 shows how its parts link. Every item carries a stable identifier, such as Q3 for a quality scenario or ADR-M05 for a decision record, so that a reviewer can follow a requirement to the decision that addresses it and the test that supports it.

The worked designs show one sound answer. An alternative earns equal credit when it protects the same business rules and explains its cost, operation and remaining uncertainty.

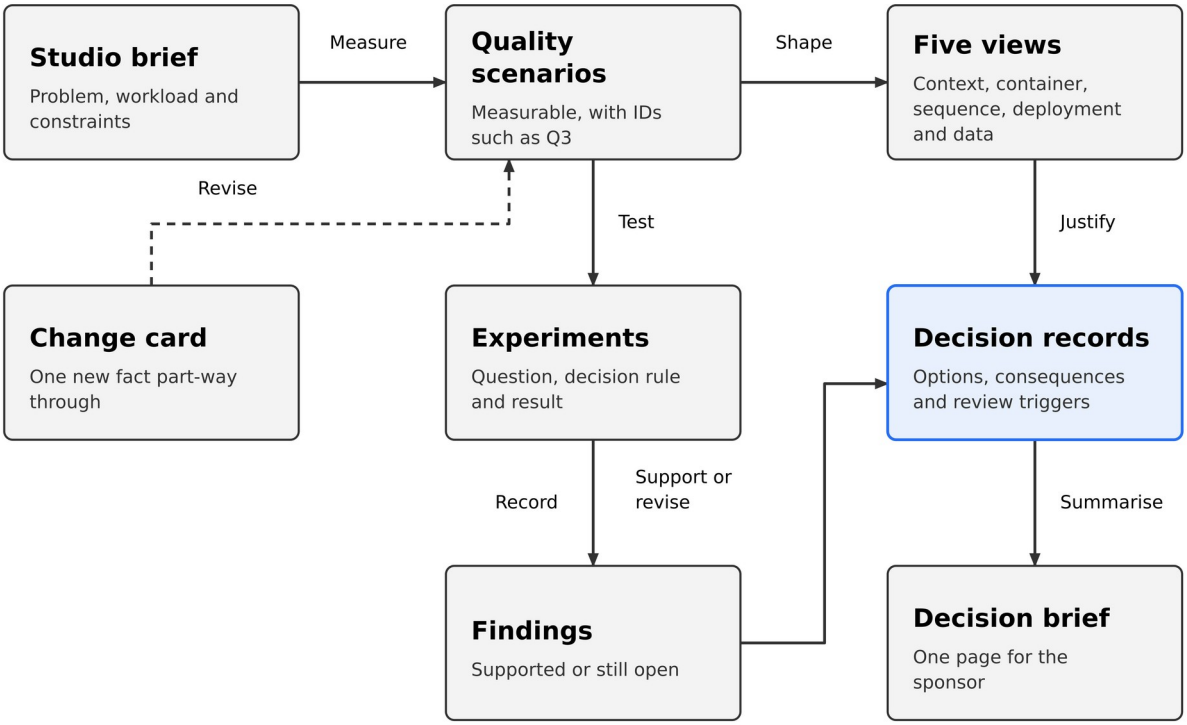


Figure 1 The parts of an architecture dossier, linked by stable identifiers so that each decision can be traced to its scenarios and tests. The dashed route shows a revision triggered by a change card.

What you will produce

- Four studio dossiers.
- Four experiment records, each with a question, a decision rule and a result.
- Four research critiques of 400 to 600 words, each applying one primary source to a studio.
- A final portfolio, prepared for an oral defence.

An eight-week study route

Plan five study days a week of four to five hours each, about 160 to 200 hours in total. Spend roughly a quarter of each day reading, half on modelling and experiments, and the rest on critique and revision.

Week	Reading and focus	Main output
1	Chapters 1 and 2	Argument method and a first experiment
2	Chapters 3 to 5	Integration dossier and recovery analysis
3	Chapters 6 to 8	Transaction dossier and failure tests
4	Chapters 9 to 11	Data dossier and publication design
5	Chapters 12 to 14	Migration dossier and cutover plan
6	Revisit all four studios	Alternative designs and research critiques
7	Chapter 15	Design review and revised decisions
8	Chapter 16	Final portfolio and oral defence

Listings, templates and sources

Listings use Python 3.10 or newer with the standard library, and SQL. The PostgreSQL listings were run on PostgreSQL 18, and the features they rely on, such as `FOR UPDATE SKIP LOCKED` (available since PostgreSQL 9.5) and `EXECUTE FUNCTION` in triggers (since PostgreSQL 11), work in every currently

supported release. Wherever the text reports how a statement behaved, for example under concurrency, that behaviour was observed by running it. The Python listings, including those that use SQLite, run anywhere Python runs. Each is printed with its exact output, and none needs a cloud account or network access. The final chapter, “Experiments, templates and research tasks”, contains the dossier templates and extensions for each experiment.

References are numbered in IEEE style in order of first citation. Research papers support general ideas; standards and official documentation support specific protocol and product behaviour. Product behaviour changes over time, so check the documentation for the exact service, version and configuration before relying on it.

CHAPTER 1

Architecture as a defensible argument

Learning objectives

After this chapter you should be able to:

- Explain how a requirement, a claim, a mechanism and a test combine into an architecture argument.
- Classify statements as constraints, preferences or assumptions, and apply constraints before any weighted scoring.
- Design a small set of views that share one vocabulary and agree about failure behaviour.
- Write an architecture decision record with alternatives, consequences and a measurable review trigger.

Start from a decision

An assignment becomes manageable when it names a decision. “Design an integration platform” invites unlimited scope. “Choose how service requests reach three CRMs while each business unit keeps its own case records” names the actors, the boundary and the consequences, so it can be answered, challenged and revised.

An architecture is therefore an argument: it claims that a particular arrangement of components, data and responsibilities meets particular needs. A strong argument shows how each promise is kept and what would show that it is broken.

The argument chain

This book uses a six-link chain to make the argument explicit:

- A **requirement** is an observable promise. For example: a valid service request receives a receipt within one second.
- A **claim** states what the design will achieve: the portal keeps accepting requests while a CRM is down for thirty minutes.
- A **mechanism** explains how: the API stores the request and a handoff record in one database transaction, and a worker delivers the handoff later.
- **Evidence** challenges the claim: a test, a calculation, a measurement or a documented contract.
- A **finding** states what the evidence supports and what remains open.
- A **decision** chooses an option, within a stated scope and under stated conditions.

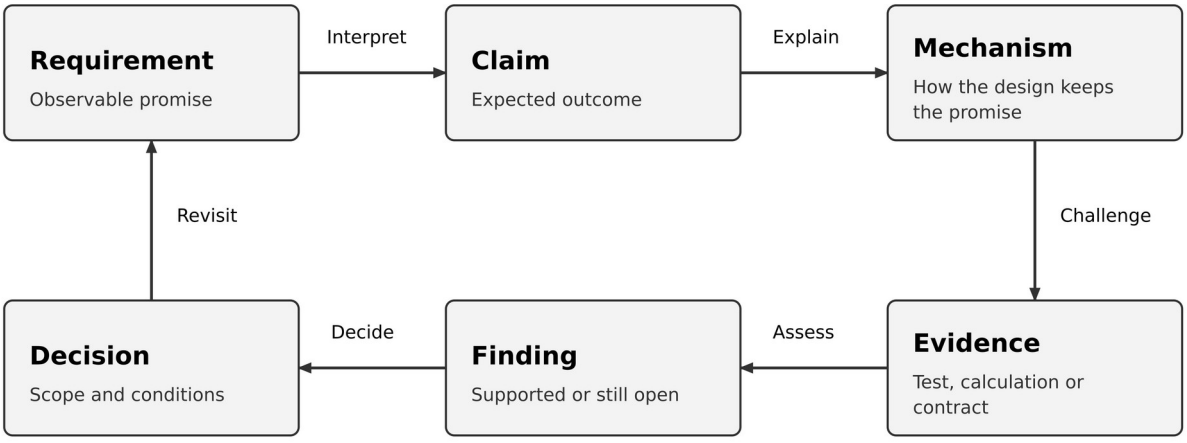


Figure 2 An architecture argument links a requirement to a decision through a claim, a mechanism, evidence and a finding.

Figure 2 draws the chain as a cycle, because new findings often send you back to the requirement. Take the claim that the portal tolerates a thirty-minute CRM outage. It needs three observations: requests are accepted during the outage, accepted requests survive a restart of the portal, and the backlog clears within the recovery objective once the CRM returns. A screenshot of a full queue shows none of these.

One counterexample breaks a claim, however polished the rest of the argument. A passing test supports the claim only for the conditions it exercised. That is why later chapters design tests that try to break the design.

Keep a small set of connected views

A **view** describes the system from one perspective. The C4 model offers a widely used hierarchy for software structure: system context, containers and components [1]. Behaviour, placement and data need further views, so a typical dossier has five:

- a **context view** of scope and of which external systems own which facts;
- a **container view** of the running applications and data stores and their responsibilities;
- a **sequence or state view** of important behaviour, including failure;
- a **deployment view** of where containers run and which failures they share;
- a **data view** of identity, ownership and lifecycle.

The views must agree. For example, if the container view shows a worker calling a CRM, the sequence view must say what happens when the CRM commits a case and the reply is lost, and the data view must hold the identifier needed to recognise that case later.

Use one vocabulary

Ambiguous nouns hide disagreements about responsibility. Keep a short register of terms and use them consistently in schemas, diagrams and user messages. The Meridian studio uses this register:

Term	Meaning	Owner
Submission	A customer’s request as stored by the portal	Portal
Receipt	The portal’s confirmation that it has durably stored a submission	Portal
Handoff	The delivery of a submission to the business unit’s CRM	Portal
Service case	The CRM’s business record for the work	CRM
Work order	The ERP record that schedules a technician or parts	ERP

Calling all of these “ticket” would hide the question of who owns what.

Constraints, preferences and assumptions

Statements in a brief fall into three categories:

- A **constraint** decides eligibility. An option that breaks it is out, whatever else it offers.
- A **preference** ranks the eligible options.
- An **assumption** is a fact you believe but have not yet verified.

A weighted scoring matrix suits preferences and misleads about constraints. Suppose three integration options are scored from 1 to 5:

Option	Cost (0.4)	Delivery speed (0.3)	Operability (0.3)	Weighted score	Keeps requests during a 30-minute outage
A	3	3	4	3.3	Yes
B	5	4	3	4.1	No
C	2	2	5	2.9	Yes

Option B scores highest, $0.4 \times 5 + 0.3 \times 4 + 0.3 \times 3 = 4.1$, but it loses accepted requests when the CRM is down. It is ineligible, and the real choice is between A and C. If the business wants B, it must change the promise, and the dossier should say who has the authority to do that.

Assumptions need an owner and a deadline. For example, if a CRM vendor’s sales team says that repeated create requests are safe, record that as assumption A-03, with the person who will verify it, the date and the consequence if it is false.

Record decisions and their history

An **architecture decision record (ADR)** captures one significant decision. Nygard's format records its context, the decision, its status and its consequences [2]. This book adds the alternatives considered, the identifiers of supporting evidence and a review trigger.

A **review trigger** is an observable condition that should reopen the decision:

- Weak: "Move to Kafka later."
- Strong: "Revisit if a second consumer needs to replay handoff events, or if outbox polling uses more than 10 percent of database CPU at peak load."

The weak version names a preferred technology; the strong version names a condition that can be checked. When a decision is replaced, keep the old record and write a new one that supersedes it and says which assumption changed.

A worked dossier fragment

The following fragment links five items by identifier:

- Requirement R-01: a valid request receives a durable receipt while a CRM is unavailable.
- Decision ADR-M01: store the submission and its handoff record in one local transaction.
- Invariant V-01: every receipt has exactly one handoff record.
- Test X-04: stop the portal process just before and just after the commit, 200 times each, and check both tables after restart.
- Finding F-04: no receipt existed without its handoff record, or the reverse. Local atomicity holds for these crash points; duplicate CRM cases remain an open risk, addressed by ADR-M03.

The finding is deliberately narrower than the design. It does not show that the CRM received the request or that the caller was authorised correctly; each of those is a separate claim with its own test.

> **Common mistake.** Treating one passing test as proof of the whole design. State which claim the test supports and list the claims that still need their own evidence.

Exercises

1.1 Turn "the platform must be reliable" into two measurable quality scenarios that observe different things. For each, state the stimulus, the environment, the expected response and the measure.

1.2 A team scores three options with weights of 0.5 for cost, 0.2 for delivery speed and 0.3 for security. Option P scores 4, 3 and 3; option Q scores 5, 4 and 2; option R scores 2, 3 and 5. Q cannot export data in an open format, which the business has made mandatory. Calculate the weighted scores and write a short decision paragraph that applies the constraint before the scores.

1.3 A payment request times out, so its outcome is unknown. Trace this situation through the context, container, sequence, data and deployment views. Name one contradiction between views that a review could detect.

1.4 Critique the review trigger "We will move to microservices when we grow." Rewrite it as a measurable trigger for a system of your choice.

1.5 Classify each statement as a constraint, a preference or an assumption: (a) customer data must stay in the EU; (b) the team would like to use a managed database; (c) the CRM vendor says its API is idempotent; (d) the first release should cost as little as possible to run.

Key points

- An architecture is an argument that links requirements to decisions through claims, mechanisms and evidence.
- One counterexample can break a claim, while a passing test supports only the conditions it exercised.
- Views must share one vocabulary and agree about failure, not just about the normal path.
- Constraints decide eligibility before preferences are scored, and assumptions need an owner and a deadline.
- A decision record with a measurable review trigger keeps the reasoning available when the facts change.

CHAPTER 2

Research experiments and architectural evidence

Learning objectives

After this chapter you should be able to:

- Compare kinds of evidence and match each to the claim it can support.
- Design an experiment with variables, controls, business-level observations and a decision rule fixed in advance.
- Calculate drain times with a constant-rate queue model, and an upper failure bound after failure-free trials.
- Evaluate whether a load test's workload model, open or closed, fits the question being asked.
- Critique a research source by separating what it shows from how you apply it.

Match the evidence to the claim

Different kinds of evidence answer different questions:

Kind of evidence	What it can establish	What it leaves open
Proof	A result that holds within a stated model	Whether the deployed system matches the model
Model or calculation	The consequences of stated assumptions	Whether real traffic follows those assumptions
Load test	How one build behaves under one workload	How another workload or configuration behaves
Fault-injection test	What happens at the injected failure point	What happens at failure points never injected
Product documentation	The vendor's stated contract	Whether your configuration relies on it correctly
Case study	What happened in one organisation	Whether the same causes apply in yours

Architecture evaluation methods apply this reasoning before full implementation. The Architecture Tradeoff Analysis Method (ATAM) examines architectural decisions against quality scenarios and identifies sensitivity points, tradeoff points and risks [3]. The studios use a lighter review with the same principle: spend investigation effort where a result could change the design.

A useful experiment asks a question that could have a negative answer. "Demonstrate our scalable platform" invites confirmation. "Does the worker pool clear 9,000 pending handoffs within thirty minutes while five new handoffs arrive each second?" can fail, so its answer informs a decision. Write down before the run what you will change if it fails.

Variables, controls and observations

An experiment separates three kinds of quantity:

- **Independent variables** are the conditions you change on purpose, such as the number of workers or the offered arrival rate.
- **Dependent variables** are the outcomes you measure, such as completed handoffs per second or the age of the oldest pending handoff.
- **Controlled conditions** are held fixed: the build, the schema, the hardware class, the test data and any competing traffic.

Times such as response time and replication lag are usually summarised by **percentiles**. The p-th percentile is the value that p percent of the observations do not exceed: **p50** is the median, **p95** is the time within which 95 percent of requests finish, and **p99** covers all but the slowest 1 percent. For example, if 1,000 receipts are sorted by response time and the 950th took 0.8 seconds, the p95 is 0.8 seconds, however slow the last 50 were. A percentile says nothing about the observations above it, so a test report also gives the maximum, and Chapter 8 shows why percentiles of separate stages cannot simply be added.

If two things change at once, the result cannot be attributed. For example, a team doubles its workers from four to eight and also moves to a larger database instance. Throughput rises by 60 percent, but nobody knows which change caused it.

Measure business outcomes as well as technical counts, and keep the business identifier, such as the submission ID, separate from the attempt identifier. Suppose 1,000 submissions produce 1,150 successful HTTP responses because 150 calls were retries, and 20 of those retries created a second case. A dashboard of successful responses shows a healthy 1,150. The business result is 1,020 cases for 1,000 requests, which is a defect.

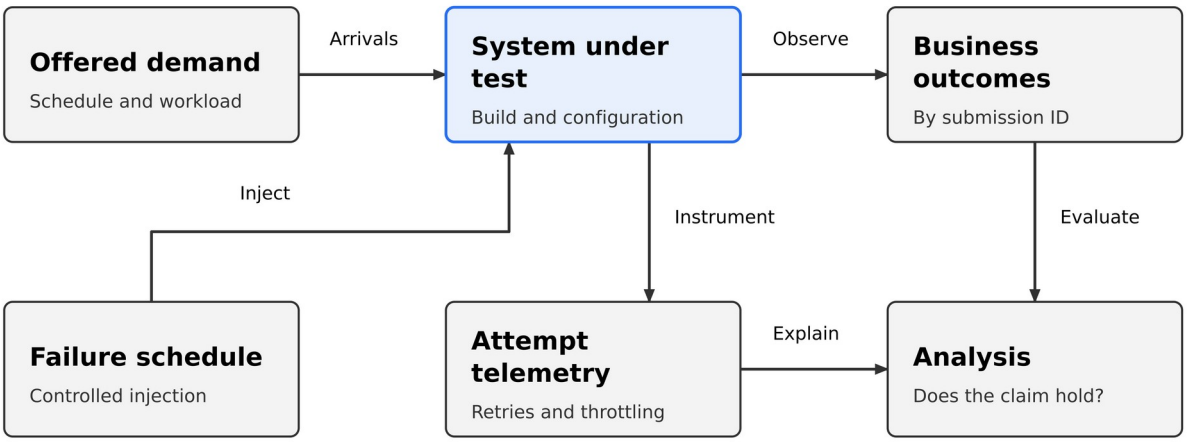


Figure 3 An experiment records demand, injected failures and business outcomes separately from attempt-level telemetry.

Figure 3 shows this separation: attempt telemetry explains what happened, while business outcomes decide whether the claim holds.

Open and closed workloads

Load generators follow one of two models. In a **closed** model, a fixed population of virtual users each sends a request, waits for the response, pauses for a think time and then sends the next. In an **open** model, requests arrive on a schedule whether or not earlier requests have finished. Schroeder, Wierman and Harchol-Balter showed that the two models can behave very differently at the same load, so that the benefit of a design change, such as a new scheduling policy, can look large under one model and small under the other [4].

A closed test can hide overload. With 40 virtual users and a one-second think time, a 0.1-second response time gives an offered rate of 40 / 1.1 ≈ 36.4 requests per second. If responses slow to 1.0 second, the rate falls to 40 / 2.0 = 20 per second. The slower the system, the less load the test offers, so the queue never grows. Customers reacting to a product recall behave like an open system: they keep arriving while the service slows.

Model before you measure

In a **constant-rate queue model**, the backlog changes at the arrival rate minus the successful completion rate. With a backlog B, an arrival rate a and a successful completion rate s, the backlog clears in B / (s – a) seconds when s exceeds a, and never otherwise.

Suppose B is 9,000, a is 5 per second and s is 10 per second. The drain time is 9,000 / 5 = 1,800 seconds: exactly thirty minutes, with no allowance for failed attempts or slow calls. A thirty-minute objective therefore needs more successful capacity, less admitted demand or a revised promise, and the model found this before any load test. The listing implements the model and the failure bound discussed below.

```
import math

def drain_seconds(backlog: int, arrivals: float, successes: float) -> float:
    """Time to clear a backlog when work arrives and completes at constant rates."""
    if successes <= arrivals:
        return math.inf                 # the backlog never shrinks
    return backlog / (successes - arrivals)

def zero_failure_bound(trials: int, confidence: float = 0.95) -> float:
    """Upper bound on the failure probability after `trials` runs with no failure."""
    return 1 - (1 - confidence) ** (1 / trials)

def trials_needed(target: float, confidence: float = 0.95) -> int:
    """Smallest number of failure-free trials whose bound falls below `target`."""
    return math.ceil(math.log(1 - confidence) / math.log(1 - target))

for backlog, arrivals, successes in [(9000, 5, 10), (9000, 5, 15), (9000, 5, 5)]:
    seconds = drain_seconds(backlog, arrivals, successes)
    shown = 'never' if math.isinf(seconds) else f'{seconds:,.0f} s'
    print(f'backlog {backlog:,}, arrivals {arrivals}/s, '
          f'successes {successes}/s: {shown}')
print(f'bound after 400 clean crash trials: {zero_failure_bound(400):.2%}')
print(f'bound if they behave as 8 independent batches: {zero_failure_bound(8):.1%}')
print(f'clean trials needed for a bound below 0.5%: {trials_needed(0.005):,}')
```

Running it prints:

```
backlog 9,000, arrivals 5/s, successes 10/s: 1,800 s
backlog 9,000, arrivals 5/s, successes 15/s: 900 s
backlog 9,000, arrivals 5/s, successes 5/s: never
bound after 400 clean crash trials: 0.75%
bound if they behave as 8 independent batches: 31.2%
clean trials needed for a bound below 0.5%: 598
```

The constant rates are an assumption that real traffic never meets exactly. The model's value is that it makes its assumptions explicit and gives the experiment a number to beat.

Little's law is a further consistency check. For a stable system over the long run, the average number of items in the system equals the arrival rate multiplied by the average time each item spends there: L = λW [5]. For example, if workers complete 15 CRM calls per second and each call takes 0.4 seconds on average, then on average 15 × 0.4 = 6 calls are in progress, so a pool limited to four concurrent calls cannot sustain that rate. The law describes averages in a stable system; it says nothing about peaks or about a backlog that is still growing.

Interpret a run with zero failures

Chapter 1's test X-04 stopped the portal 200 times just before the commit and 200 times just after it, and found no lost or orphaned record. What does a clean run of 400 trials allow you to claim?

Suppose the trials are independent and each has the same unknown failure probability p. The probability of seeing no failures in n trials is (1 – p) to the power n. Setting that probability to 0.05 gives a one-sided 95 percent upper bound after zero failures: p_upper = 1 – 0.05^(1/n). For X-04 the bound is 0.75 percent, so the clean run is consistent with a defect that strikes up to about one crash in 133. Pushing the bound below 0.5 percent needs 598 clean trials.

Independence matters more than the count. Suppose the 400 trials ran in 8 batches, each on a freshly deployed build, and a defect would appear in every trial of a batch or in none, for example a misconfigured connection pool. The evidence is then closer to 8 trials than to 400, and the bound on a batch failing is 31.2 percent. Spread trials across the conditions that could hide a defect, such as builds, hosts and tenants, before multiplying them within one condition.

Statistical confidence and scenario coverage are different things. A thousand requests cannot exercise a failure that the test never introduces, while one targeted crash at a transaction boundary can expose a defect that a week of clean traffic would miss.

> **In practice.** Write the decision rule before the run, for example "both policies must lose no accepted request; among those, prefer the one that clears the backlog sooner". Choosing the rule afterwards invites choosing the one that favours the result you hoped for.

Read a paper for transfer

Each studio asks for a short research critique. Identify the source's question, method, assumptions, supported conclusion and limits. Then state which part of your design corresponds to the source's model, and which does not.

For example, Parnas's information-hiding argument helps you judge boundaries around design decisions that are likely to change [6]. It does not say that every hidden decision should become a network service. If Meridian expects one business unit to replace its CRM within eighteen months, an adapter interface that hides all CRM-specific behaviour confines that change to one module. A separate deployable service would add network and operational coupling that the information-hiding argument does not assess.

Exercises

2.1 Recalculate the recovery time for 9,000 pending handoffs with 5 arrivals per second and 9 successful completions per second. Does it meet a thirty-minute objective?

2.2 A test report states: "Zero failures in 2,000 trials proves 99.99 percent reliability." Calculate the 95 percent upper bound that the run supports, and explain two distinct defects in the conclusion.

2.3 Design an experiment that compares two worker retry policies. Specify the independent variables, controlled conditions, business observations, injected failures and a decision rule, all before generating any results.

2.4 A closed load test uses 60 virtual users with a two-second think time. Calculate the offered request rate when the response time is 0.5 seconds and when it is 3 seconds. Explain what the difference means for the test's conclusions.

2.5 A data team publishes corrections once a night. About 3,000 corrected records arrive each day, and a monitoring query shows that on average 1,875 of them are waiting to be published. Use Little's law to find the average time a correction waits. A new rule requires every correction to be published within 24 hours: does the average show that the rule is met? State what Little's law assumes here and what it cannot tell you.

Key points

- Each kind of evidence supports a particular kind of claim, so choose evidence that fits the question.
- A good experiment asks a question that can fail and fixes its decision rule in advance.
- Count business outcomes by business identifier, separately from attempts and retries.
- Closed load generators reduce their offered load as the system slows, which can hide overload.
- Simple models such as B / (s – a) and Little's law expose risks before testing, within their assumptions.
- Zero observed failures still leaves a quantifiable upper bound, and no clean run covers a failure that was never injected.

CHAPTER 3

Integration studio brief and boundary design

Learning objectives

After this chapter you should be able to:

- Explain why a shared portal can promise durable receipts but not one duplicate-prevention guarantee across three different CRMs.
- Model handoff state separately from business case state, using a fixed set of handoff states.
- Design a routing rule that keeps each accepted request with one business unit's CRM.
- Compare three integration alternatives for isolation, operation and cost.

The Meridian portal in brief

> **Building on Book 1.** Book 1 designed Meridian's portal around a single CRM: a durable receipt, a transactional outbox whose handoff records move through five states, idempotency keys scoped to tenant and user, and a recovery estimate for a thirty-minute outage. This studio gives each business unit its own CRM with a different contract for creating cases, adds a product recall that concentrates load on one of them, and asks what the portal can still promise.

Meridian maintains equipment for 600 customer organisations, each of which is a **tenant** of its service portal. Three internal business units employ 40 service agents. Customers use the portal to submit service requests, attach files and follow progress.

The submission path is settled: the API stores each submission and its handoff record in one transaction, the **transactional outbox** pattern [7], returns a receipt, and a worker delivers the handoff later, possibly more than once.

Ownership is divided by fact. The portal owns the receipt and the status that customers see. The CRM owns the service case once the handoff is confirmed. The ERP system owns assets, contracts, entitlements, inventory and work orders.

The studio assignment

In this studio, each of Meridian's three business units runs its own CRM, called CRM A, CRM B and CRM C, and they will stay separate for at least eighteen months. Twelve engineers work on the portal, two of them on integration. The workload and targets are:

- 100,000 service requests a year, about 0.0032 per second on average;
- an exceptional peak of 20 valid submissions per second for ten minutes, with files of up to 10 MB;
- receipts for 95 percent of valid submissions within one second, excluding upload, and 99.9 percent monthly success for eligible requests;
- status changes visible to customers within one minute under normal conditions;
- no accepted request lost when one CRM is unavailable for thirty minutes.

The sponsor adds a recall scenario. A safety notice for one product line, serviced by business unit B, drives 5 submissions per second to CRM B for two hours. The sponsor wants a published recovery objective for the case where CRM B fails during that surge, and a slow CRM must not exhaust the workers or database connections that serve the other two business units.

Staff access follows a fixed rule. A staff member may read a request only with the service-agent role, membership of the request's business unit and an authorised scope that includes the request's tenant. Staff often serve several tenants, so the scope is a set, never a single tenant ID. Customers read their own requests; reading every request of their organisation needs an explicit delegation.

Three CRM contracts

An interface investigation found three different contracts for creating a case:

CRM	Create request	Effect of repeating it	Finding an earlier result
A	Carries an external reference that CRM A keeps unique	Returns the existing case	Exact lookup by external reference
B	Has no unique reference	Creates a second case	Search on a reference field whose index lags, usually by under five minutes, with no stated maximum
C	Carries an `Idempotency-Key` header	Returns the earlier result for 24 hours	A status lookup by key for the same 24 hours

CRM C follows the pattern of the `Idempotency-Key` header described in an IETF Internet-Draft, which expired without becoming a standard [8]. The portal sends the submission ID as the key. For 24 hours after first use, a repeat with the same key and body returns the first response; a repeat with a different body is rejected with status 422, and one that arrives while the first is still running is rejected with 409. After 24 hours CRM C forgets the key, so a repeat would create a new case. The status lookup has the same 24-hour lifetime, so one window governs both retries and lookups.

CRM B allows 15 create calls per second, and its searches use a separate allowance in the current contract.

What the portal can promise

The portal controls its own database, so it can promise durable acceptance: once a receipt is issued, the submission and its handoff record exist. Duplicate prevention depends on each CRM's contract. CRM A makes repeats harmless, CRM C makes them harmless for 24 hours, and CRM B makes them dangerous. A common API should therefore expose common states while keeping each CRM's uncertainty visible behind them.

Handoff state

Every handoff record is in exactly one of the five states, with `retry_due` defined more sharply than before:

- `pending`: stored and not yet attempted.
- `in_flight`: a worker holds a time-limited lease on the record and is calling the CRM.
- `confirmed`: the CRM case exists and its case ID is recorded.
- `retry_due`: the last attempt failed or its outcome is unknown, and the next action that the CRM's contract allows, a resend or a lookup, is scheduled for a recorded time.
- `manual_review`: the system cannot settle the outcome safely by itself, so a person decides.

A handoff that cannot be completed automatically waits in `manual_review` with an owner; nothing is dropped silently. Customers see a simpler picture:

Situation	Customer-visible status
`pending`, `in_flight` or `retry_due`, for up to one minute	Received
Any unconfirmed state for longer than that, including `manual_review`	Received, awaiting service team
`confirmed`	The CRM's case status, for example "In progress" or "Resolved"

Business state is separate. A service case moves from "In progress" to "Resolved" in the CRM, independently of any handoff attempt, so the two kinds of state live in different fields. A late delivery retry must never overwrite the CRM's newer status. The portal's status projection stores the CRM case ID, the CRM status and the version the CRM supplies with it.

Route each request to one authority

The ERP service contract for each tenant names the business unit that serves it. At acceptance, the portal stores that business unit with the submission, and every later delivery uses the stored value.

For example, customer T042 moves from business unit B to business unit A on 1 March. A request accepted on 28 February stays with CRM B, even if its handoff is still pending on 2 March. Moving it would need an explicit transfer that closes the case in one CRM and creates it in the other; without one, a request could be created twice or not at all.

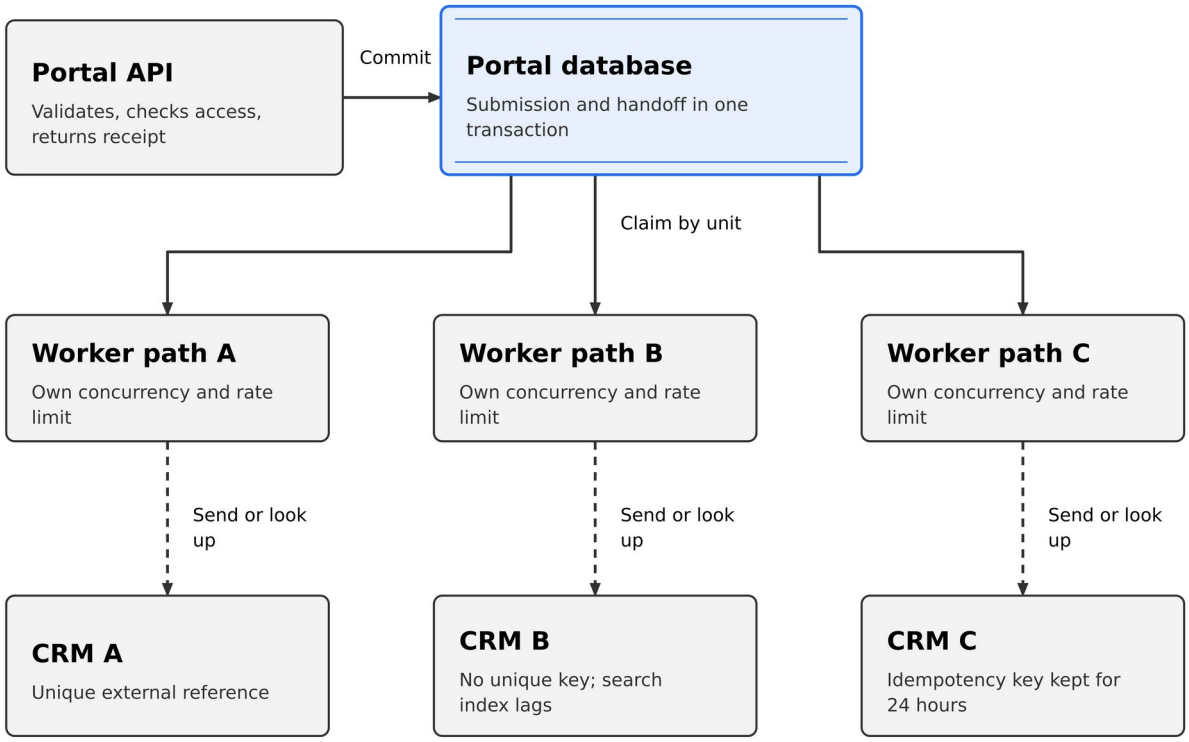


Figure 4 One shared receipt transaction feeds three separately limited delivery paths, one for each business unit's CRM. Dashed arrows are calls to external systems.

Figure 4 separates the shared acceptance boundary from the three delivery paths, each with its own concurrency limit and recovery behaviour.

Isolate the business units

Logical isolation comes first: each worker selects work for one business unit, each path has its own concurrency limit and rate limiter, and alerts are raised per business unit. For example, the portal might allow at most eight simultaneous calls to CRM B, so that if CRM B slows down, only those eight calls wait.

Shared resources still need care. A worker must never hold a database transaction open while it calls a CRM. If each call to CRM B suddenly takes 20 seconds while holding a connection, eight stuck calls tie up eight connections and can drain the pool for everyone. The safe pattern is to claim work in a short transaction, commit, call the CRM, and record the result in a second short transaction.

Separate deployments give stronger isolation at extra cost, and a specific failure should justify them. If CRM B sometimes returns responses so large that they exhaust the worker's memory, running B's adapter in its own process confines the crash. A shared database outage affects every path however the workers are deployed.

Three credible alternatives

The studio compares three designs, none of which removes the need to understand each CRM's create semantics:

- **Option A** is a relational outbox with one adapter per CRM; workers poll the handoff table.
- **Option B** adds a message broker; a relay publishes outbox records, and the adapters consume them.
- **Option C** uses an integration platform with CRM connectors and a durable workflow engine.

Option	Main strength	Main cost or uncertainty
Outbox with polling adapters	Few moving parts and explicit local atomicity	Polling load, fair work selection and replay must be built
Outbox plus broker	Independent consumers and buffering	Another durability boundary to operate and reason about
Integration platform	Managed workflow, timers and connector operations	Connector retry semantics, licence cost and export of history

A broker helps when several consumers need independent progress or retained delivery. The database and the broker cannot commit atomically, so Option B still needs the outbox and a relay that may publish a record twice.

Your first submission

Before reading Chapter 4, produce:

1. a one-page problem brief;
2. eight measurable quality scenarios;
3. a table showing which system owns each fact;
4. container views for two of the alternatives;
5. a handoff contract for one CRM, including what happens when the outcome is unknown;
6. a proposed recovery objective for the recall scenario, with the calculation behind it;
7. one abuse case with its control and audit record, for example a staff member whose tenant scope was revoked an hour ago replaying an old request, and the audit event that records who created or read which submission;
8. one service level objective for the handoff path with the alert that protects it, for example “95 percent of handoffs confirmed within five minutes outside declared recovery periods”, alerting on the age of the oldest pending handoff for each business unit.

Mark every unverified CRM behaviour as an assumption. The brief deliberately leaves the recovery objective open.

Size the manual work too, because correctness depends on it. For example, suppose 0.5 percent of calls to CRM B time out during the two-hour surge of 36,000 requests. That gives 180 unknown outcomes. If a search fifteen minutes later resolves 80 percent of them, 36 reach manual review, which at four minutes each is 2.4 hours of agent time that the business unit can plan for.

Exercises

3.1 Explain why a retry policy that is safe for CRM A is unsafe for CRM B.

3.2 Define a routing rule for a request whose customer moves to another business unit after the request was accepted. State what must happen if the move must apply to open requests too.

3.3 Compare logical worker isolation with separate deployments. Describe one failure that justifies a separate deployment and one that it does not help with.

3.4 During a recall, CRM B receives 36,000 requests and 0.8 percent of calls time out. A later search resolves 75 percent of the timeouts. Each remaining case takes five minutes of agent time. Calculate the manual review workload.

3.5 Write the customer-visible status for three situations: CRM C is down and a handoff is in `retry_due`; a CRM B handoff is in `manual_review`; a confirmed case has been resolved in the CRM. Justify each message.

Key points

- A shared portal can promise durable acceptance, while duplicate prevention depends on each CRM’s contract.
- Five handoff states, `pending`, `in_flight`, `confirmed`, `retry_due` and `manual_review`, keep every request accounted for.
- Handoff state and business case state are separate, and a retry must never overwrite the CRM’s newer status.

- The business unit is stored at acceptance, so an in-flight request keeps one authority unless an explicit transfer moves it.
- Isolation is designed against named failures, and manual work is sized like any other capacity.

CHAPTER 4

Integration contracts and competing designs

Learning objectives

After this chapter you should be able to:

- Design a duplicate-safe submission contract with a scoped idempotency key and a payload check.
- Implement an outbox schema and a claim query that lets several workers share work safely.
- Use the attempt number as a fencing token, and explain why a worker lease protects the local record but not the CRM.
- Analyse an unknown handoff outcome under each CRM contract and choose the next safe action.
- Compare polling, a broker and a workflow platform as ways to deliver handoffs.

The submission contract

A submission passes through four steps. The portal authenticates the caller and establishes their tenant and role. It validates the request and reads the business unit assigned to the tenant, then checks the idempotency key. Finally, it stores the submission and its handoff record in one transaction and returns a receipt.

An **idempotency key** is a client-chosen identifier that lets the server recognise a repeated request. The client sends it in an `Idempotency-Key` header. The header is widely used but not yet standardised: the IETF Internet-Draft that described it has expired without becoming an RFC [8]. As in Book 1, Meridian scopes the key to the tenant and the submitting user and stores a digest of the request body beside it: the same key and body return the stored receipt, a different body is rejected with status 422, and keys are kept for seven days.

The portal returns 201 Created, because the receipt is a new resource. HTTP defines the meaning of the status codes [9]; the business meaning of the receipt is Meridian's to define. The receipt says that Meridian has durably stored the request and taken responsibility for the handoff. The CRM case comes later and has its own identifier.

A repeated request must pass the current access checks again, because the key identifies a request and grants no rights. For example, if user u7 leaves tenant T042, a retry carrying u7's old key must be refused rather than answered with the stored receipt.

The outbox schema

The following PostgreSQL schema holds submissions and their handoff records. The unique constraint enforces the key scope, and the check constraint fixes the five handoff states from Chapter 3. A small policy table records what may follow an unknown outcome at each business unit's CRM, and how long to wait first. CRMs A and C may receive the same create again after a minute. CRM B needs a lookup after fifteen minutes. Each handoff carries its own `next_action`, `send` or `lookup`. The `uuid`, `timestamptz` and `interval` types, partial indexes, `ON CONFLICT`, data-modifying `WITH` clauses and `FOR UPDATE SKIP LOCKED` used in this chapter are PostgreSQL features; other databases express the same ideas with different syntax.

```
CREATE TABLE submission (
  submission_id   uuid PRIMARY KEY,
  tenant_id       text NOT NULL,         -- customer organisation
  submitted_by    text NOT NULL,
  business_unit   text NOT NULL CHECK (business_unit IN ('A', 'B', 'C')),
  idempotency_key text NOT NULL,
  payload_digest  text NOT NULL,
  received_at     timestamptz NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, submitted_by, idempotency_key)
);

CREATE TABLE crm_policy (           -- ADR-M03: the next action after an unknown outcome
  business_unit text PRIMARY KEY,
  after_unknown text NOT NULL CHECK (after_unknown IN ('send', 'lookup')),
  wait          interval NOT NULL
);
INSERT INTO crm_policy VALUES ('A', 'send', '1 minute'),
  ('B', 'lookup', '15 minutes'), ('C', 'send', '1 minute');

CREATE TABLE handoff (
  submission_id   uuid PRIMARY KEY REFERENCES submission,
  business_unit   text NOT NULL REFERENCES crm_policy,
  state           text NOT NULL DEFAULT 'pending' CHECK (state IN
                  ('pending', 'in_flight', 'confirmed', 'retry_due', 'manual_review')),
  next_action     text NOT NULL DEFAULT 'send'
                  CHECK (next_action IN ('send', 'lookup')),
  attempts        integer NOT NULL DEFAULT 0,
  first_attempt_at timestamptz,           -- set by the first claim
  next_attempt_at timestamptz NOT NULL DEFAULT now(),
  lease_owner     text,
  lease_until     timestamptz,
  last_outcome    text,                   -- for example 'timeout' or 'created'
  crm_case_id     text
);

CREATE INDEX handoff_due ON handoff (business_unit, next_attempt_at)
  WHERE state IN ('pending', 'retry_due');
```

The partial index covers only the rows that workers look for, so the confirmed handoffs that soon fill the table stay out of it. One statement writes the submission and its handoff record, so they commit or fail together:

```
WITH s AS (
  INSERT INTO submission (submission_id, tenant_id, submitted_by, business_unit,
                          idempotency_key, payload_digest)
  VALUES ($1, $2, $3, $4, $5, $6)
  ON CONFLICT (tenant_id, submitted_by, idempotency_key) DO NOTHING
  RETURNING submission_id, business_unit
)
INSERT INTO handoff (submission_id, business_unit)
SELECT submission_id, business_unit FROM s
RETURNING submission_id;
```

If the statement returns no row, the key already exists. The portal then reads the stored submission and compares digests: a match returns the stored receipt and a mismatch returns 422. Two identical requests that race are also safe, because PostgreSQL makes the second insert wait for the first and then skip.

Workers claim work with leases

Several workers share the handoff table. Each claims a small batch for one business unit:

```
WITH due AS (
  SELECT submission_id
  FROM handoff
  WHERE business_unit = $1
    AND state IN ('pending', 'retry_due')
    AND next_attempt_at <= now()
  ORDER BY next_attempt_at
  LIMIT $2
  FOR UPDATE SKIP LOCKED
)
UPDATE handoff AS h
SET state = 'in_flight',
    attempts = h.attempts + 1,
    first_attempt_at = coalesce(h.first_attempt_at, now()),
    lease_owner = $3,
    lease_until = now() + interval '2 minutes'
FROM due
WHERE h.submission_id = due.submission_id
RETURNING h.submission_id, h.attempts, h.next_action;
```

`FOR UPDATE SKIP LOCKED` locks the selected rows and skips any row that another transaction has already locked, instead of waiting for it [10]. For example, worker 1 claims the two oldest handoffs for business unit B; worker 2 runs the same query a moment later and immediately receives the third and fourth.

The worker commits the claim, then performs the handoff's `next_action` outside any database transaction: it sends the create request, or, for a CRM B handoff with an unknown outcome, searches for the submission ID. Each claim increases `attempts`, and the claim returns that number to the worker. The worker passes it back with its result, so the number acts as a **fencing token**: an update succeeds only for the attempt that currently holds the lease. Four statements record results and enforce the time limits:

```
-- Confirm. The attempt number returned by the claim is the fencing token.
UPDATE handoff
SET state = 'confirmed', crm_case_id = $4, last_outcome = 'created',
    lease_owner = NULL, lease_until = NULL
WHERE submission_id = $1 AND state = 'in_flight'
  AND lease_owner = $2 AND attempts = $3;

-- Unknown outcome: schedule the next action that the CRM's contract allows.
UPDATE handoff AS h
SET state = 'retry_due', last_outcome = $4,
    next_action = p.after_unknown, next_attempt_at = now() + p.wait,
    lease_owner = NULL, lease_until = NULL
FROM crm_policy AS p
WHERE p.business_unit = h.business_unit
  AND h.submission_id = $1 AND h.state = 'in_flight'
  AND h.lease_owner = $2 AND h.attempts = $3;

-- Sweep expired leases. The abandoned call's outcome is unknown as well.
UPDATE handoff AS h
SET state = 'retry_due', last_outcome = 'lease_expired',
    next_action = p.after_unknown, next_attempt_at = now() + p.wait,
    lease_owner = NULL, lease_until = NULL
FROM crm_policy AS p
WHERE p.business_unit = h.business_unit
  AND h.state = 'in_flight' AND h.lease_until < now();

-- CRM C forgets keys after 24 hours: hand over to a person after 20.
UPDATE handoff
SET state = 'manual_review', last_outcome = 'key_window_closing'
WHERE business_unit = 'C' AND state = 'retry_due'
  AND first_attempt_at < now() - interval '20 hours';
```

The second and third statements give `retry_due` its meaning: the next action allowed by the CRM's contract is scheduled for a recorded time. For example, worker w1 sends a create for request R17 to CRM B as attempt 1 and stalls. Its lease expires, and the sweep moves the handoff to `retry_due` with `next_action = 'lookup'` and `next_attempt_at` fifteen minutes ahead, so no worker can claim it earlier. Tested in PostgreSQL 18, a claim made straight after the sweep returned nothing, and a claim made after the wait returned attempt 2 with the action `lookup`. A definite failure, such as a create that CRM B throttled before processing it, is recorded with `next_action = 'send'` and an increasing backoff, because nothing was created.

The fourth statement applies ADR-M03's limit for CRM C, using the `first_attempt_at` value set when the handoff was first claimed. A lookup that finds exactly one matching case is recorded with the confirm

statement. A lookup that finds no match or several moves the handoff to `manual_review`, using the same guard on owner and attempt number.

A lease protects the record, not the CRM

A lease can expire while its worker is still alive, for example during a long garbage-collection pause. The sweep then moves the handoff to `retry_due`, and a later claim takes it as the next attempt. When the first worker wakes, the attempt number stops it from overwriting the record. A check on `lease_owner` alone would not be enough: a worker process that restarts, or runs several threads, under the same owner ID would pass it. In the same test, a confirm carrying the stale attempt 1 changed no row, while the confirm for attempt 2 succeeded. Nothing, however, stops the call that the first worker already sent.

A fencing token protects a write only when the system receiving the write checks it and rejects older numbers [11]. The portal database checks the attempt number, so the local record is safe. None of the three CRMs checks such a token, so each CRM's own contract decides whether a second call is harmless. Chapter 13 returns to fencing in a case where the organisation controls the receiving system.

When the outcome is unknown

A timeout describes what the worker observed, not what the CRM did. Figure 5 traces request R17 through CRM B. The worker sends a create request; CRM B commits case C91, but the reply is lost, so the worker records a timeout. A search for R17 returns nothing, because the search index has not caught up. Two histories fit these observations: the case exists but is not yet visible, or the create never happened.

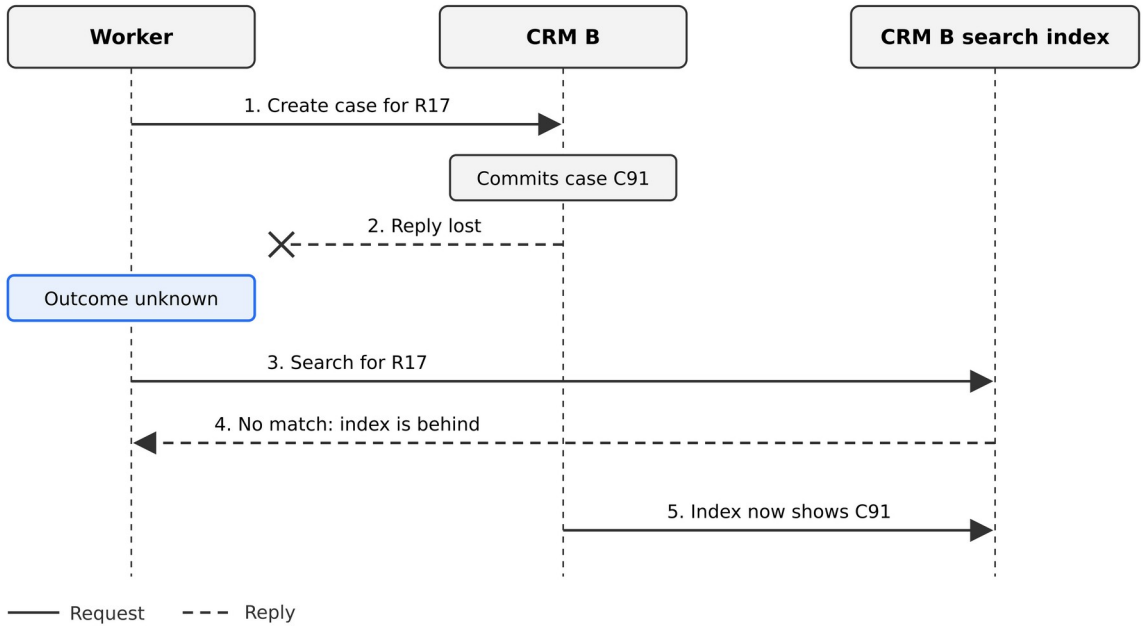


Figure 5 A lost reply and a lagging search index leave the outcome of a CRM B create unknown.

Retrying blindly may create a second case, while treating the timeout as a failure may lead an agent to act on a false status. Helland describes the general principle: messages get retried, so the receiver must make processing a repeat have the same effect as processing it once [12]. CRM A provides that property permanently and CRM C for 24 hours; CRM B does not provide it.

Decision record ADR-M03 therefore sets a separate policy for each CRM:

CRM	Next action after an unknown outcome	Normal path	Goes to `manual_review` when
A	After one minute, send the create again with the same external reference	`retry_due`, then `confirmed`	The returned case belongs to a different submission
B	After fifteen minutes, look up the submission ID	`retry_due`, then `confirmed` if exactly one case matches	The lookup finds no match or more than one
C	After one minute, send again with the same key, or query the key's status	`retry_due`, then `confirmed`	The outcome is still unknown 20 hours after the first attempt

CRM C's limit is 20 hours rather than 24 so that an agent has four hours to settle the case while lookups still work. For CRM B, a negative search never justifies a new create, because the index has no stated maximum lag. Routing every CRM B handoff to agents instead is ruled out by the recall: 36,000 requests at three minutes each would need 1,800 agent hours. Automatic search with a bounded manual queue is the feasible choice until CRM B offers a unique reference.

Resolve a manual review safely

Each record in `manual_review` keeps the submission ID, tenant, business unit, CRM, payload digest, time of first attempt, last outcome, candidate case IDs and resolution history, and nothing more than recovery and audit need. The agent's decision is applied as a guarded update that moves the record to `confirmed` with the chosen case ID only if it is still in `manual_review`. A late reply from an old attempt is compared with the recorded case ID and logged; it never replaces the decision.

Status updates flowing back from a CRM need ordering too. Clocks on different machines disagree, so timestamps from different systems cannot reliably order events [13]. Use the CRM's own version number for each case: if version 7, "Resolved", arrives before version 6, "In progress", the projection keeps version 7.

Compare the implementation choices

Polling outbox. Workers query the partial index in bounded batches, so the database carries both transaction load and work-selection load; test the two together at the peak. At Meridian's volumes the extra load is modest, for example eight indexed queries per second from eight workers polling once a second.

Outbox plus broker. A relay reads the outbox, publishes to the broker and records its progress. If it crashes after the broker accepts a message but before it records progress, it publishes that message again, so consumers must deduplicate by submission ID. RabbitMQ distinguishes publisher confirms, which mean the broker has taken responsibility for a message, from consumer acknowledgements, which mean a consumer has finished with it [14]. Neither says that a CRM case exists.

Workflow platform. A workflow engine provides timers, retries and history. Ask of each connector whether it reuses the same idempotency key on every retry, whether an operator can pause retries for one CRM, and whether the history can be exported for reconciliation.

Decision record ADR-M02 chooses the polling outbox for the first release, because each receipt has one delivery path and the team already operates PostgreSQL. Its review trigger: a second consumer needs to replay handoffs, polling uses more than 10 percent of database CPU at peak, or any business unit's oldest pending handoff exceeds its target at the agreed peak.

Mapping the roles to Azure and AWS

Each option needs the same few runtime roles, and both major clouds offer managed services for them. Microsoft's comparison pages pair the services as follows [15], [16], [17]:

Role in Meridian's design	Azure	AWS
Portal API and workers in containers	Azure Container Apps	Amazon ECS on AWS Fargate
PostgreSQL for submissions and handoffs	Azure Database for PostgreSQL	Amazon RDS for PostgreSQL
Queue for the broker option	Azure Service Bus queues	Amazon SQS
Durable workflow for the platform option	Azure Durable Functions or Logic Apps	AWS Step Functions

Each pair fills the same role, but limits, pricing, failover behaviour and delivery guarantees differ, so check the decisions that depend on them, such as the database's failover and the queue's duplicate handling. Both managed database services run PostgreSQL, so the SQL in this chapter applies to either within their supported versions. The other studios add only a few roles, noted where they arise.

> **Common mistake.** Treating a broker acknowledgement as delivery to the CRM. Each boundary, from local commit to broker to CRM, makes its own promise, and the design must connect them explicitly.

Exercises

4.1 Worker process w3 runs several threads under one owner ID. Thread 1 claims a CRM C handoff as attempt 4 and stalls; the lease expires, the sweep runs, and thread 2 of w3 claims the same handoff as attempt 5 and sends the create again. Thread 1 then receives CRM C's reply to attempt 4 and runs the confirm statement. Trace what happens if the guard checks only `lease_owner`, and what happens with the attempt number added. Can CRM C end up with two cases?

4.2 An agent reviewing a CRM B handoff finds two candidate cases. Design the resolution transition. State what the system must avoid claiming and what happens to a late reply from an earlier attempt.

4.3 Write an ADR that compares the polling outbox with an outbox plus broker for Meridian. Include the context, alternatives, consequences and a measurable review trigger.

4.4 CRM B times out after a create. Five minutes later, before the scheduled lookup, an impatient agent searches CRM B by hand, finds no match and asks for the create to be sent again. Give two histories consistent with this evidence and defend the next action under ADR-M03.

4.5 The claim query lets any number of workers take CRM B handoffs at once. Change the design so that at most eight CRM B handoffs are `in_flight` at any time. Explain how your change behaves when two workers claim at the same moment.

Key points

- An idempotency key scoped to tenant and user, plus a payload digest, makes repeated submissions safe without weakening access checks.
- One statement stores the submission and its handoff record, so neither can exist alone.
- `FOR UPDATE SKIP LOCKED` lets workers share a queue table without blocking each other.
- The attempt number returned by the claim is a fencing token for the local record; only the receiving system can reject a stale write, so each CRM's contract decides what is safe after a timeout.
- After an unknown outcome, `retry_due` schedules the next action that the CRM's contract allows: a resend for CRMs A and C, a delayed lookup for CRM B.
- Each delivery boundary makes its own promise, and the design must state how they connect.

CHAPTER 5

Integration evaluation and a changed quota

Learning objectives

After this chapter you should be able to:

- Calculate the backlog, drain time and required capacity for a CRM outage under a call quota.
- Design observations that reveal starvation of one business unit and duplicate effects.
- Evaluate responses to a reduced shared quota in terms of cost, customer promise and support effort.
- Assemble a complete dossier: problem brief, quality scenarios, fact ownership, decision record and decision brief.

Establish the recovery scenario

During the recall surge from Chapter 3, 5 new requests per second arrive for CRM B for two hours. The worst case for recovery is an outage at the start of the surge: CRM B fails for its first thirty minutes, and the surge continues for ninety minutes after CRM B returns. The portal keeps accepting requests, so the backlog grows to 5 × 1,800 = 9,000 pending handoffs. Decision record ADR-M04 proposed that this backlog must clear within thirty minutes of CRM B's return, while requests keep arriving at 5 per second.

CRM B's contract allows 15 create calls per second. With the drain model from Chapter 2, the net drain is 15 – 5 = 10 per second, and recovery takes 9,000 / 10 = 900 seconds, or fifteen minutes.

A call quota limits attempts, not successes. If 10 percent of attempts fail without creating anything, successful creates run at 13.5 per second, the net drain falls to 8.5 per second, and recovery takes about 1,059 seconds, or seventeen and a half minutes. The objective still holds, with less margin than the simple calculation suggested.

Observe the right quantities

A recovery test should record, for each business unit:

- accepted receipts and distinct confirmed handoffs, counted by submission ID;
- create attempts, searches, throttled calls and unknown outcomes;
- the age of the oldest pending handoff and the size of the manual review queue;
- database wait time and connection use.

An aggregate graph can look healthy while one business unit starves. For example, 100 completed attempts per second across the portal says nothing about whether CRM B's backlog is shrinking. The test should stop CRM B at the peak, restart workers during recovery and inject duplicate deliveries. It passes only if every accepted receipt ends in `confirmed` or in a visible `manual_review` item.

The change card

After the first design review, Meridian receives this change card.

Field	Content
Card	CC-M1, CRM B quota change
New fact	From next month, CRM B allows 8 calls per second per customer account, shared by creates, searches and status reads
Source	Vendor notice, confirmed by the vendor's account manager
Assumptions affected	A-05: "CRM B allows 15 creates per second, with searches counted separately"
Decisions affected	ADR-M04, the thirty-minute recovery objective
Question	How should Meridian respond, and what does each response change for customers?

The portal polls CRM B for status changes so that customers see updates within a minute, which uses about 2 calls per second and leaves 6 for creates. Even if every create succeeds, the net drain is now 6 – 5 = 1 per second. By the end of the surge, at minute 120, 5,400 handoffs have cleared and 3,600 remain; with no new arrivals, 6 creates per second clear them in ten more minutes, so recovery takes 100

minutes. With 10 percent failed attempts, the net drain is 0.4 per second, 6,840 handoffs remain when the surge ends, and recovery takes about 111 minutes, until minute 141. Only the end of the surge stops the backlog from lasting far longer. Adding workers cannot help, because the quota limits the whole account; more workers would only waste quota on throttled calls.

To meet thirty minutes, successful creates must reach 5 + 9,000 / 1,800 = 10 per second. With 10 percent failures that needs about 11.1 attempts per second, and with status polling about 13.1 calls per second, so the account needs a quota of at least 14. The calculation turns a compute question into a contract question.

Revise the recommendation

Every response changes a business or technical contract. Silently dropping accepted requests is never one of them, because it breaks the promise made by the receipt. The credible options, all assuming 10 percent failed attempts, are:

Option	Calls per second	Recovery after CRM B returns	Annual cost	Effect on customers
1. Buy a permanent quota of 16	16	About 20 minutes with polling on; about 16 with polling paused	€14,400	Thirty-minute objective kept
2. Keep 8, pause status polling during recovery	8	About 68 minutes with polling paused	None	Objective becomes 75 minutes; B’s statuses stay “Received, awaiting service team” for longer
3. Temporary uplift to 16 during declared recalls	16 while a recall is declared	About 20 minutes with polling on; about 16 with polling paused	About €1,200 at €100 a day, if switch-on within an hour is confirmed	Objective kept during recalls

Option 1 leaves 16 – 2 = 14 calls for creates, of which 12.6 succeed per second, so the backlog clears in about 9,000 / (12.6 – 5) ≈ 1,184 seconds. Option 2 has 7.2 successful creates per second and needs about 9,000 / (7.2 – 5) ≈ 4,091 seconds. Options 1 and 3 improve further if status polling pauses while more than 1,000 handoffs are pending. All 16 calls then go to creates and 14.4 succeed per second. The backlog clears in about 8,000 / 9.4 + 1,000 / 7.6 ≈ 983 seconds, or about 16 minutes.

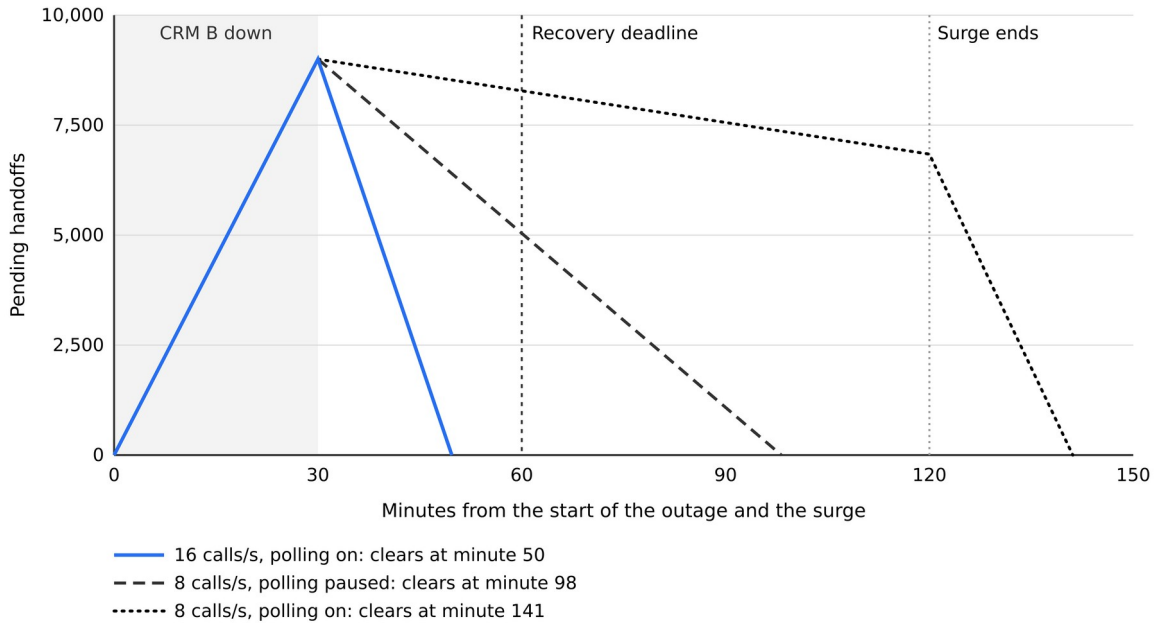


Figure 6 Backlog of CRM B handoffs when CRM B fails for the first thirty minutes of a two-hour recall surge, with 5 new requests per second until minute 120 and 10 percent of attempts failing. The legend states whether status polling is on.

Figure 6 plots three ways of using the quota. Only 16 calls per second clears the backlog before the deadline at minute 60, at about minute 50; pausing polling as well brings that forward to about minute 46. With 8 calls per second and polling on, the backlog lasts until about minute 141, kept alive by the surge until minute 120.

Two controls apply to every option. The rate limiter must match the scope of the quota: a limiter inside each worker replica multiplies the permitted rate by the number of replicas, so Meridian needs one limiter shared across the account. And the quota must be shared fairly: a strict priority for recall requests could starve ordinary CRM B requests indefinitely, while a weighted share, for example 90 percent for recall work with unused capacity lent back, keeps both moving.

Count the costs on the same basis

The vendor’s price list offers two ways to buy capacity. A permanent upgrade to 16 calls per second costs €1,200 a month, or €14,400 a year. A temporary uplift costs €100 a day while a recall is declared. The risk register expects up to two declared recalls a year, each lasting about six days, so the temporary uplift would cost about 2 × 6 × €100 = €1,200 a year. The account manager has not yet confirmed that the uplift can be switched on within an hour of a declaration, which is what the recovery promise needs.

The support team estimates that a recall coinciding with a CRM B outage produces about 400 extra “where is my request” calls. At six minutes each, that is 40 agent hours, or €1,800 at an internal rate of €45 per hour, and the risk register expects one such event a year. Support savings of €1,800 a year do not justify €14,400 a year, so the case for buying capacity rests on the safety promise: customers with potentially unsafe equipment expect the service team to see their request quickly. That judgement belongs to the sponsor; the architect’s job is to show the trade clearly, including the cheaper option 3 and the assumption it depends on.

The worked Meridian dossier

The following dossier is one sound answer to the studio. Compare it with your own first submission.

Problem brief

Meridian’s three business units each run their own CRM, with three different contracts for creating cases. Customers need one portal that accepts service requests reliably, including during a product recall, and shows each request’s true status. The first release must deliver every accepted request to the right CRM, with no losses and no unmanaged duplicates.

- **Sponsor:** head of service operations.
- **Stakeholders:** the three business unit managers, the portal team, the three CRM vendors and the service agents.
- **In scope:** submission, receipt, handoff to CRMs A, B and C, and the customer-visible status.
- **Out of scope:** replacing any CRM, and changes to how the work-order integration creates ERP work orders.
- **Open questions:** CRM B’s index lag, the price of a temporary quota uplift, and agent time for manual review during recalls.

Quality scenarios

ID	Stimulus and environment	Required response	Measure
Q1	20 valid submissions per second for ten minutes	Receipts returned	95 percent within one second, excluding upload
Q2	Normal operation over a month	Eligible submissions receive receipts	99.9 percent success
Q3	CRM B unavailable for thirty minutes while 5 requests per second arrive	Submissions still accepted	No accepted request lost; every receipt has a handoff record
Q4	CRM B returns after Q3	Backlog cleared	Within thirty minutes of CRM B's return (ADR-M05)
Q5	A CRM changes a case status	Portal shows the new status	Within one minute for 99 percent of changes, outside recovery periods
Q6	A staff member without tenant T042 in scope requests a T042 submission	Request refused	No disclosure in the release access test matrix
Q7	A client repeats a submission with the same key	Same receipt returned	No second submission; no second case in CRM A or CRM C
Q8	CRM B takes 20 seconds per call	CRM A and C handoffs unaffected	Oldest pending A and C handoff under one minute

Fact ownership

Fact	Owner	Portal's treatment	Conflict rule
Submission and receipt	Portal	Durable record with the idempotency result	Same key and body return the same receipt
Handoff state	Portal	One five-state record per submission	Changed only by the lease owner or a guarded review
Business unit for a tenant	ERP service contract	Copied at acceptance	Later changes affect only new submissions
Service case and status	The business unit's CRM, after confirmed handoff	Projection with the CRM's version	Older versions ignored
Customer-visible status	Portal, derived	"Received" until confirmed, then the CRM's status	Recomputed from handoff state and projection
Assets, contracts and entitlements	ERP	Cached read model labelled with source version and freshness	ERP wins; stale entries flagged
Work orders	ERP	Read-only reference	Created only by the work-order integration after a `ServiceCaseApproved` event
Staff scope	Identity administration	Role, business unit and set of tenants	Checked on every request

Decision record

ADR-M05: Respond to CRM B's shared quota. Status: accepted. Supersedes the capacity assumption in ADR-M04.

- **Context.** CRM B now allows 8 calls per second per account for all call types (change card CC-M1), and status polling uses about 2 of them. With 10 percent failed attempts, recovery from a thirty-minute outage during a recall would take about 68 minutes with status polling paused and about 111 minutes with polling on, because the two-hour surge keeps adding requests until minute 120. The objective is thirty minutes (Q4).

- **Decision.** Buy the temporary uplift to 16 calls per second for declared recalls, at the listed €100 a day, provided the vendor confirms within four weeks that it can be switched on within an hour of a declaration. If the vendor cannot confirm, buy the permanent upgrade to 16 calls per second at €1,200 a month, and cancel it if the temporary uplift is confirmed later. In all cases, run one account-wide rate limiter, and pause CRM B status polling while more than 1,000 CRM B handoffs are pending.
- **Alternatives.** The permanent upgrade alone (objective kept, €14,400 a year); no upgrade with a 75-minute objective (no cost, slower status); the vendor's nightly bulk import (rejected as far too slow).
- **Consequences.** With 16 calls per second and polling paused, the backlog clears about 16 minutes after CRM B returns. During that time, CRM B customers with unconfirmed requests see "Received, awaiting service team", and confirmed CRM B cases show their last known status. Recall declarations become a vendor dependency. No accepted request is lost under any option.
- **Evidence.** The drain calculations in this chapter; recovery test X-07 at 8 and 16 calls per second; vendor notice CC-M1.
- **Review triggers.** The vendor changes the quota again; a forecast recall exceeds 5 requests per second; status polling exceeds 2 calls per second; or two years pass without a CRM B outage during a recall, which would reopen the cost question.

Decision brief

Decision needed by 30 April from the head of service operations: how to keep the recall recovery promise after CRM B's quota cut.

What changed. CRM B cut its call allowance from 15 to 8 per second. If CRM B fails for thirty minutes at the start of a recall, requests could now wait almost two hours to reach the service team, instead of under thirty minutes.

What stays the same. In every option, the portal accepts every request, loses none, and issues a receipt.

Options.

1. Buy permanent extra capacity: €14,400 a year, recovery in about 16 minutes with status polling paused.
2. Buy nothing and change the promise to 75 minutes: no cost, but status updates for business unit B pause during recovery.
3. Buy extra capacity only during declared recalls: €100 a day, about €1,200 a year, recovery in about 16 minutes with status polling paused, if the vendor confirms that it can be switched on within an hour.

Recommendation. Option 3, with option 1 as the fallback if the vendor cannot confirm the one-hour switch-on within four weeks. Support savings alone, about €1,800 a year, do not pay for option 1; the case rests on how quickly recall requests must reach the service team.

Risks and conditions. The one-hour switch-on for option 3 is unconfirmed, and a recall larger than 5 requests per second, or longer than two hours, or an outage longer than thirty minutes, would need a new calculation.

> **In practice.** Write the decision brief last, for someone who reads nothing else. The sponsor should be able to decide from this page alone, and engineers should find every number on it traced in the dossier.

Studio assessment

Submit a revised dossier containing the earlier and the changed assumptions, both recovery calculations, the failure test design, the ADR that responds to the quota, and your abuse case and service level objective with their audit record and alert. Include one research critique of 400 to 600 words. A good choice is the end-to-end argument of Saltzer, Reed and Clark: some functions, such as reliable delivery, can be implemented completely only at the endpoints, and lower-level mechanisms serve mainly as performance enhancements [18]. Explain which of Meridian's guarantees sit at the endpoint, which rely on CRM contracts and which the portal controls locally.

Use the rubric in Chapter 16. An answer that changes the thirty-minute objective with the sponsor's agreement can score higher than one that keeps the number by assuming unlimited vendor capacity.

Exercises

5.1 During the recall surge, CRM B fails for 45 minutes instead of thirty. Calculate the backlog when it returns, the minimum successful handoff rate that clears it within thirty minutes while 5 new handoffs arrive each second, and the call quota needed if 15 percent of attempts fail and status polling uses 2 calls per second. Is the 16-call quota enough with polling on, and with polling paused?

5.2 A dashboard reports 100 completed attempts per second across all business units during recovery. Give three reasons why this may not show successful recovery.

5.3 Propose a response to the 8-call quota that keeps every accepted request. Explain its effect on customer expectations and on support work.

5.4 The vendor offers a quota of 12 calls per second. With 2 calls per second of status polling and 10 percent failed attempts, calculate the recovery time for the 9,000 backlog and state whether it meets the thirty-minute objective.

5.5 Using the end-to-end argument of Saltzer, Reed and Clark, explain why the portal’s receipt cannot by itself guarantee that a service agent sees the request. Identify the check that belongs at the endpoint.

Key points

- Recovery depends on net successful capacity: successful completions minus new arrivals, not worker count.
- A quota counts attempts and every call type, so failures and status polling consume recovery capacity.
- A changed external limit can require a changed business promise, and the dossier must show that dependency.
- Economic comparisons need explicit units, a stated basis for every price and a plain statement of what the money buys.
- A complete dossier links scenarios, ownership, decisions and evidence, and ends with a one-page brief for the decision maker.

CHAPTER 6

Transaction studio brief and business invariants

Learning objectives

After this chapter you should be able to:

- Define transaction correctness as business invariants, and state when the message “Your order is confirmed” is true.
- Design a reservation request that returns the same result however often it is repeated.
- Calculate the utilisation of the hottest record and its throughput ceiling from the lock-hold time.
- Explain how a conditional stock update behaves under PostgreSQL’s READ COMMITTED, REPEATABLE READ and SERIALIZABLE isolation levels.
- Compare a modular single-database design with separate inventory and order services.

Aster launches a limited-stock event

> **Building on Book 1.** Book 1 introduced Aster at an everyday peak of 50 checkout attempts per second, protected its stock with one conditional update and handled uncertain payments with a saga. This studio raises the peak tenfold for a limited-stock event, so the questions become how busy one stock row can get, which isolation level to use, how to admit customers fairly and how to share stock with a partner.

Aster Retail sells physical goods through its website. Its everyday traffic is modest, but this studio sets a harder problem, a limited-stock sales event for a heavily promoted product:

- a peak of 500 checkout attempts per second for ten minutes, with 70 percent of attempts aimed at the promoted product;
- 2,000 units of that product allocated to the online event, and no confirmed sale beyond the allocation;
- a checkout response target: 95 percent of checkout submissions receive a final answer or an explicit pending status within five seconds;
- a payment provider that accepts an operation ID, offers an outcome lookup and honours repeated requests with the same ID for 24 hours;
- payment authorisation that usually takes a few seconds but can take up to thirty, with a timeout leaving the outcome unknown;
- reservations that expire after five minutes unless the order is confirmed.

Aster’s two warehouses keep their own physical stock records. The 2,000 units are a selling allowance agreed with them for the event, so the event system controls the allowance, not the shelves.

The team’s first proposal draws six services: catalogue, cart, order, inventory, payment and notification. The product manager wants one simple message at the end of checkout: “Your order is confirmed.” Before accepting the diagram, define what must be true for that message to be correct.

Derive the invariants

An **invariant** is a condition that must hold after every transaction. Let A be the units still available, R the units held by active reservations, C the units committed to confirmed orders and L the allocation of 2,000. With no replenishment during the event, A + R + C = L, with each quantity at or above zero. A reservation moves one unit from A to R; confirmation moves it from R to C; release or expiry moves it back from R to A.

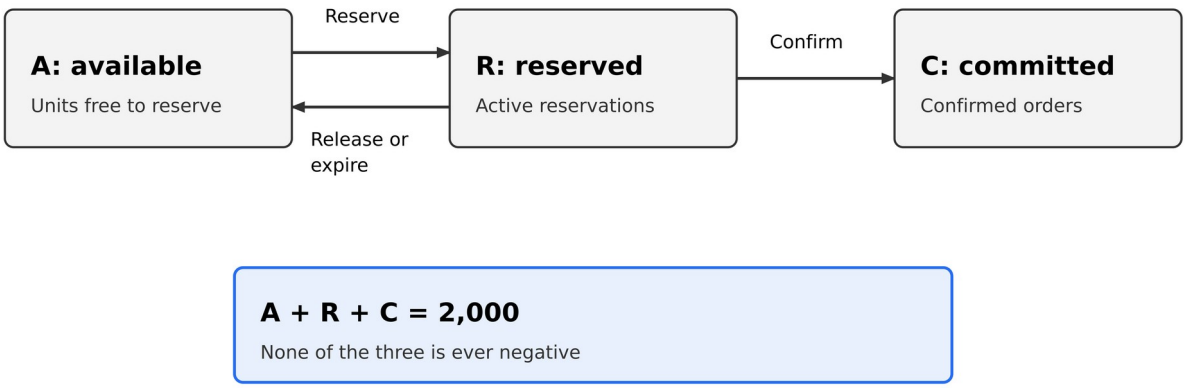


Figure 7 Reservation, confirmation and release move units between three quantities whose sum stays fixed at the event allocation.

Figure 7 shows these movements. Aster's invariants for the event are:

- **I1.** A + R + C = 2,000, and A, R and C are never negative.
- **I2.** Each reservation is in exactly one state: active, committed, released or expired.
- **I3.** A reservation operation changes the stock at most once, however often it is repeated.
- **I4.** An order is confirmed only when its reservation is committed and its payment authorisation is verified.
- **I5.** Every admitted checkout ends confirmed, rejected, or in a visible pending state with an owner.

I4 answers the product manager. "Your order is confirmed" is true only when I4 holds; before that, the correct message is "We have your order and are confirming payment."

Find the contention point

Seventy percent of 500 attempts per second, 350 per second, target the single allocation row of the promoted product. For example, spreading the catalogue across ten database servers leaves that row exactly as busy as before, so contention must be analysed at the invariant itself.

The reservation statement

The following PostgreSQL schema keeps the available count on one row per product, and one row per reservation:

```
CREATE TABLE stock (
  sku       text PRIMARY KEY,
  available integer NOT NULL CHECK (available >= 0)
);

CREATE TABLE reservation (
  reservation_id uuid PRIMARY KEY,         -- the client's operation identity
  order_id       uuid NOT NULL,
  sku            text NOT NULL REFERENCES stock,
  state          text NOT NULL CHECK (state IN
                 ('active', 'committed', 'released', 'expired')),
  expires_at     timestamptz NOT NULL
);
```

Here A is the `available` column, while R and C are the counts of active and committed reservations. One statement decrements the stock and records the reservation:

```
-- Step 1: reserve. A returned row is a new reservation.
WITH taken AS (
  UPDATE stock SET available = available - 1
  WHERE sku = $1 AND available >= 1
  RETURNING sku
)
INSERT INTO reservation (reservation_id, order_id, sku, state, expires_at)
SELECT $2, $3, sku, 'active', now() + interval '5 minutes' FROM taken
RETURNING reservation_id, state, expires_at;
```

The heart of it is the conditional update `UPDATE stock SET available = available - 1 WHERE sku = $1 AND available >= 1`. When a unit remains, the statement takes it and returns the new reservation. When none remains, nothing changes and the statement returns no row.

Make a repeated request return the same answer

A client that loses the reply resends its request with the same `reservation_id`. The first statement alone cannot answer that repeat correctly, and the failure appears exactly at sell-out. Suppose a customer reserves the last unit and the reply is lost. The retry finds `available` at zero, so the update matches no row, the insert adds nothing, and the customer who holds the unit is told “sold out”. While units remain, the retry fails differently: the update takes a second unit, the insert fails with a unique-key violation, SQLSTATE 23505, and the whole statement rolls back, including the decrement.

Both outcomes therefore need a second step, a lookup by the same identifier:

```
-- Step 2: only after no row or SQLSTATE 23505, look up the same ID.
SELECT order_id, sku, state, expires_at
FROM reservation
WHERE reservation_id = $2;
```

The service applies one rule:

1. If step 1 returns a row, answer with the new reservation.
2. If step 1 returns no row or fails with 23505, run step 2.
3. If step 2 finds the reservation with the same order and product, answer with it. If it finds the identifier with a different order or product, reject the request as a reused identifier.
4. If step 2 finds nothing, answer “sold out”. This answer changes no data, so a later retry with the same identifier is simply a new attempt.

Step 2 always sees the earlier reservation when there is one. A concurrent duplicate waits for the first transaction’s row lock or unique-index entry, and that wait ends only when the first transaction commits or aborts. If it commits, step 2 is a new statement, so under READ COMMITTED it reads the committed row; if it aborts, the duplicate simply proceeds as a new reservation. Run against PostgreSQL 18, the rule gave these results:

Situation	Step 1	Answer	Stock afterwards
Retry of the winning request after sell-out	No row	The existing reservation	0, one reservation
Retry while units remain	23505, rolled back	The existing reservation	Unchanged
Two copies at once for the last unit	Second waits, then no row	The existing reservation	0, one reservation
Another customer at once for the last unit	Second waits, then no row	Sold out	0, one reservation
Same identifier sent for a different product	23505, rolled back	Rejected	Unchanged

Invariant I3 holds in every case: a reservation changes the stock at most once, however often it is repeated.

What the isolation level changes

Suppose two transactions, T1 and T2, reserve the same product at the same moment. T1 updates the row first and holds its row lock until it commits. What T2 does next depends on the isolation level [19]:

- **READ COMMITTED**, PostgreSQL’s default: T2 waits for T1 to commit, then re-evaluates its `WHERE` clause against the newest version of the row. If a unit is still available, T2 takes it; if not, T2 updates nothing. There is no error and no retry, and `available >= 1` alone prevents overselling.
- **REPEATABLE READ** or **SERIALIZABLE**: T2 waits for T1 to finish. If T1 commits, T2 fails with “could not serialize access due to concurrent update”, a serialization failure with SQLSTATE 40001, and the application must retry the whole transaction [20]. Only if T1 rolls back does T2 continue.

The second behaviour surprises many teams. With two units in stock, T2 fails under both stricter levels even though a unit is still available.

How often would that happen? The answer comes from the row’s **utilisation**: the fraction of time its lock is held, equal to the arrival rate multiplied by the lock-hold time. At 350 reservations per second and a lock held for 1 millisecond, the row is locked 350 × 0.001 = 0.35 of the time. With random arrivals, an attempt finds the row locked with that same probability, so under REPEATABLE READ about one attempt in three would wait and then fail. If each failed attempt retries until it succeeds, the database handles about 350 / (1 − 0.35) ≈ 540 attempts per second to complete 350 reservations, and every failed attempt holds a connection while it waits. Each retry also adds waiting, and when the retries pile up faster than the row

clears them, the result is a **retry storm**. For this statement, READ COMMITTED is both correct and much cheaper.

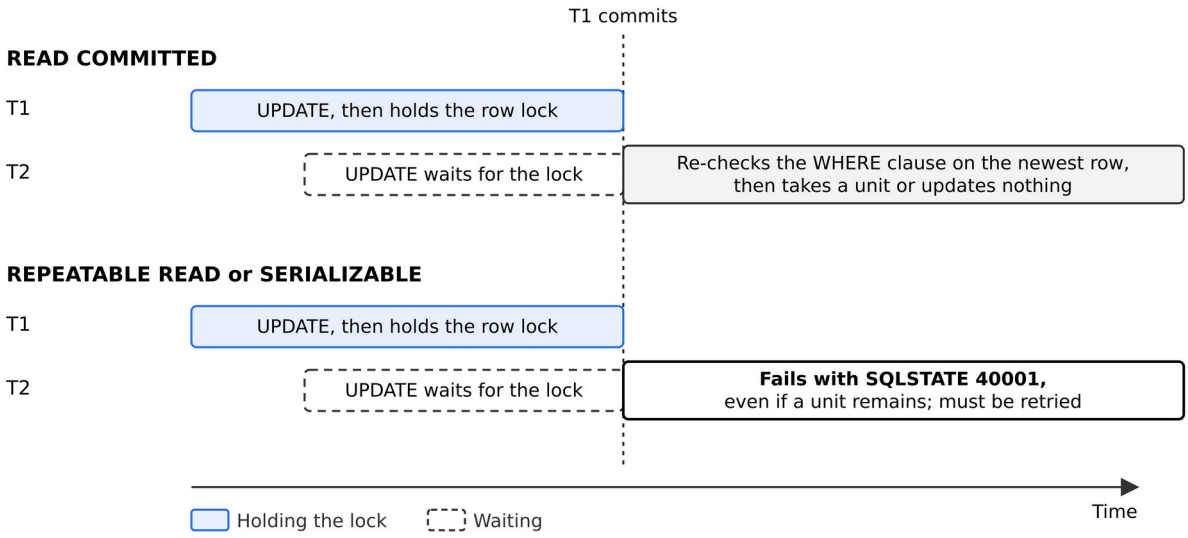


Figure 8 Two transactions update the same stock row: under READ COMMITTED the second waits and re-checks, while under SERIALIZABLE it waits and then fails with SQLSTATE 40001.

Figure 8 compares the two timelines. In both, T2 holds a database connection while it waits.

Estimate the hot-row ceiling

A row lock is held from the moment the update runs until the transaction commits. If that **lock-hold time** is h seconds, the row can serve at most about 1 / h reservations per second, however many servers sit in front of it, and its utilisation at an arrival rate λ is λ × h. For example:

- sent as the single statement above, in one network round trip, h may be about 1 millisecond, giving a ceiling near 1,000 per second;
- sent as `UPDATE`, `INSERT` and `COMMIT` in three round trips of about 1 millisecond each, plus 1 millisecond to flush the commit, h is about 4 milliseconds and the ceiling falls to about 250 per second.

At 250 per second against 350 offered, the utilisation would be 1.4, which is impossible to sustain: the 2,000 units sell out in 8 seconds while about 100 attempts per second pile up in the lock queue. Roughly 800 transactions are then waiting, each holding a database connection, and if the pool runs dry, checkouts for every other product stall too. Waiting grows sharply well before utilisation reaches 1, so a row run at 0.9 already queues badly. These are estimates; Chapter 8 measures h directly. The design consequences are:

- keep the lock short, with one statement and one round trip;
- cap the waiting, for example at 50 reservation attempts in flight for the hot product, and send the rest a “high demand, please try again” page or a queue;
- once `available` reaches zero, answer requests with a new reservation ID from a cached sold-out flag, answer a repeated ID with the step 2 lookup, which reads the reservation table and never locks the stock row, and clear the flag when an expiry returns a unit;
- leave this statement at READ COMMITTED;
- split the row into allocation buckets if one row cannot keep up, as Chapter 7 develops.

Two first designs

Design A keeps orders and the event allocation in one modular application with one transactional database. The payment call stays external and is coordinated through durable state, and the stock invariant needs no distributed transaction.

Design B gives inventory and orders their own services and databases. A reservation then becomes an explicit contract with a reservation ID, quantity, owner, expiry, version and final outcome. Helland describes this style: each entity lives inside one transactional scope, and entities cooperate through messages that must be processed idempotently [21].

Separate ownership, or an inventory capability shared with stores and marketplaces, can justify that extra protocol. Without such a need, Design B adds a network boundary to every reservation and gains nothing for the event.

Both designs need payment idempotency and reconciliation, bounded admission and a way to explain each customer's outcome after a crash. The choice is about where the stock authority lives and what it costs to operate. On either cloud, Design A runs on the managed PostgreSQL and container services mapped in Chapter 4; Design B would add a queue between the services, such as Azure Service Bus or Amazon SQS [17].

Your first submission

Produce:

1. a state model for orders, reservations and payments;
2. container views for Designs A and B;
3. a sequence for a payment authorisation that arrives after its reservation has expired;
4. the exact customer message at each step of checkout;
5. a capacity hypothesis for the promoted product, with your own estimate of the lock-hold time;
6. an experiment that could reveal overselling even when response times look good.

Use provider tokens in place of card details; the studio is about responsibility and uncertainty, not about handling card data.

Exercises

6.1 Start from A = 2,000, R = 0 and C = 0. Reserve three units, confirm two of them under a policy that allows partial confirmation, and release the third. Show A, R and C after each step and check the invariant.

6.2 Explain why partitioning the catalogue into ten equal groups does not reduce the contention on the promoted product.

6.3 The reservation transaction holds the row lock for 2.5 milliseconds. Calculate the hot-row ceiling, compare it with the 350 attempts per second offered, and calculate how long the 2,000 units last if every attempt succeeds.

6.4 A smaller promotion sends 200 reservation attempts per second to one stock row, and each reservation holds the row lock for 1.5 milliseconds. Calculate the row's utilisation. A developer proposes running the reservation under REPEATABLE READ "to be safe". Estimate the fraction of attempts that would fail with SQLSTATE 40001 and the total attempt rate if every failure retries until it succeeds, and explain why READ COMMITTED is already safe for this statement.

6.5 Define an `OrderConfirmed` event contract that distinguishes verified business completion from the acceptance of a checkout request.

6.6 A customer's reservation request times out on the client. The client resends it with the same `reservation_id` three seconds later, by which time the product has sold out. Trace both steps of the reservation rule for this retry, state the answer the customer receives, and explain what would go wrong if the service answered from the sold-out flag first.

Key points

- Transaction design starts from business invariants, and invariant I4 decides when "confirmed" is true.
- A popular product concentrates load on one row, so contention must be analysed at that row, not across the catalogue.
- Under READ COMMITTED, a conditional update waits, re-checks and never oversells; under REPEATABLE READ or SERIALIZABLE it fails with SQLSTATE 40001 and must be retried.
- A row's utilisation is its arrival rate times its lock-hold time; its throughput is limited to about one divided by the lock-hold time, so short transactions and bounded admission matter more than server count.
- A repeated reservation must find the original result, which needs a lookup by reservation ID whenever the reservation statement returns no row or a unique-key violation.
- Splitting inventory into its own service replaces a local transaction with a protocol that must be designed and operated.

CHAPTER 7

Reservations, payments and coordination

Learning objectives

After this chapter you should be able to:

- Design guarded state transitions so that exactly one of expiry and confirmation wins.
- Evaluate reserve-first and pay-first orderings, including a late authorisation and a failed compensation.
- Calculate the throughput gain and the end-of-sale cost of splitting a hot stock row into allocation buckets.
- Compare orchestration and choreography for the order lifecycle.

Model the order as durable progress

The order coordinator records each order's progress in durable states: checkout received, reserved, payment pending, payment unknown, confirmed, cancellation pending, cancelled and rejected. Every transition names the event that triggers it, the guard that must hold, its durable effect, and the operation ID that makes a repeat harmless.

A payment timeout moves the order to payment unknown, never to "declined", because the provider may have authorised the payment. The coordinator keeps the provider's operation ID and queries the outcome within the 24-hour window. Even a confirmed authorisation still needs a usable reservation: a successful payment cannot restore a unit that has already been released.

Garcia-Molina and Salem introduced **sagas**: a long-lived transaction is split into a sequence of smaller transactions, each paired with a compensating transaction that semantically undoes it if the sequence cannot finish [22]. For Aster, compensation means a business action such as releasing a reservation or voiding a payment authorisation, and a compensation can itself fail or stay pending.

Let exactly one of expiry and confirmation win

In Design A, confirmation and expiry both act on the reservation row in the same PostgreSQL database, and each statement is guarded by the reservation's state:

```
-- Confirm: wins only while the reservation is active and unexpired.
UPDATE reservation SET state = 'committed'
WHERE reservation_id = $1 AND state = 'active' AND expires_at > now()
RETURNING reservation_id;

-- Expire: returns the unit only if this statement wins the same guard.
WITH expired AS (
  UPDATE reservation SET state = 'expired'
  WHERE reservation_id = $1 AND state = 'active' AND expires_at <= now()
  RETURNING sku
)
UPDATE stock SET available = available + 1
FROM expired WHERE stock.sku = expired.sku;
```

If confirmation commits first, the expiry statement finds the reservation no longer active and changes nothing. If expiry commits first, confirmation returns no row, and the order takes the late-authorisation path below. When the statements collide, the second waits for the first's row lock and then re-checks `state = 'active'` against the newest row version, as in Chapter 6. The database server's clock decides expiry, so there is one time authority.

Figure 9 shows the two outcomes. Timestamps alone cannot settle the race, because two processes can both read an unexpired reservation and then act; the guard and the state change must happen together at the reservation's owner. In Design B, the inventory service owns that arbitration, and a timed-out confirm call from the order service must be retried with the same reservation ID or followed by a query. Splitting the services adds an uncertain boundary before payment is even considered.

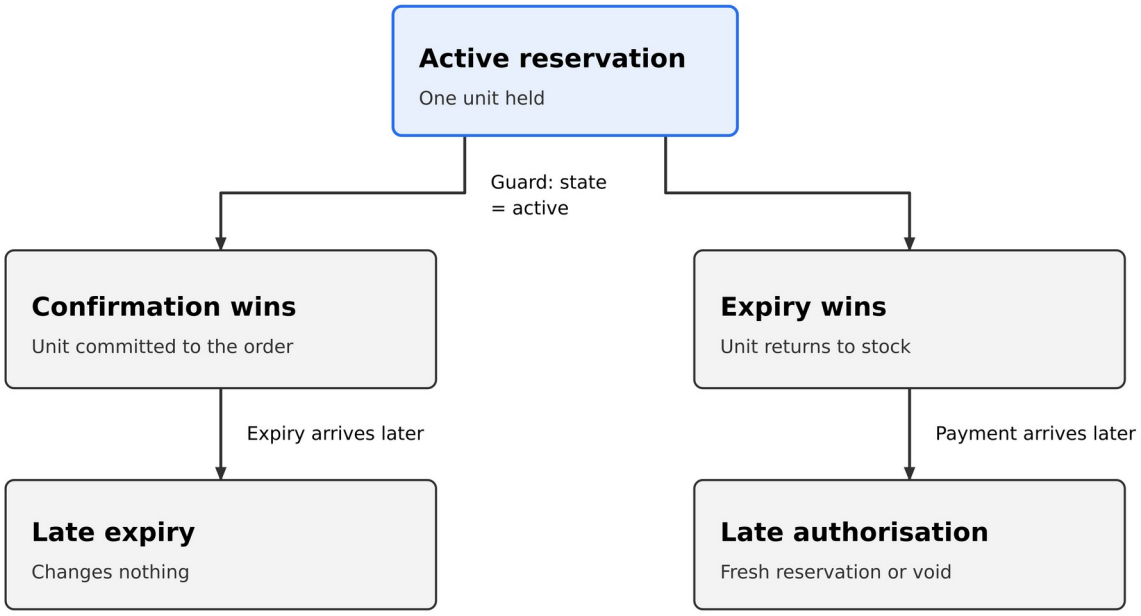


Figure 9 Expiry and confirmation compete for one guarded transition on the same reservation, and only one of them can consume it.

Late authorisation and payment ordering

Aster can reserve stock before taking payment, or authorise payment before reserving stock. Neither ordering removes partial failure; each chooses which intermediate state the business must own.

Question	Reserve first	Authorise first
Is stock held while payment runs?	Yes, for up to five minutes	No
Main risk	Abandoned or automated checkouts hold scarce units	A payment is authorised but no unit remains
Late or failed outcome	Authorisation after expiry needs a fresh reservation or a void	Sold out after authorisation needs a void
Message while waiting	“Reserved, confirming payment”	“Payment authorised, checking stock”

Aster reserves first, so that customers who pay do not find the product gone. The cost is a careful policy for late results. For example, reservation r1 expires at 12:05:00 and its unit returns to stock. At 12:06 the provider reports that the authorisation sent at 12:04:50 succeeded. The coordinator now:

1. tries a fresh reservation, r2, keeping the same payment operation ID;
2. confirms the order with r2 if a unit is available and the event is still open;
3. otherwise requests a void of the authorisation and moves the order to cancellation pending;
4. moves the order to cancelled only when the provider confirms the void.

A **void** cancels an authorisation before the money is captured; a **refund** returns money already captured. Which one applies depends on the payment’s state at the provider. A void request can time out like any other call, and the order then stays in cancellation pending with an owner. The system must never charge again, invent stock, or mark the order cancelled merely because it sent a void.

A stricter policy holds the reservation while the payment outcome is unknown, up to a limit such as ten minutes. That reduces late authorisations but lets slow or malicious checkouts lock scarce stock. Customers need three distinct messages in any case: confirmed, payment pending, and “we are resolving your payment”.

Split the hot row into allocation buckets

Chapter 6 showed that one stock row serves at most about 1 / h reservations per second. One way to raise that ceiling is to divide the allocation across several rows of the same product, called **buckets**. Each

reservation picks a bucket at random, runs the same conditional update on that bucket, and records the bucket so that a release returns the unit to it.

With k buckets and evenly spread load, the ceiling rises to about k / h. For example, with h at 4 milliseconds, one row tops out near 250 reservations per second, while eight buckets of 250 units raise the ceiling to about 2,000 per second. The invariant becomes: the sum of `available` across all buckets, plus R, plus C, equals 2,000. Each bucket's conditional update keeps that bucket at or above zero, so the total never goes negative.

Buckets have costs at the end of the sale:

- **Misses.** When units remain in only some buckets, a random pick often finds an empty one. With 30 units left in 3 of 8 buckets, the first pick misses 5 / 8 = 62.5 percent of the time, so the service must try the other buckets before answering "sold out".
- **Fairness.** Arrival order is not preserved across buckets, so a later customer can win a unit an earlier one missed.
- **Rebalancing.** Moving units between buckets in one database is a single local transaction.

Some retailers instead place allocations in separate regional databases, so that each region can keep selling while the regions are disconnected. Moving units then needs a transfer protocol. If West moves 100 units to East, West debits its allocation into a durable in-transit record with a transfer ID, and East credits the units exactly once using that ID. Conservation reads: available, plus reserved, plus committed, plus in transit, equals 2,000. If East's reply is lost, West must ask East about the transfer ID rather than reclaim the units, and a region restored from an old backup must replay the transfer history before it sells again.

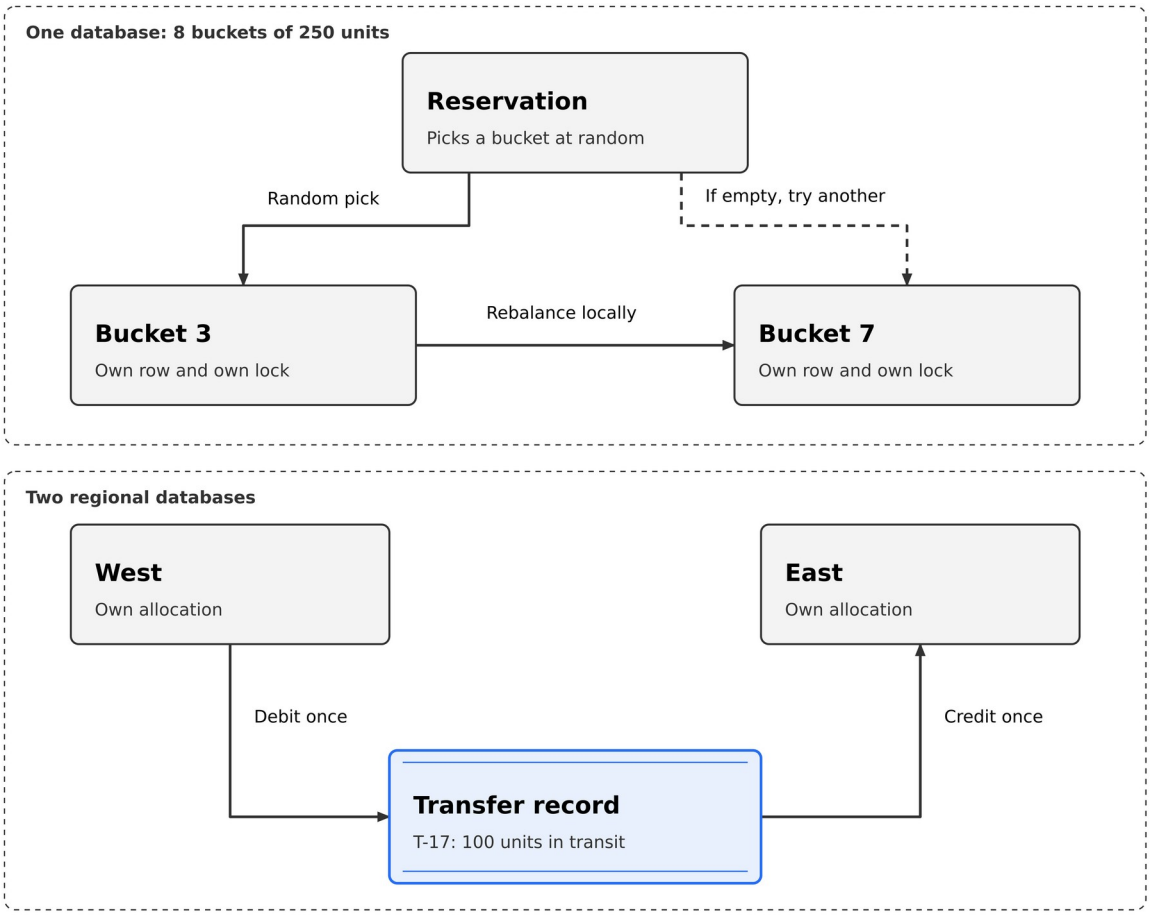


Figure 10 Buckets in one database rebalance with one local transaction, while allocations in two regions need a transfer record that is debited once and credited once.

Figure 10 contrasts the two settings:

Property	One stock row	Buckets in one database	Allocations in two regions
Throughput ceiling	About 1 / h	About k / h	About k / h in each region
Sold-out detection	Exact	Check every bucket	Each region sees only its own units
Moving units	Not needed	One local transaction	Transfer protocol with an in-transit state
Selling while regions are disconnected	No	No	Yes, until local units run out

Buckets are a throughput tool; regional allocations are an availability tool with a much heavier protocol. Aster's event needs throughput, so buckets fit.

Orchestration or choreography

A central coordinator makes each order's progress and exceptions explicit, at the cost of one more component to operate. Choreography, in which services react to each other's events, spreads the logic and makes the lifecycle harder to reconstruct. Either can preserve the invariants when contracts and ownership are clear. For this studio a durable order coordinator is a sound start, because payment, expiry and compensation need a visible process. Notifications can consume `OrderConfirmed` events independently, because a failed email should never reverse a valid order.

> **Check your understanding.** Two proofs are needed. The stock proof shows that guarded transitions cannot spend more units than exist. The workflow argument shows that every admitted order reaches confirmed, cancelled or a visible pending state. Which of this chapter's mechanisms supports each one? A short answer is given at the start of the Chapter 7 answers.

Exercises

7.1 A coordinator opens a database transaction at 12:04:58, checks the order and its payment record, and runs the confirm statement at 12:05:03 for a reservation whose `expires_at` is 12:05:00. The expiry job has not run yet. In PostgreSQL, `now()` returns the start time of the current transaction. Does the confirmation win? State whether invariants I1 and I3 still hold, what rule is broken, and how you would change the design.

7.2 Compare holding a reservation until the payment outcome is known with releasing it at a fixed deadline. Give one business risk and one operational control for each policy.

7.3 Aster splits its 2,000 units into 8 buckets, and the lock-hold time is 4 milliseconds. Calculate the theoretical ceiling. Late in the sale, 40 units remain in 2 of the 8 buckets. What is the probability that a random first pick misses, and what must the service do before answering "sold out"?

7.4 West and East hold separate allocations. West starts a transfer of 100 units to East and loses the reply. Write the conservation equation and the recovery steps, and explain why West cannot simply reclaim the units.

7.5 Evaluate payment-first and reservation-first for this event. Include a late payment callback and a void that times out in both analyses.

Key points

- An order's progress is a durable state machine in which each transition has a trigger, a guard, an effect and an operation ID.
- Guarded updates at the reservation's owner ensure that exactly one of expiry and confirmation wins.
- A late authorisation needs a fresh reservation or a void, and a void that times out stays visible as cancellation pending.
- Buckets raise a hot row's ceiling to about k / h but create end-of-sale misses that the service must handle.
- Allocations in separate regions trade a transfer protocol and harder recovery for the ability to sell while disconnected.

CHAPTER 8

Transaction experiments and failure review

Learning objectives

After this chapter you should be able to:

- Design correctness tests that exercise invariants under adverse orderings before any load test.
- Explain how a small state model exposes missing transition rules, and extend one.
- Evaluate admission policies with customer-outcome measures as well as latency.
- Calculate why stage means add on a serial path while stage percentiles do not.
- Evaluate a change card that adds a second sales channel, and record the response in a decision record and a decision brief.

Test the boundary before the throughput

Start with tiny, exact tests. With one unit in stock, send two concurrent reservations and require exactly one success. Repeat the winning request with the same reservation ID, once while units remain and once after the product has sold out, and require the same answer with no second decrement both times. Release the reservation twice and require the second release to change nothing. Then inject failures at the dangerous points:

- a crash after the provider authorises the payment but before Aster records it;
- a duplicate payment callback;
- expiry before the callback arrives;
- a failure while voiding a payment.

For each, state the expected stock invariant and the customer-visible status in advance. A test passes when it accounts for the allocation, the reservation, the order and the payment; “no exception was thrown” proves very little.

Check histories, not just final counters

A concurrent test produces a **history**: the start and end of each operation, with its result. **Linearizability** gives a precise standard for judging it: each operation must appear to take effect at a single instant between its start and its end, in an order that the object’s sequential specification allows [23]. For a reservation, that specification is short. A new reservation ID takes a unit if one remains. A repeated ID that already holds a reservation returns that reservation, even after the product has sold out. A release returns a held unit at most once.

For example, consider one unit and two overlapping reservations that both return success. No sequential order can explain two successes for one unit, so the history is invalid even if the final counter reads zero. A lost update, where both transactions read 1 and both write 0, produces exactly that picture, so compare the returned reservation IDs and committed obligations as well as the counter. Two identity checks complete the set: the same reservation ID sent twice at once must produce one reservation, and an ID reused with a different product or quantity must be rejected.

A state model for event orderings

A small executable model can expose missing rules before any database is involved. The listing models one unit of stock and one order. Every permitted transition is an explicit rule, any event without a rule raises an error, and each event carries an ID so that a repeated request or a redelivered message has no effect.

```
class IllegalTransition(Exception):
    """An event that the state model does not allow in the current state."""

# (reservation, payment, event) -> (new reservation, new payment, stock change)
RULES = {
    ('none', 'none', 'reserve'): ('active', 'pending', -1),
    ('active', 'pending', 'authorise'): ('active', 'authorised', 0),
    ('active', 'authorised', 'confirm'): ('committed', 'authorised', 0),
    ('active', 'pending', 'expire'): ('expired', 'pending', +1),
    ('committed', 'authorised', 'expire'): ('committed', 'authorised', 0),
    ('expired', 'pending', 'authorise'): ('expired', 'void_requested', 0),
    ('expired', 'void_requested', 'void_confirmed'): ('expired', 'voided', 0),
}

def run(events: list[tuple[str, str]]) -> str:
    reservation, payment, available, seen = 'none', 'none', 1, set()
    for name, event_id in events:
        if event_id in seen:
            continue                    # a repeated request or message changes nothing
        key = (reservation, payment, name)
        if key not in RULES:
            raise IllegalTransition(f'{name} while {reservation}/{payment}')
        reservation, payment, change = RULES[key]
        available += change
        seen.add(event_id)
        held = reservation in ('active', 'committed')
        assert available >= 0 and available + held == 1   # one unit in total
    return f'{reservation}/{payment}, available={available}'

R, A, X = ('reserve', 'r1'), ('authorise', 'p1'), ('expire', 'x1')
C, V, A2 = ('confirm', 'c1'), ('void_confirmed', 'v1'), ('authorise', 'p2')
for title, events in [('confirm before expiry', [R, A, C, X]),
                      ('reservation repeated after the sale', [R, A, C, R]),
                      ('late authorisation after expiry', [R, X, A, V]),
                      ('duplicate callback after void', [R, X, A, V, A]),
                      ('confirm after expiry', [R, X, C]),
                      ('new authorisation after void', [R, X, A, V, A2])]:
    try:
        print(f'{title}: {run(events)}')
    except IllegalTransition as error:
        print(f'{title}: rejected ({error})')
```

The model runs six fixed schedules and prints:

```
confirm before expiry: committed/authorised, available=0
reservation repeated after the sale: committed/authorised, available=0
late authorisation after expiry: expired/voided, available=1
duplicate callback after void: expired/voided, available=1
confirm after expiry: rejected (confirm while expired/pending)
new authorisation after void: rejected (authorise while expired/voided)
```

Each line tests one rule. A late expiry after confirmation changes nothing. A repeated reservation after the unit is sold changes nothing either, because its ID has been seen; in the database, the step 2 lookup from Chapter 6 plays that part and returns the stored reservation. A late authorisation leads to a void, and the unit stays available for other customers. The duplicate callback after the void is ignored because its event ID has been seen. Confirmation after expiry is rejected rather than consuming a released unit. A genuinely new authorisation after the void is also rejected, which raises a question for the business: should a customer who pays again for a cancelled order start a new order?

The model implements the simpler late-authorisation policy, in which a late result always leads to a void; Chapter 7's fresh-reservation path is left as an exercise. With one unit, one thread and no database, it checks the transition table rather than concurrency, which the database tests above cover.

Admission is a business control

When demand exceeds useful capacity, unbounded pending work makes every customer's outcome worse. Aster can reject new checkouts with a clear message, accept them into a bounded pending state, or place customers in a waiting room with a stated admission rule. A fast rejection is not a sale, so

measure the journey with separate indicators: storefront availability, valid checkouts admitted, confirmed orders, payments still unresolved, and time from checkout to a final outcome.

The following results are invented teaching data for the full ten-minute peak of Chapter 6: 500 attempts per second for 600 seconds, or 300,000 attempts. In every row, confirmed orders, unresolved payments and unsold units add up to the 2,000 allocated units.

Policy	Admitted attempts	Confirmed orders	Unresolved payments at cutoff	Units unsold at cutoff	p95 time to final or pending answer
Unbounded admission	300,000	1,942	58	0	7,900 ms
Bounded: 50 in flight for the hot product	24,000	1,988	12	0	620 ms
Strict: close admission for good once every unit is reserved	2,400	1,981	4	15	410 ms

Unbounded admission misses the five-second target and leaves the most payments unresolved, because the provider was flooded with slow requests. Strict admission answers fastest but leaves 15 units unsold: admission closed for good once every unit was reserved, so when some of those reservations later expired, nobody was admitted to take the returned units. Bounded admission sells the most.

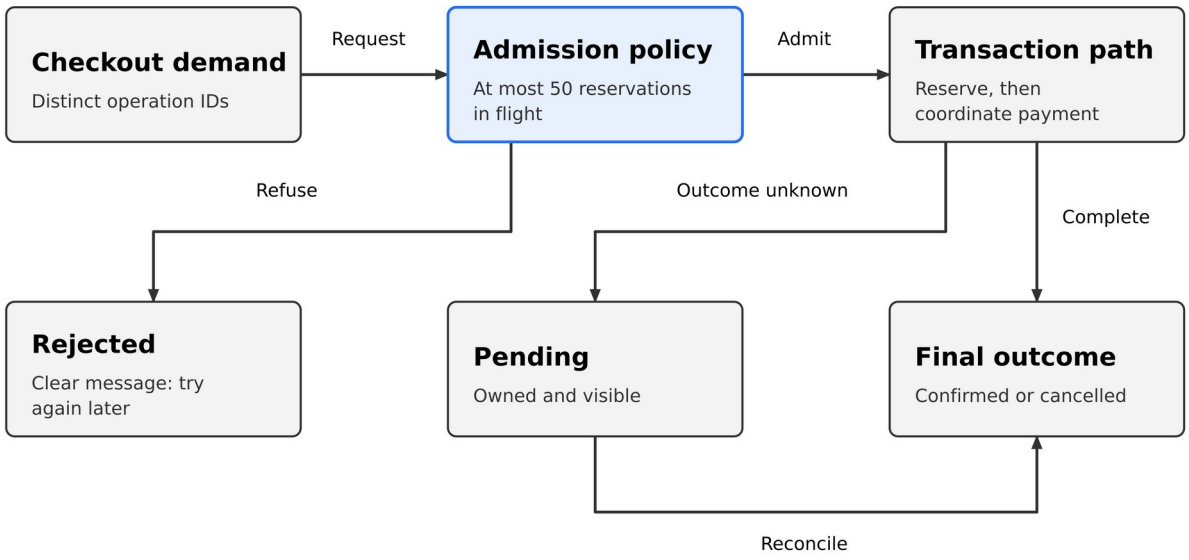


Figure 11 Admission control protects the transaction path while keeping rejected and pending outcomes explicit.

The table alone cannot select a winner. Reviewers should ask whether admitted attempts include duplicates, how many distinct customers they represent, what rejected customers did next, and how many unresolved payments completed after the cutoff. Figure 11 shows the paths that each count must cover. The next experiment should investigate the 15 unsold units, for example by reopening admission whenever a unit returns, and the 58 unresolved payments; repeating the benchmark on larger hardware would answer neither question.

Tail latency on a serial path

Dean and Barroso explain why rare slow responses dominate when one request depends on many components [24], so Aster keeps optional catalogue enrichment and notifications off the confirmation path.

For a serial path, the end-to-end time is the sum of the stage times. Means therefore add exactly: by linearity of expectation, the mean of a sum equals the sum of the means, whether or not the stages are correlated. Three stages with means of 50, 100 and 200 milliseconds give an end-to-end mean of 350 milliseconds.

Percentiles do not add. Suppose each of two independent stages takes 10 milliseconds with probability 0.96 and 1,000 milliseconds with probability 0.04. Each stage’s p95 is 10 milliseconds, so the sum of the

p95 values is 20 milliseconds. But both stages are fast together only with probability 0.96 × 0.96 = 0.9216, which is below 0.95, so the end-to-end p95 is 1,010 milliseconds. The error can also run the other way: if each stage is slow with probability 0.10, each p95 is 1,000 milliseconds and their sum is 2,000, while the end-to-end p95 is 1,010. Measure the whole journey, and use stage timings to diagnose it.

Timeouts follow the customer promise. A single payment authorisation can take thirty seconds, longer than the five-second checkout target, so after about four seconds the checkout returns “payment pending” and the coordinator continues in the background. Retrying the authorisation inside the request could never fit within five seconds.

> **In practice.** Set each timeout by working backwards from the customer promise and the recovery design. A library’s default timeout reflects nobody’s business requirement.

Measure the lock-hold time

Chapter 6 estimated the lock-hold time h at about 1 millisecond for the single-statement reservation. The hot-row experiment measured it. Run alone, with no competing clients, the reservation statement took a mean of 1.4 milliseconds on the database server from the start of the update to the end of the commit, with a p99 of 3.0 milliseconds. Running alone matters, because under contention the measured time also includes waiting for the lock. With h = 1.4 milliseconds, the row’s ceiling is about 1 / 0.0014 ≈ 714 reservations per second, and at 350 attempts per second its utilisation is 350 × 0.0014 = 0.49. One row is enough for Aster’s own checkout, and buckets are not yet needed.

A change card: a marketplace partner joins

Four weeks before the event, Aster receives this change card.

Field	Content
Card	CC-A1, marketplace partner joins the event
New fact	The supplier raises the event allocation from 2,000 to 3,000 units if Aster’s marketplace partner also sells the product. The partner’s checkout expects to reserve up to 150 units per second at the peak and retries a request that gets no reply within 2 seconds
Source	Supplier agreement, confirmed by the partner’s integration lead
Assumptions affected	A-A1: “only Aster’s checkout reserves event stock”; A-A2: “the stock row receives at most 350 attempts per second”
Decisions affected	ADR-A01, one stock row at READ COMMITTED; ADR-A02, bounded admission with 50 reservations in flight
Question	How should the partner draw on the stock, and what does each choice change for customers?

Three responses keep every unit sold at most once:

1. **Share the stock row through Aster’s reservation API.** The partner reserves each unit with its own reservation ID. Sold-out is exact across both channels and nothing is stranded. The row’s utilisation rises to 500 × 0.0014 = 0.70, which calls for two buckets (0.35 each). The partner’s checkout depends on Aster’s API at the busiest moment of the year, and its retries must reuse the reservation ID, because a retry with a new ID after a lost reply takes a second unit.
2. **Split the allocation: 2,000 units for Aster and 1,000 for the partner.** Nothing changes at Aster’s row, which stays at a utilisation of 0.49, and there is no runtime coupling. If the partner’s demand fades, or many of its reservations expire unpaid, its unsold units stay idle while Aster’s customers are told “sold out”; moving them later needs a transfer protocol anyway.
3. **Allocate to the partner in blocks of 100 units.** The partner requests a block when its own stock runs low, and each block moves through the transfer protocol from Chapter 7. Aster debits its allocation into an in-transit record with a transfer ID, the partner credits the block once, and a lost reply is settled by asking about the transfer ID. About ten transfers cover the event, so Aster’s row barely notices them. The partner holds at most two unsold blocks at a time, and returns unsold units through the same protocol when its sale closes.

Option	Aster's row utilisation	Units that can be stranded	Runtime coupling	Aster engineering effort
1. Shared row	0.70, or 0.35 per bucket with two	None	Partner calls Aster for every unit	About 20 days
2. Fixed split	0.49	Up to the partner's whole unsold share	None	About 3 days
3. Blocks of 100	0.49	At most 200 held by the partner	About ten transfers per event	About 12 days

Aster's invariant gains a term in options 2 and 3. Seen from Aster, available plus reserved plus committed plus in transit plus units held by the partner equals 3,000; the partner keeps the same equation for its own units.

Decision record

ADR-A03: Share event stock with the marketplace partner in blocks. Status: accepted. Amends ADR-A01 and ADR-A02.

- **Context.** CC-A1 raises the allocation to 3,000 units on condition that the partner also sells. The measured lock-hold time is 1.4 milliseconds, so Aster's own 350 attempts per second use 0.49 of the row.
- **Decision.** Allocate to the partner in blocks of 100 units through a transfer API with transfer IDs and an in-transit state. The partner may hold at most two unsold blocks and returns unsold units at its close. Aster's checkout keeps one stock row at READ COMMITTED and bounded admission.
- **Alternatives.** A shared row with per-unit reservations, rejected because it couples the partner's checkout to Aster's peak and needs buckets; a fixed split, kept as the fallback.
- **Consequences.** At most 200 units can be held unsold by the partner at any moment. Each event ends with a reconciliation of transfers and returns. The partner owns its customers' payments and confirmations.
- **Evidence.** The utilisation figures in this chapter; joint test X-A5, in which the reply to a block transfer is lost and the partner retries with the same transfer ID.
- **Review triggers.** The partner asks for per-unit reservations; measured h exceeds 2 milliseconds, which would put Aster's own row above 0.7; a second partner joins; or more than 100 units remain stranded at a close.

Decision brief

Decision needed two weeks before the event, from Aster's head of e-commerce: how to share the event stock with the marketplace partner.

What changed. The supplier adds 1,000 units if the partner also sells. The partner's checkout expects to reserve up to 150 units per second at the peak.

What stays the same. In every option, no unit is sold twice, and Aster's own checkout keeps its admission rule and customer messages.

Options.

1. Let the partner reserve every unit through Aster's systems: no unsold units, about 20 days of work, and the partner's sale depends on Aster's systems at the peak.
2. Give the partner a fixed 1,000 units: about 3 days of work, but units can sit unsold on one channel while the other has sold out.
3. Give the partner blocks of 100 units on request: about 12 days of work, and at most 200 units can sit with the partner unsold.

Recommendation. Option 3, with option 2 as the fallback if the partner cannot pass the joint lost-reply test four weeks before the event.

Risks and conditions. The partner must implement transfer IDs correctly; a partner retry with a new transfer ID would move a second block.

Studio assessment

Submit a transaction dossier with the invariants, the reservation rule and its tests, the state model, the admission experiment, the lock-hold measurement and your response to CC-A1. One sound worked

conclusion keeps the order and allocation in one local transaction (ADR-A01) and adds buckets only if the measured lock-hold time requires them. It coordinates payment through durable states, applies bounded admission with a reviewed pending message (ADR-A02), and shares stock with the partner in blocks (ADR-A03). It postpones a separate inventory service until a multichannel ownership need or a measured contention limit justifies the extra protocol.

A distributed alternative can also pass review if it specifies reservation identity, expiry arbitration, duplicate-safe confirmation, payment reconciliation and observability across every boundary. Reviewers should challenge unsupported guarantees and unexplained operating effort, not mark a design down for differing from the worked one.

Exercises

8.1 Explain why a duplicate-callback test needs assertions about stock and payment state rather than only about the HTTP response.

8.2 Using the synthetic report, explain why p95 latency alone cannot establish which policy gives the best customer outcome. Name three further measurements.

8.3 Design a failure experiment for a payment authorisation that arrives after its reservation has expired. Include the expected compensation state and one observation that would fail the design.

8.4 With two units in stock, a concurrency test records three overlapping operations: reserve r1 returns success, a repeat of r1 sent at the same moment also returns success, and reserve r2 returns “sold out”. Afterwards `available` is 0 and the reservation table holds one row. Is the history valid under the sequential specification? Which invariant fails, and what implementation defect would produce this result?

8.5 Three serial stages have mean times of 40, 60 and 150 milliseconds. Calculate the end-to-end mean. Then take two independent stages that each take 20 milliseconds with probability 0.97 and 800 milliseconds with probability 0.03. Calculate each stage’s p95, the sum of those p95 values, and the end-to-end p95.

8.6 Extend the state model so that a late authorisation first tries a fresh reservation, and voids the payment only if no unit is available. Write the new rules and two schedules that test them.

8.7 The partner’s forecast rises to 300 reservations per second. For option 1 of CC-A1, calculate the single row’s utilisation with the measured lock-hold time, and the smallest number of buckets that keeps each bucket’s utilisation below 0.4. What does the extra bucket cost at the end of the sale?

8.8 During testing, the partner’s client retries a block transfer with a new transfer ID whenever a call times out. Explain what can go wrong, and write the contract term and the test that prevent it.

Key points

- Small, exact tests with one unit expose concurrency and identity defects before load testing begins.
- A history is valid only if some sequential order explains every result, so check reservation IDs as well as final counters.
- An explicit transition table turns missing rules into visible errors instead of silent behaviour.
- Admission policy is a business decision, measured by confirmed orders, unresolved payments, unsold units and time to outcome.
- A second channel changes the stock invariant; sharing a row, splitting the allocation and transferring blocks trade utilisation, stranded units and coupling.
- On a serial path, means add exactly but percentiles do not, so measure the whole journey.

CHAPTER 9

Data studio brief and semantic contracts

Learning objectives

After this chapter you should be able to:

- Define a metric contract with its population, event boundaries, time zone, exclusions and version.
- Calculate combined measures correctly from groups of different sizes.
- Design a grain-and-key table that prevents double counting when facts are joined.
- Explain event time, processing time and publication time, and how windows, watermarks and triggers handle late data.

Beacon must explain its numbers

> **Building on Book 1.** Book 1 built Beacon's first platform: metric definitions, a declared grain, event time, windows, watermarks and triggers, and a daily report that is revised when late events arrive. This studio scales Beacon to six sources and three reporting products, asks how a reader can trust a number that has been restated, and tests the design against a regulator's deadline for corrections.

Beacon Analytics combines records from six operational systems and publishes a daily service dashboard at 07:00 in the business time zone. About ten million new or changed records arrive each day. Three sources provide ordered change streams with positions, two provide daily extracts, and one sends events that may arrive up to seven days late. Older corrections are allowed through an approved exception process.

Three groups use the data differently. Managers compare service completion time by division and customer segment. Finance wants a monthly snapshot that it can reproduce exactly later. Operations wants to see changes during the day. These are three reporting products, even if they share ingestion infrastructure, and a single table marked "latest" cannot serve them all.

The team proposes a streaming lakehouse and claims that real-time ingestion will make the figures trustworthy. Your task is to evaluate that claim, design two feasible alternatives and state what a reader may conclude from each published version.

Define the question before the pipeline

A **metric contract** fixes what a measure means, so that a change in meaning can be told apart from a change in the business:

Element	Definition for "time to first closure", version 1
Population	Requests first closed during the reporting day
Start	The time the source accepted the request
End	The first verified closure
Time zone	The business reporting time zone, applied to both timestamps
Exclusions	Test records; negative intervals and missing timestamps go to quarantine
Reopened requests	Measured at first closure; a separate measure tracks reopen rates
Published alongside	The number and age of open requests
Owner	Head of service quality

The population of closed requests leaves out open work, so a division that leaves difficult requests open will look faster than one that closes them, even as its customers wait longer. Publishing the open backlog beside the closure time prevents that misreading. When the definition changes, for example from first closure to final closure, the version number changes too.

A median for the whole organisation cannot be rebuilt from division medians, and a mean must be weighted by group size. For example, division N closes 40 requests with a mean of 3 hours, and division S closes 160 with a mean of 8 hours. The combined mean is (40 × 3 + 160 × 8) / 200 = 7.0 hours, while

averaging the two division means gives 5.5 hours, because it treats the small division as if it were as large as the other.

Grain and identity

The **grain** of a fact table states exactly what one row represents. Kimball distinguishes several kinds of fact table, each suited to a different analytical purpose [25]. Beacon declares three grains:

Fact	One row per	Key
Service event	Source event and version	Source, event ID and version
Daily request snapshot	Request, reporting date and publication	Request key, date and publication ID
Completion	Qualifying first closure	Request key

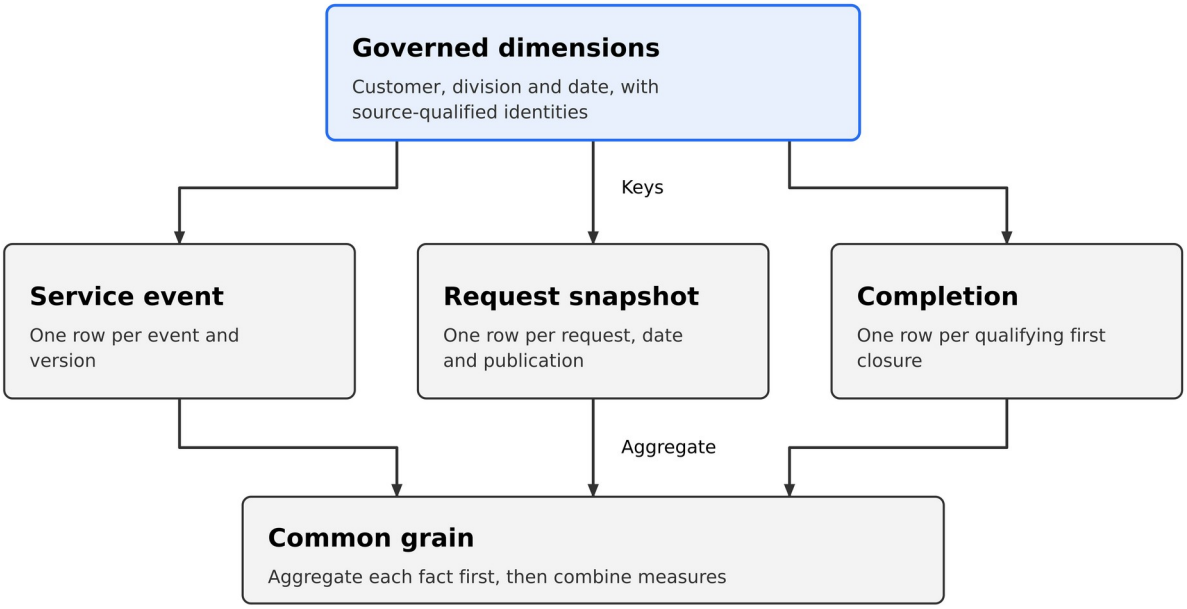


Figure 12 Facts at different grains share governed dimensions and are aggregated to a common grain before their measures are combined.

Figure 12 shows the three facts sharing governed dimensions. Joining facts of different grain multiplies rows: a completion of 6 hours joined to its request's five status events becomes five rows, and summing the duration gives 30 hours for one request. Aggregate each fact to the comparison grain before combining measures.

Identifiers must carry their source. Customer 17 in CRM A and customer 17 in CRM B may be different organisations, so an enterprise customer mapping needs an owner, a rule for ambiguous matches and visible handling of unresolved cases.

Three kinds of time

Beacon must keep three timestamps apart:

- **Event time** is when the source says the business event happened.
- **Processing time** is when Beacon received and processed the record.
- **Publication time** is when a particular result became visible to users.

The Dataflow Model of Akidau and colleagues names the parts [26]: a **window** groups events by event time, a **watermark** estimates how complete the input is, and a **trigger** decides when to emit a result. A **refinement mode**, which the paper calls an accumulation mode, says how a later result relates to an earlier one: discarding (only the new data), accumulating (the full updated value) or accumulating and retracting (the updated value plus an explicit retraction of the old one).

Beacon's daily dashboard maps onto these ideas. The window is the business day by event time. Each night a run starts at 01:00 with every record received by then and publishes at 07:00. The first trigger for a

day fires at 07:00 the next morning, and the result is marked provisional if the late-arriving source has not caught up. Later triggers fire at each following nightly run for seven days: any day whose input has changed is restated, and each later result replaces the earlier one and records which publication it supersedes, which corresponds to the accumulating and retracting refinement mode. For example, a request closes at 23:50 on 3 March, but its record reaches Beacon at 08:15 on 4 March, after the run that published 3 March. The run that starts at 01:00 on 5 March includes it, so the 3 March figure is restated at 07:00 on 5 March, and the dashboard shows that it changed and why.

Beacon must answer two questions about history: what is the current, corrected result for a past period, and what did the report show at a particular past publication? A design can store source versions and materialised publication versions, or rebuild old results from controlled history. Reproducibility does not mean keeping every personal field forever; Chapter 11 shows how deletion interacts with this history.

Your first submission

Produce:

1. a metric contract for the daily measure and one for the monthly snapshot;
2. a grain-and-key table;
3. a data ownership map for the six sources;
4. two pipeline alternatives;
5. a worked example containing a duplicate, a late arrival and a corrected event;
6. a sketch showing where publication is gated, where rejected records go, and how a user learns that yesterday’s number changed;
7. a service level objective for the 07:00 publication with the alert that warns before it is missed, for example when the nightly run has not passed its gate by 06:15;
8. one abuse case for the export path, such as an analyst exporting another division’s detail through a scheduled job, with its control and the audit record that would reveal it.

The monthly snapshot may need a formal close with a restatement policy, while the daily report may allow a rolling correction window. Both policies need business owners, and the brief leaves them for you to propose.

Exercises

9.1 Explain how a completion-time dashboard can improve while service quality gets worse if open work is excluded. Propose a second measure to publish alongside it.

9.2 One division has 10 requests averaging 2 hours and another has 90 averaging 6 hours. Calculate the combined mean and explain why averaging the two means equally is wrong.

9.3 Specify the keys and time fields needed to tell a duplicate event from a corrected version of that event.

9.4 An event with an event time of 22:10 on 10 May reaches Beacon three days later. Using the Dataflow Model terms, explain which publications change, which refinement mode Beacon uses, and what a dashboard user should see.

9.5 A completion fact holds 1,000 requests: 700 that took 3 hours on average and 300 from large customers that took 8 hours on average. A faulty customer dimension holds two current rows for each large customer, and the completion fact is joined to it. Calculate the total hours and the mean reported after the join, and the correct values. Describe two tests that would catch the fault.

Key points

- A metric contract fixes meaning, population, time zone and version, so that definition changes are visible.
- Means combine only with weights, and medians cannot be rebuilt from group medians.
- Each fact has a declared grain, and facts are aggregated to a common grain before their measures are combined.
- Event time, processing time and publication time differ, and windows, watermarks and triggers make the handling of late data explicit.
- Beacon must reproduce both the current corrected result and what was shown at each past publication.

CHAPTER 10

Pipeline alternatives and versioned publication

Learning objectives

After this chapter you should be able to:

- Calculate average arrival rates and the processing rate that a batch window requires.
- Compare batch and streaming designs for each of Beacon's three reporting products.
- Design a replay boundary and an idempotent rule for source identity, version and content.
- Implement a publication ledger that explains both current and past results, including restated means.

Design A: a governed batch platform

A batch design lands consistent extracts or source-supported incremental changes in restricted storage, validates them, transforms them into curated tables and computes the daily report under a run identifier. The report is published only after the required checks pass; if a run fails, the previous valid report stays visible with its timestamp.

Volume figures need care. Ten million records a day is an average arrival rate of 10,000,000 / 86,400 ≈ 115.74 records per second, which says little about batch capacity. If the nightly window is six hours, processing ten million records within it needs at least 10,000,000 / 21,600 ≈ 462.96 records per second on average, before joins, retries and checks.

Batch fits when the daily deadline and the source extract contracts dominate. It gives clear, reproducible boundaries and is simple for a small team to run, although it still needs correction handling, idempotent reruns, access control and reconciliation.

Design B: continuous ingestion with versioned publication

A streaming design ingests change streams continuously, records each source's position, validates events and updates curated state as they arrive. The daily dashboard can still be published as a versioned snapshot, because freshness of internal state and stability of a published decision product are separate properties.

Kafka's documentation describes ordering within each partition, and transactions that let a read-process-write cycle within Kafka achieve exactly-once results [27]. A pipeline that also writes to an external warehouse must examine that warehouse's own transaction and replay contract, because the broker's guarantee stops at its boundary. Streaming also adds continuous operation, state recovery, schema evolution and a late-data policy, and it cannot make a once-daily extract any fresher. The fair comparison is product by product:

Product	Design A, batch	Design B, streaming
Daily dashboard	Fits the 07:00 deadline	Fits; published as a daily snapshot
Monthly snapshot	Fits; a formal close is natural	Fits, with the same close process
Intraday operations view	Cannot provide it	Provides it for the three streaming sources

Many teams choose a hybrid: streaming for the sources and products that benefit, batch for the daily extracts. On Azure, Event Hubs can carry the change streams, Azure Data Factory the extracts and Fabric Data Warehouse the curated tables; on AWS, Amazon Kinesis Data Streams, AWS Glue and Amazon Redshift fill the same roles [28].

Decision record ADR-B01 chooses the hybrid for Beacon's first release. The three sources with change streams are ingested continuously, the two extract sources nightly, and the late-arriving source through the stream with a seven-day restatement window. Every product passes one publication gate and is recorded in one publication ledger, described below. The intraday view covers only the streaming sources and says so on screen. Its review triggers are: an extract source starts offering a change stream; a product needs extract data fresher than the nightly run; or restatements affect more than 5 percent of published days in a month.

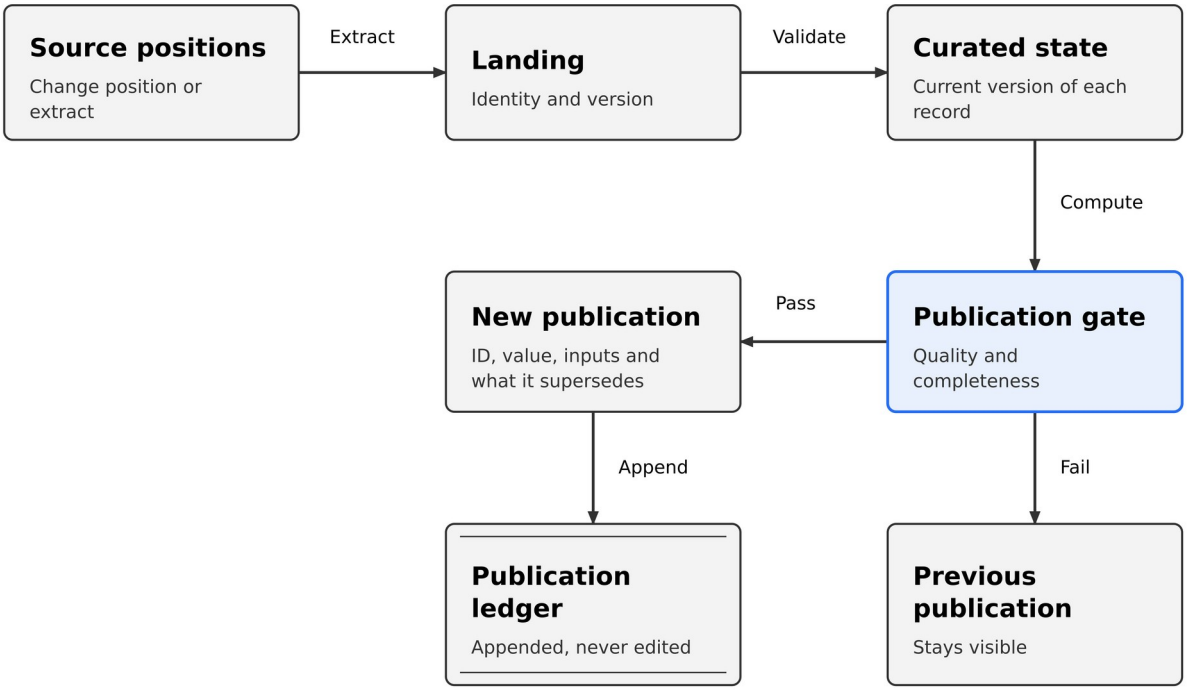


Figure 13 Source positions and quality checks gate a named publication version, and the previous report stays visible when a gate fails.

Figure 13 shows the path common to both designs, in which ingestion progress and a publishable business result are separate claims.

Define a replay boundary

Record the extraction or change position that each batch of input came from. When a snapshot is combined with a change stream, both must share a documented boundary, because an arbitrary file export followed by a stream started at an unrelated time can leave a gap or apply some changes twice.

Transformations must also tolerate reruns. For example, take an event with identity E:

- replaying version 3 of E must not add its measure a second time;
- a late copy of version 2 must not overwrite version 3, under a contract in which versions only increase;
- two records with the same identity and version but different content are a conflict, to be quarantined or settled by an approved rule.

Arrival order alone is no evidence of which record is right. A deletion marker needs a defined meaning too: it may mean that the source entity was deleted, that an erroneous event was withdrawn, or that a copy of personal data must be removed, and these affect the accounts differently.

Worked publication history

Beacon's daily measure, time to first closure, is a mean, so a restatement can change both the value and the number of requests behind it. The listing keeps a **publication ledger**: each publication is appended with an identifier, its value, the exact record versions it used and the publication it supersedes, and no entry is ever edited. On day 1, two requests close at first: r1 after 4 hours and r2 after 10. A third, r3, closed after 1 hour, arrives late. The source then replays r1 unchanged, and later corrects r2's start time, so version 2 of r2 records 16 hours.

```
Record = tuple[str, int, int, float]      # (request, version, day, hours to close)

def latest(records: list[Record]) -> dict[str, Record]:
    """Highest version of each request; a replayed copy changes nothing."""
    best: dict[str, Record] = {}
    for r in records:
        if r[0] not in best or r[1] > best[r[0]][1]:
            best[r[0]] = r
    return best

ledger: list[dict] = []                   # publications are appended, never edited

def publish(records: list[Record], day: int, reason: str) -> None:
    rows = [r for r in latest(records).values() if r[2] == day]
    inputs = sorted((r[0], r[1]) for r in rows)
    before = next((p for p in reversed(ledger) if p['day'] == day), None)
    if before and before['inputs'] == inputs:
        print(f'{reason}: inputs unchanged, no new publication')
        return
    mean = sum(r[3] for r in rows) / len(rows)
    entry = {'id': f'P{len(ledger) + 1}', 'day': day, 'mean': mean,
             'inputs': inputs, 'supersedes': before['id'] if before else None}
    ledger.append(entry)
    note = f", supersedes {entry['supersedes']}" if before else ''
    print(f"{entry['id']} day {day}: mean {mean:.1f} h over {len(rows)} "
          f"requests{note} ({reason})")

feed = [('r1', 1, 1, 4.0), ('r2', 1, 1, 10.0)]
publish(feed, 1, 'first run')
feed.append(('r3', 1, 1, 1.0))            # a day-1 closure that arrives late
publish(feed, 1, 'late arrival of r3')
feed.append(('r1', 1, 1, 4.0))            # the source replays r1
publish(feed, 1, 'replay of r1')
feed.append(('r2', 2, 1, 16.0))           # the source corrects r2's start time
publish(feed, 1, 'correction of r2')
first, last = ledger[0], ledger[-1]
print(f"{first['id']} and {last['id']} both show {first['mean']:.1f} h:")
print(f"  {first['id']} used {first['inputs']}")
print(f"  {last['id']} used {last['inputs']}")
```

It prints:

```
P1 day 1: mean 7.0 h over 2 requests (first run)
P2 day 1: mean 5.0 h over 3 requests, supersedes P1 (late arrival of r3)
replay of r1: inputs unchanged, no new publication
P3 day 1: mean 7.0 h over 3 requests, supersedes P2 (correction of r2)
P1 and P3 both show 7.0 h:
  P1 used [('r1', 1), ('r2', 1)]
  P3 used [('r1', 1), ('r2', 2), ('r3', 1)]
```

Three things follow:

- The replay produces no publication, because the inputs did not change, so reruns are idempotent at the level of publications as well as records.
- Each restatement names its reason and the publication it replaces, so the dashboard can show "restated from 5.0 h: correction of r2".
- P1 and P3 show the same 7.0 hours for different reasons. A reader comparing values would conclude that nothing happened, while the ledger shows a late closure and a correction that happened to cancel out. That is why each publication stores its inputs as well as its value.

The model computes in memory; a real platform also persists the run identity, source positions, metric version, access policy and lifecycle decisions. "Never edited" describes the identity and recorded value of a publication; it grants no right to keep the underlying personal data indefinitely, as Chapter 11 discusses.

Publication failure and partial data

When a critical source is missing, the report owner chooses between blocking the publication, publishing a clearly marked partial report, or keeping the previous version. Decide in advance which sources are mandatory for which measures, so that one late source does not block unrelated products, and make completeness machine-readable, for example with a completeness flag on each published measure.

A quality gate must be able to fail. If every anomaly is downgraded to a warning so that 07:00 is met, the gate protects nothing. Record each exception with its owner, the measures affected and an expiry date; repeated exceptions point to a requirement or source-quality problem that needs a design decision.

> **Common mistake.** Reporting only “pipeline succeeded”. The pipeline’s success and the report’s fitness for a decision are separate claims, and only the second matters to the reader of the dashboard.

Exercises

10.1 Calculate the minimum average processing rate for ten million records in a four-hour window, ignoring overhead. Explain why this is not yet a capacity plan.

10.2 A day-2 report starts with r4, closed after 2 hours, and r5, closed after 6 hours. Then r6, a day-2 closure of 13 hours, arrives late, and r4 is corrected to 5 hours as version 2. List the ledger entries that the listing would produce, with each mean and the publication it supersedes. Explain why a manager who sees only the latest mean might wrongly conclude that day 2’s service got slower.

10.3 Design an idempotent rerun rule based on source identity, version and content. Include the case of two records with the same version but different content.

10.4 Compare Designs A and B for the monthly snapshot and for the intraday operations view. Recommend a design for each product and justify the choice.

10.5 Extend the publication model so that a source can withdraw an erroneous closure. Define the withdrawal record, show the publication that follows if r3 is withdrawn after P3, and state what the ledger must show.

Key points

- An average arrival rate says little about the processing rate that a batch window requires.
- Batch and streaming suit different products, and many platforms combine them.
- Replay needs a recorded source position and rules for identity, version and conflicting content.
- A publication ledger that stores each value with its inputs explains both the current corrected result and what each earlier publication showed.
- A quality gate that can never fail protects nothing, and exceptions need owners and expiry dates.

CHAPTER 11

Data quality, security and analytical evaluation

Learning objectives

After this chapter you should be able to:

- Design reconciliation checks that go beyond row counts, and state the blind spot of each.
- Evaluate access control across every path to derived data, including exports and restored backups.
- Test an access policy against an independently written matrix that includes a tenant restriction.
- Evaluate a generative AI explanation feature for confabulation, access control and traceable support.
- Calculate the worst-case delay of a publication schedule, and respond to a change card with a decision record and a decision brief.

Reconciliation beyond counts

Equal row counts can hide offsetting errors. For example, suppose a source and a target each hold three records, but one source record is missing from the target and the target holds one record that the source never sent. The counts match while two records are wrong. Each stronger check covers a different failure and has its own blind spot:

Check	Finds	Misses
Key-set comparison	Missing and extra records	Wrong values on matching keys
Version comparison	Stale records	Errors in the source's own versioning
Relationship check	Orphans, such as a closure without a request	A valid but wrong reference
Grouped totals	Some mapping errors, such as a misassigned division	Errors that cancel out within a group
Digest of canonical content	Any change in content	Whether an approved transformation has the right business meaning

Beacon reconciles each source at its declared change position or extract boundary, and the accounting must close: received records equal accepted plus quarantined plus excluded. A record excluded by policy is listed with its reason and the measures it affects. Approved transformations need their own expected values: if a status code is normalised through an approved mapping, generate the expected values from that mapping, and have the business owner review the mapping itself.

Quality rules differ in kind. "Completion time must not be negative" is structural, while "a closure requires an earlier valid receipt" is relational. A sudden shift in a distribution is a reason to investigate, because a real change in the business can also produce unusual values.

Figure 14 shows how these checks combine into one publication decision.

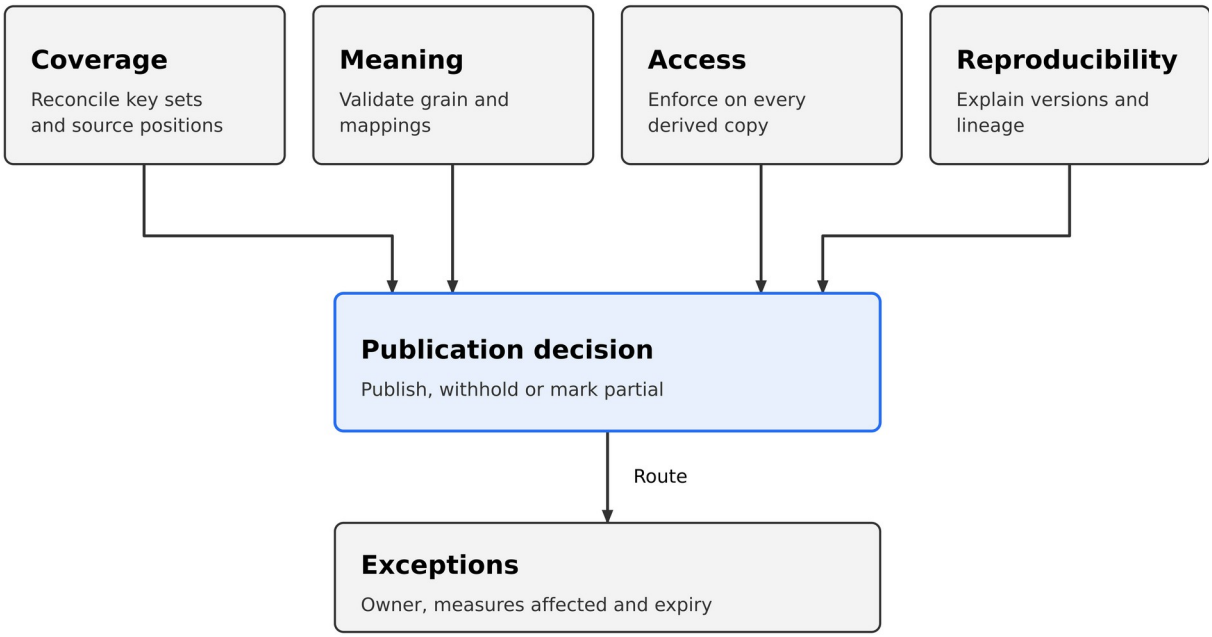


Figure 14 A publication decision combines coverage, meaning, access and reproducibility, and routes exceptions to owners.

Security follows derived copies

The same information appears in the raw landing area, curated tables, notebooks, dashboard exports and cached query results, and each is a separate place to enforce access. A catalogue label saying “restricted” enforces nothing by itself, and a dashboard filter protects nothing if users can query the unfiltered table directly.

Removing names does not make data anonymous. For example, a report of “service requests from hospitals in one town with more than 50 machines” may describe exactly one customer. Assess the audience and purpose, and apply aggregation, suppression or other approved controls where needed. The GDPR includes purpose limitation, data minimisation, storage limitation and accountability among its principles [29]. Beacon therefore needs a policy owner, an inventory of where data lives, and mechanisms that apply the approved lifecycle across operational and derived stores.

Deletion and recovery must agree. When a deletion or restriction is approved, identify every copy it affects. A later replay or backup restore must not quietly bring the data back, so one design keeps a controlled list of lifecycle decisions and reapplies it during every replay and restore. Some aggregates may be kept under the applicable policy while the identifiable detail behind them is removed; document what that does to reproducibility.

Two access designs

Beacon’s access rule has three parts:

- An ordinary analyst may read detail records of their own division, and only for customer organisations in their approved scope. Some customer contracts restrict who may analyse their records, so each analyst’s scope is a set of tenants.
- A group analyst may read only approved consolidated aggregates, never raw detail.
- Only a designated publishing identity may publish aggregates. An export is a new copy with its own audience and lifecycle.

Alternative A publishes separated data products. Each division receives a curated product with a narrow audience, and consolidated reporting is a separately approved product. The access boundary is easy to see, at the cost of duplicated processing and the need to carry corrections and deletions into every product.

Alternative B uses one shared curated store with enforced policies. Every query carries an independently established identity and resource context, and the data service enforces the policy on every supported path. Shared views reduce duplication, but a misconfigured export identity or a direct table

permission can bypass the filter, so the policy engine and the store's configuration become part of what must be assured.

NIST's zero trust architecture argues for explicit access decisions at each resource rather than trust based on network location [30], and both alternatives apply that idea at the data boundary.

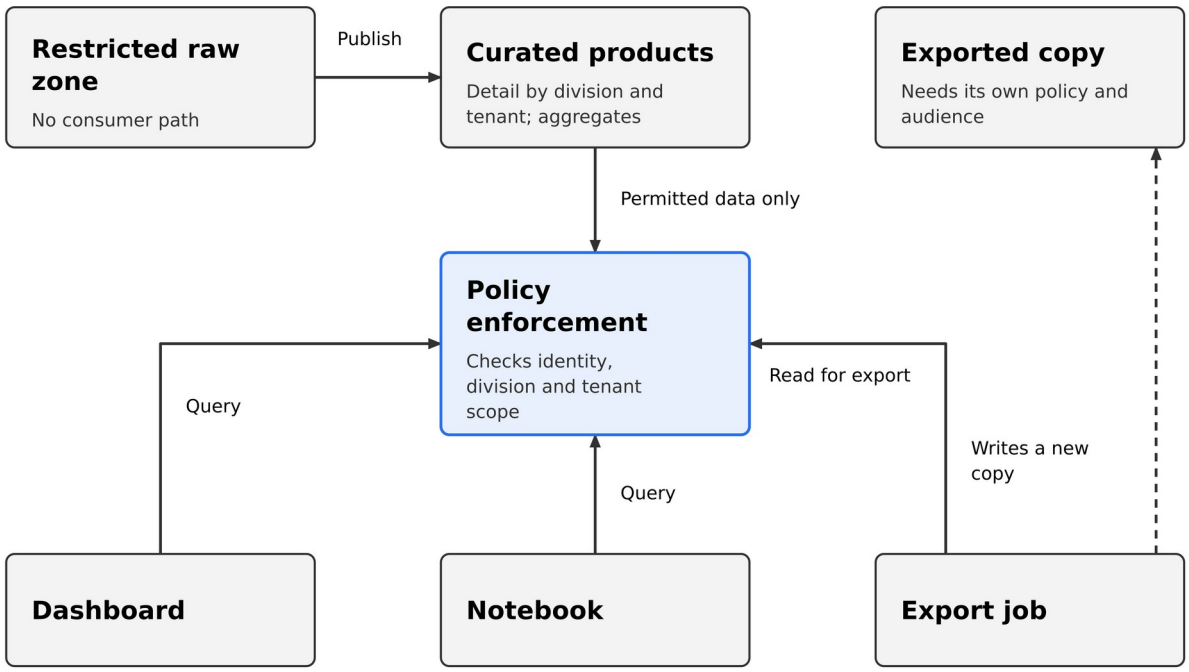


Figure 15 Dashboard, notebook and export requests all pass the same policy enforcement point, and an exported copy needs its own policy.

Figure 15 shows the paths. The restricted raw zone has no consumer path, and an export leaves the enforcement point as a new copy.

Test the policy against a written matrix

A policy test is only as good as its expected results: if the same logic generates both the policy and the expected answers, a mistake in that logic passes its own test. The listing writes the expected reads by hand from the policy statement and compares them with the policy function for every principal and resource.

```
PRINCIPALS = {      # authenticated identities with their approved scope
    'ana': {'role': 'analyst', 'division': 'N', 'tenants': {'T1', 'T2'},
            'active': True},
    'sam': {'role': 'analyst', 'division': 'S', 'tenants': {'T1', 'T2', 'T3'},
            'active': True},
    'gita': {'role': 'group', 'active': True},
    'pat': {'role': 'publisher', 'active': True},
}
RESOURCES = {f'{d}{t}': {'kind': 'detail', 'division': d, 'tenant': f'T{t}'}
             for d in 'NS' for t in '123'}
RESOURCES['G1'] = {'kind': 'aggregate'}

def allowed(who: str, what: str, action: str = 'read',
            check_tenant: bool = True) -> bool:
    p, r = PRINCIPALS.get(who), RESOURCES.get(what)
    if p is None or r is None or not p['active']:
        return False                                  # deny by default
    if action == 'publish':
        return p['role'] == 'publisher' and r['kind'] == 'aggregate'
    if action != 'read':
        return False
    if r['kind'] == 'aggregate':
        return p['role'] == 'group'
    return (p['role'] == 'analyst' and p['division'] == r['division']
            and (r['tenant'] in p['tenants'] or not check_tenant))

# Written by hand from the policy statement, not generated from allowed().
EXPECTED_READS = {('ana', 'N1'), ('ana', 'N2'), ('sam', 'S1'), ('sam', 'S2'),
                  ('sam', 'S3'), ('gita', 'G1')}

def failures(**options) -> list[tuple[str, str]]:
    return [(p, r) for p in PRINCIPALS for r in RESOURCES
            if allowed(p, r, **options) != ((p, r) in EXPECTED_READS)]

print('read matrix failures:', failures())
print('publishers of G1:', [p for p in PRINCIPALS if allowed(p, 'G1', 'publish')])
print('unknown action allowed:', allowed('ana', 'N1', 'delete'))
print('division-only policy failures:', failures(check_tenant=False))
PRINCIPALS['sam']['active'] = False                   # access revoked
print('sam after revocation:', [r for r in RESOURCES if allowed('sam', r)])
```

It prints:

```
read matrix failures: []
publishers of G1: ['pat']
unknown action allowed: False
division-only policy failures: [('ana', 'N3')]
sam after revocation: []
```

All 28 read combinations match the written matrix, only the publisher may publish, and an unknown action is denied. The fourth line shows why the tenant field matters. A plausible defect, a policy that checks the division and forgets the tenant scope, lets analyst ana read N3, a record of tenant T3 outside her scope, and the hand-written matrix catches it. After revocation, sam can read nothing. The function models the policy in memory; the same matrix must also be run against each real path: the dashboard, the notebook, a direct query and an export.

Policy changes need the same care. If an analyst loses access, can a cached result or a scheduled export still deliver records to them? Whether a job keeps a durable approval or needs current user authority must be an explicit decision, not something a token copied into a scheduler decides by accident.

Evaluate an AI explanation feature

Beacon’s sponsor asks for an assistant that explains why completion time rose last month. A grounded summary can report that the measure rose, cite its definition and point to the source tables. It should not invent a cause from a correlation, because a change in case mix, a missing source, a policy change and a genuine delay are competing explanations.

Generative AI systems can produce confident but false statements, which NIST’s generative AI profile calls confabulation [31]. The evaluation must therefore test the actual task:

- each statement is supported by the cited data and metric version;
- important caveats, such as a provisional source, are kept;
- retrieval runs with the same access scope as the user who asks;
- the assistant says so when the evidence is absent;
- a reviewer can trace every consequential statement to approved data.

NIST's AI Risk Management Framework offers a voluntary structure for organising such risks [32]. A legitimate first release could provide a controlled metric summary with links to the data and omit causal explanations; a more ambitious feature needs stronger evaluation and clear operating responsibility.

> **Check your understanding.** The assistant reports that "division S is slower because its staff are less efficient". List the evidence needed before that sentence could be published, and the alternative explanations it must rule out. A short answer is given at the start of the Chapter 11 answers.

A change card: a deadline for corrections

Beacon's daily service figures are also sent to the sector regulator. Three months into the first release, Beacon receives this change card.

Field	Content
Card	CC-B1, deadline for published corrections
New fact	From 1 July, a correction to any published daily service figure must appear on the dashboard within 24 hours of the corrected record reaching Beacon, with the reason for the restatement
Source	Regulator's notice, confirmed by Beacon's compliance officer
Assumptions affected	B-A3: "restatements can wait for the next nightly run"; B-A4: "corrections older than seven days go through an exception process with no deadline"
Decisions affected	ADR-B01, nightly publication with a seven-day restatement window
Question	How should Beacon meet the deadline, and what does each option cost?

Start with the current design's worst case. The nightly run starts at 01:00 with every record received by then and publishes at 07:00. A corrected record that arrives at 01:01 misses that run, joins the next one 24 hours later and appears at 07:00 the following morning, about 30 hours after it arrived. If that run fails its gate, the previous publication stays visible and the correction waits another day: about 54 hours. Corrections more than seven days old are worse, because the exception process takes a median of three working days. Last quarter, about 3,000 corrected records a day touched published days, and about 40 a month were older than seven days.

The worst case depends on two things: the longest gap between runs and the time a run takes. A run that restates only the days whose inputs changed takes about 40 minutes, against six hours for the full nightly run. Three options meet the deadline when nothing fails:

Option	Worst case	After one failed run	Extra compute	Engineering
1. A second full run at 13:00	About 18 hours	About 30 hours	About €2,100 a month	About 1 week
2. Restatement runs at 09:00, 13:00, 17:00 and 21:00	About 10 hours	About 14 hours	About €960 a month	About 3 weeks
3. Continuous restatement, published hourly	About 2 hours	About 3 hours	About €3,000 a month	About 8 weeks

Option 1 meets the deadline only if every run succeeds, which ignores the gate that Chapter 10 insisted must be able to fail. Option 3 gives far more margin than the rule needs, and figures that change every hour are harder for managers to read. Option 2 survives two consecutive failed runs at about 18 hours and three at about 22 hours, and its restatements arrive at predictable times.

Old corrections need a separate answer. The deadline applies to the daily service figures, so a correction older than seven days is restated by the next run like any other, and a data owner reviews it the following working day instead of approving it first. The finance monthly snapshot is a different product with its own close: its corrections stay as adjustments in the next month's close, with no change to the closed figures.

Decision record

ADR-B02: Meet the 24-hour correction deadline with four-hourly restatement runs. Status: accepted. Amends ADR-B01.

- **Context.** CC-B1 requires corrections to published daily figures within 24 hours. The nightly-only design takes up to about 30 hours, or 54 hours after one failed run, and the exception process for old corrections has no deadline.
- **Decision.** Add restatement runs at 09:00, 13:00, 17:00 and 21:00 that recompute only days with changed inputs and publish through the same gate and ledger. Restate corrections older than seven days automatically, with review by the data owner on the next working day. Leave the finance snapshot's close unchanged.
- **Alternatives.** A second full run (fails the deadline after one failed run); continuous restatement (more cost and operation than the rule requires).
- **Consequences.** A gate failure during the day needs an owner on duty from 08:00 to 22:00. The ledger records a reason for every restatement. Daily figures may change up to five times a day while corrections arrive.
- **Evidence.** The schedule calculations in this section; a rehearsal in which three consecutive runs are failed on purpose and the correction still appears within 24 hours.
- **Review triggers.** A restatement run exceeds 90 minutes; more than one restatement run fails in a week; the regulator shortens the deadline; or corrections exceed 10,000 records a day.

Decision brief

Decision needed by 1 June, from Beacon's head of data: how to publish corrections to the daily service figures within 24 hours.

What changed. The regulator now requires every correction to appear within 24 hours. Today a correction can take 30 hours, or 54 if a nightly run fails.

What stays the same. In every option, the 07:00 publication continues, every restatement shows its reason, and the finance monthly snapshot keeps its close.

Options.

1. Run the full publication twice a day: about €2,100 a month; meets the deadline only if no run fails.
2. Add four short correction runs during the day: about €960 a month and three weeks of work; meets the deadline even after three failed runs in a row.
3. Publish corrections continuously: about €3,000 a month and eight weeks of work; figures change every hour.

Recommendation. Option 2, with a daytime owner for failed runs.

Risks and conditions. Correction volume could grow beyond what a 40-minute run handles; the review trigger at 90 minutes catches that early.

Studio assessment

Submit a data dossier with metric contracts, two pipeline alternatives, a publication ledger example, reconciliation and quality checks, a lifecycle design, the access matrix for your chosen design, your response to CC-B1, the publication objective with its alert, the export abuse case with its audit record, and an experiment record. A valid design may be batch, streaming or hybrid if its guarantees fit the reporting products.

Include one research critique. Codd's relational model is a good choice [33]: Codd argued that users and programs should be protected from changes in how data is stored and organised internally, a property he called data independence. Explain how Beacon's curated layer gives analysts that protection when ingestion changes underneath, and identify one decision, such as the grain of a fact table, that the relational model leaves to the designer.

Exercises

11.1 A source and a target each contain one million rows. For each defect, name a check from this chapter that finds it and one that would miss it: (a) 200 records were loaded twice and 200 others never arrived; (b) one division code was mapped to the wrong division for 5,000 records; (c) 30,000 amounts were rounded by a different rule; (d) 50 closures refer to requests that were never loaded.

11.2 A restored backup of the raw zone contains a record that was removed under an approved deletion. Design the recovery control needed before anything is republished.

11.3 An AI-generated narrative says that slower service in one division is caused by its staff's performance. List the evidence and the alternative explanations that must be examined before the claim is accepted.

11.4 Choose Alternative A or B. Specify six access test cases, including a tenant outside an analyst's scope and a revoked user, and state which paths, from dashboard, notebook, direct query and export, each case must be run against.

11.5 Using Codd's argument for data independence, explain how a curated view can stay stable while Beacon changes from daily extracts to change streams for one source. Name one thing data independence does not decide for Beacon.

11.6 Beacon considers restatement runs every six hours, at 07:00, 13:00 and 19:00, in addition to the nightly run from 01:00 to 07:00; each restatement run takes 40 minutes. Calculate the worst-case delay for a correction with no failed run and with one failed run. How many consecutive failed runs can the schedule absorb within 24 hours?

11.7 The finance team asks for its closed monthly snapshot to be restated within 24 hours as well, "to be consistent". Evaluate the request, and propose a response that respects both the regulator's rule and finance's need to reproduce a closed month.

Key points

- Equal counts can hide offsetting errors, so reconcile key sets, versions, relationships and grouped totals, each with known blind spots.
- Access must be enforced on every derived copy and path, including exports, caches and restored backups.
- An access test needs an expected-results matrix written independently of the policy, and it should include the tenant restriction.
- A generative explanation feature must be evaluated for support, omissions, access scope and the handling of missing evidence.
- Deletion, replay and restore must agree, so that recovery never brings removed data back.
- A publication deadline is met by the gap between runs plus their duration, with margin for failed runs, and applies only to the product it names.

CHAPTER 12

Modernisation studio brief and discovery

Learning objectives

After this chapter you should be able to:

- Design a discovery plan that finds every writer to a business object, with a confidence level for each.
- Compare direct replacement, phased migration and temporary rehosting against a fixed deadline.
- Explain how the strangler fig approach applies when some writers bypass the routing seam.
- Evaluate a partner interface change as a contract negotiation with technical, commercial and timing terms.

Orion has a deadline and an incomplete inventory

> **Building on Book 1.** Book 1 introduced Orion's migration as a sequence of operable transition states, with a strangler fig routing seam, a write fence and a latest safe decision point. This studio works from measurements instead: a writer inventory built from audit logs, replication lag per table, dual-run results, three timed rehearsals and a partner delay, and it enforces the write fence inside the new database.

Orion sells industrial supplies to business customers. Its order management runs on an ageing platform whose hosting agreement ends in nine months. The studio's task is to move order intake and order management to a new order service, with its own PostgreSQL order store, before that date. Warehouse execution stays with Orion's warehouse partner. The brief gives these facts:

- the legacy database holds 12 million historical orders;
- about 45,000 new orders arrive each day, and the order tables see about 1.5 million row changes a day;
- order intake may pause for at most 45 minutes, in a Sunday window from 02:00 to 02:45, and customers must still be able to look up order status during the pause;
- the new service supports two order states that the legacy platform cannot represent: "partially dispatched" and "awaiting customs";
- the warehouse partner writes status updates directly into the legacy order tables through its own database account.

The migration must keep every accepted order and never dispatch an order twice. The business may postpone the two new states if the architect shows why that reduces risk.

Find every writer

A migration fails most dangerously through a writer nobody knew about. Inventory every path that can create or change an order: application interfaces, batch jobs and file imports, database accounts used by partners and other teams, and support scripts. Combine owner interviews, scheduler entries, database account lists and observed activity. Orion enabled database audit logging on the order tables for four weeks, deliberately including a month end; for example, a one-week trace would have missed the finance job, which runs only at month end.

The audit produced this inventory, with a confidence level for each entry:

Writer	Evidence	Confidence	Planned action
Legacy application	Code, configuration and audit log	Confirmed	Replaced by the new order service
Marketplace import job	Scheduler entry and audit log	Confirmed	Rebuilt as a client of the new order API
Partner account `wh_partner`	Audit log shows updates between 06:00 and 22:00	Confirmed	Moved to a new status API by contract change
Finance month-end adjustment	Owner interview and one observed run	Observed once	Observed again at the next month end, then rebuilt
Account `ops_batch`	Audit log shows 212 status updates at 00:30	Observed, owner unknown	Investigated

The last row is what discovery exists for. Investigation traced `ops_batch` to a script, written years earlier by the support team, that closes orders with no activity for 90 days. Nobody had listed it, because nobody thought of it as part of order management. It must become a function of the new service or be retired by decision; either way, it cannot keep writing to the legacy tables after the move.

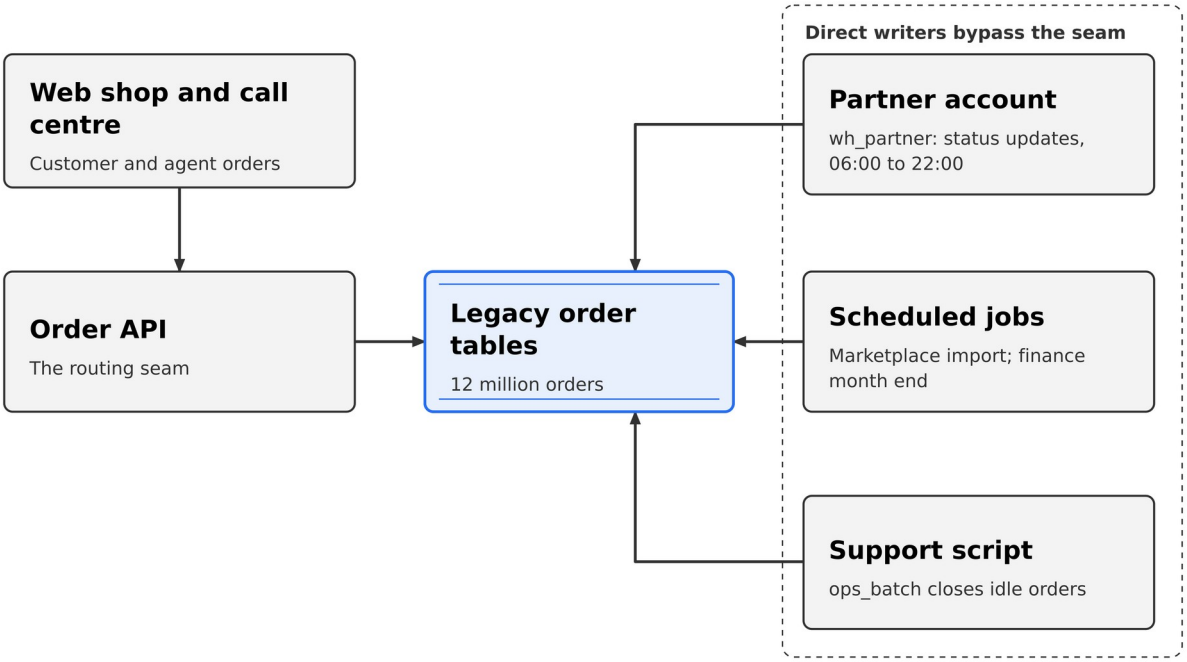


Figure 16 The migration inventory must include writers that bypass the main application, such as partner accounts, scheduled jobs and support scripts.

Figure 16 shows why a diagram of the main application alone misleads. Readers matter too, with different consequences: for example, a reporting extract can tolerate a delay that a warehouse dispatch update cannot.

Compare the three strategies

Direct replacement copies the data and switches all traffic at one moment. It keeps coexistence short but concentrates every kind of change into one event, and the 45-minute window rules out a bulk copy of 12 million orders inside it.

Phased migration keeps the legacy platform authoritative while change data capture fills the new store, runs the two systems side by side, and then transfers write authority in a short, rehearsed cutover. Each step is smaller, but the intermediate states must be operated.

Temporary rehosting moves the legacy platform unchanged to new hosting for, say, twelve months. It removes the deadline risk but keeps the technical debt and adds a second migration later.

Martin Fowler coined the strangler fig image in 2004: gradually create a new system around the edges of the old one until the old one can be removed [34], and his 2024 description adds the activities around it: understanding the desired outcomes, breaking the problem into parts, delivering those parts, and changing the organisation, with transitional architecture that lets old and new coexist [35]. Microsoft's pattern guidance describes the same idea with a routing façade that sends each request to the old or the new implementation [36]. In Orion, the natural seam is the order API used by the web shop and the call centre. The seam controls only the traffic that passes through it, however, and the partner account, the finance job and the support script all write to the tables directly. They must be brought behind the seam, or fenced off, before the new service can own the orders.

A sound first recommendation is phased migration, with temporary rehosting as a fallback whose decision date is the end of month six, after which rehosting would no longer fit before the deadline.

The partner contract is part of the architecture

The partner's direct database writes cannot continue after the move, so the interface must change, and that is a negotiation as much as a design task. Orion's proposal covers six terms:

- **Interface:** the partner sends each status change to a new HTTPS status API, with a unique event ID that makes a resent event harmless.
- **Service level:** status events within 15 minutes of the warehouse scan, and resends with the same event ID for up to 24 hours.
- **Authority:** every dispatch command from Orion carries an authority epoch, and the partner rejects commands with an older epoch than the latest it has seen.
- **Schedule:** ten weeks of partner development and two weeks of joint testing, including one joint cutover rehearsal.
- **Cost:** a one-off charge of €18,000 quoted by the partner.
- **Fallback:** if the API is unavailable, the partner delivers a status file every 15 minutes, which Orion converts into API calls.

The authority term protects the partner's side, where Orion's own controls cannot reach: after cutover, the legacy platform must never send a dispatch command that the partner accepts. Chapter 13 develops the same mechanism inside Orion's new store. The fallback term decouples Orion's timeline from the partner's. Orion estimates 15 engineer-days to build the file adapter, an option that Chapter 14 will need.

Your first submission

Produce:

1. a dependency map of the current estate, with a confidence level for each writer and reader;
2. a comparison of the three strategies against the nine-month deadline;
3. a write-authority table showing who may write orders in each phase of your plan;
4. the earliest step that can be reversed easily, and the first step after which recovery must preserve new business data;
5. your negotiating position for the partner contract, with the terms you would give up first.

Treat the deadline as a constraint to analyse, not as proof that the plan fits. If the evidence does not support replacement in time, propose the fallback, its cost and the date by which it must be chosen.

Exercises

12.1 Explain why a one-week observation of database activity may miss a writer that matters for the migration. Name two further sources of evidence.

12.2 Compare temporary rehosting with phased migration under the nine-month deadline. State what evidence would favour each.

12.3 Design a rule that keeps each in-flight order under one write authority during a phased migration in which orders move in cohorts.

12.4 The audit finds 212 unexplained updates from account `ops_batch`. Write the investigation steps and the two possible outcomes, with the design consequence of each.

12.5 The partner rejects the authority-epoch term as too costly. Evaluate what Orion loses, and propose an alternative control at Orion's side.

Key points

- Discovery must find every writer, including partner accounts, scheduled jobs and support scripts, and record how strong the evidence is for each.
- Observation windows must cover infrequent work such as month-end jobs.
- Phased migration with change data capture keeps one authority while reducing the size of each step.
- A strangler seam controls only the traffic that passes through it, so direct writers must be moved behind it or fenced.
- A partner interface change is a negotiated contract with technical, timing, cost and fallback terms.

CHAPTER 13

Data movement, write fencing and recovery

Learning objectives

After this chapter you should be able to:

- Design an initial load and change data capture that share one consistent boundary, and calculate the catch-up time.
- Turn replication lag measurements into a cutover precondition, distinguishing the applier's row rate from its transaction rate.
- Implement a daily dual-run reconciliation that compares key sets and values up to the position the target has reached.
- Implement a write fence that the destination enforces with an authority epoch on every replicated table.
- Compare recovery before and after customer writes reach the new system.

Load once, then follow the log

Change data capture (CDC) reads committed changes from the source database's transaction log and applies them to a target. Because it reads the log, it sees every writer that commits to the database, including partner accounts and forgotten scripts.

The initial load and the change stream must meet at one boundary. Orion takes a consistent snapshot of the legacy database at log position P0, loads it into the new store, and starts CDC from P0. A snapshot taken at one moment and a stream started at an unrelated later moment can lose the changes made in between.

Changes keep arriving during the load. In rehearsal, loading 12 million orders took 6 hours 40 minutes, or 24,000 seconds. At about 1,500,000 / 86,400 ≈ 17.4 row changes per second, about 417,000 changes queue up. The applier commits once per source transaction, so that an order and its lines always appear together, and that commit sets its speed. For ordinary transactions of about three rows, the applier manages about 280 row changes per second; inside one large transaction, which commits only once, it reaches about 1,200 rows per second. Ordinary traffic dominates the queue, so with the drain model from Chapter 2 it clears in about 417,000 / (280 – 17.4) ≈ 1,590 seconds, or 26 minutes. On the clouds, Azure Database Migration Service and AWS Database Migration Service can play the applier's role for supported source and target pairs; support varies by source, target and online or offline mode [16].

Measure lag where it matters

Replication lag is the time between a change committing in the source and the same change being applied in the target. Orion replicates all three order tables through one subscription, which applies transactions in their commit order, so one lag figure describes the whole stream: a change to `orders` cannot overtake a large transaction on `status_history` ahead of it. Orion measures that lag with a heartbeat: every five seconds, a job writes a timestamped row to a replicated heartbeat table, and the target records when the row arrives. The servers' clocks agree within a few milliseconds, which is small compared with lags measured in seconds. Over a four-week dual run, the measurements were:

Period	p50 lag	p95 lag	p99 lag	Maximum lag
Ordinary business days	0.8 s	2.9 s	7 s	46 s
The month-end day	1.1 s	4.2 s	12 s	8 min 40 s
All four weeks	0.8 s	3.0 s	8 s	8 min 40 s

The typical figures look excellent. The maximum came from the finance month-end job, which updates 600,000 rows of `status_history` in a single transaction that the applier must apply whole: at about 1,200 rows per second that takes 500 seconds, and every change committed after the job waits behind it, so the whole stream fell about eight and a half minutes behind. Orion turns this into a precondition: start the cutover only when the stream's lag has stayed below 10 seconds for thirty minutes, and never within 24 hours after the month-end job.

Map meaning, not columns

A mapping can copy every row successfully and still change what the data means. Two examples from Orion show how:

- **VAT totals.** The legacy platform rounds VAT on each order line and stores the sum, while the new service rounds VAT once per order, so totals can differ by a cent or more. Customers were invoiced the legacy totals, so mapping version 2 copies historical totals exactly as recorded, and the new rule applies only to new orders.
- **Promised dates.** The legacy `promised_date` is a local date with no time. Converting it to a timestamp at midnight would invent precision and could shift the date across time zones, so the target keeps it as a date.

Record the mapping version used for each record. For example, if a customer later queries an old invoice total, the record shows that mapping version 2 copied it unchanged.

Run both systems side by side

During a four-week **dual run**, the legacy platform stays authoritative and CDC keeps the new store current. The new order service runs in shadow: it receives a mirrored copy of every order request, computes its own prices, VAT and warehouse assignment, and stores the result where no customer sees it. Each day, one comparison checks the replicated data against the legacy data, and another checks the shadow results against the legacy results.

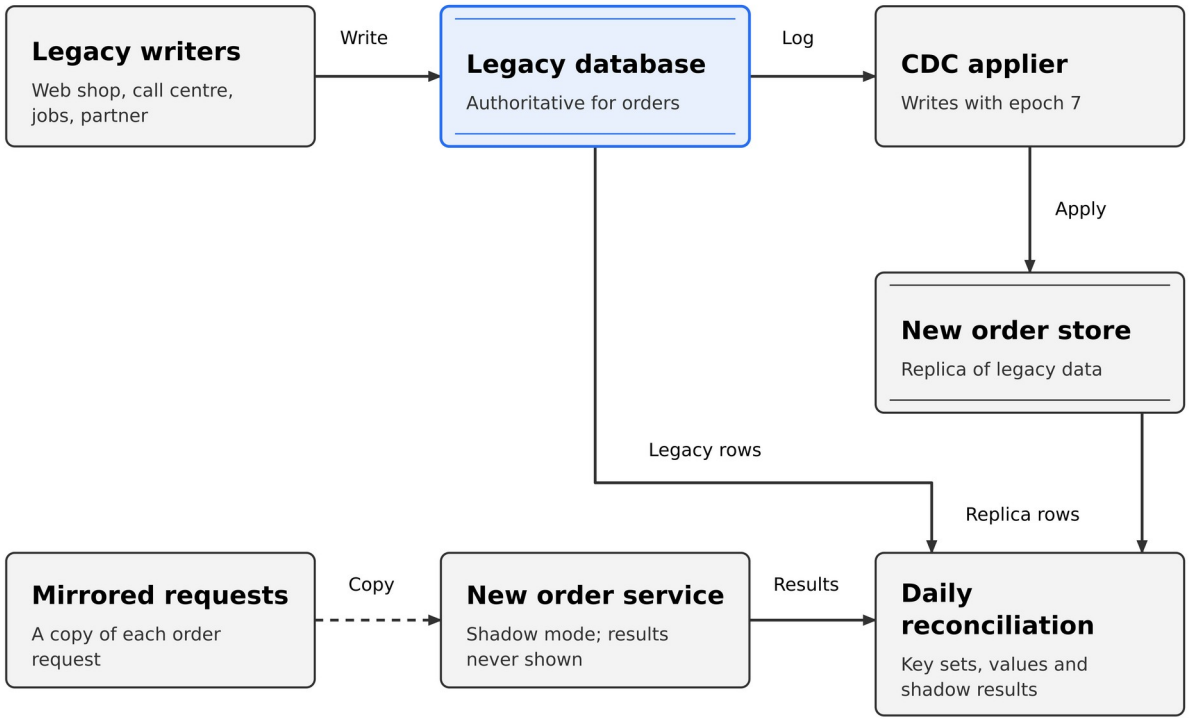


Figure 17 During the dual run, the legacy database stays authoritative, CDC fills the new store, and daily reconciliation compares both replicated data and shadow results.

Figure 17 shows the arrangement. Selected daily results of the data comparison were:

Day	Orders compared	Missing in target	Extra in target	Value differences	Cause and action
1	44,180	0	0	1,326	VAT rounding in mapping version 1; version 2 copies recorded totals
2	45,020	0	0	37	Marketplace orders with a blank currency; default rule agreed with finance
6	43,870	486	0	0	A marketplace partition missing from the CDC scope; scope fixed and backfilled
9	46,310	0	0	0	Month end; comparison waited until the status history lag cleared
12	44,950	0	3	0	Smoke-test orders from the internal route; test tenant now excluded
16 to 20	About 45,000 a day	0	0	0	Five clean days in a row

The exit criterion was five consecutive business days with no unexplained difference and a daily p99 lag under 10 seconds, which days 16 to 20 met. The only day above it was the month end, day 9, with a p99 of 12 seconds. The shadow comparison found a different kind of problem: on day 1, warehouse assignment differed for 2.1 percent of orders, because the new service chose the nearest warehouse while the legacy platform chose the warehouse with stock. The product owner kept the stock-first rule for the first release.

The reconciliation query

The daily comparison checks key sets and values, not just counts, and it must respect the replication boundary. Each legacy order records the log position of its last change, and the target knows the position up to which the applier has committed changes. An order changed after that position will differ in the target for a few seconds without anything being wrong. The listing runs the comparison in SQLite on five legacy orders. The target has applied the log up to position 1000:

```
import sqlite3

db = sqlite3.connect(':memory:')
db.executescript("""
CREATE TABLE legacy_order (order_id TEXT PRIMARY KEY, status TEXT,
                           total_cents INTEGER, last_lsn INTEGER);
CREATE TABLE target_order (order_id TEXT PRIMARY KEY, status TEXT,
                           total_cents INTEGER);
INSERT INTO legacy_order VALUES ('O-201', 'shipped', 12500, 880),
    ('O-202', 'open', 4999, 905), ('O-203', 'open', 800, 910),
    ('O-204', 'packed', 2300, 1003), ('O-206', 'open', 1500, 1004);
INSERT INTO target_order VALUES ('O-201', 'shipped', 12500),
    ('O-202', 'open', 5000), ('O-204', 'open', 2300), ('O-205', 'open', 800);
""")
RECONCILE = """
SELECT 'missing in target' AS issue, l.order_id
FROM legacy_order AS l LEFT JOIN target_order AS t ON t.order_id = l.order_id
WHERE t.order_id IS NULL AND l.last_lsn <= :applied
UNION ALL
SELECT 'extra in target', t.order_id
FROM target_order AS t LEFT JOIN legacy_order AS l ON l.order_id = t.order_id
WHERE l.order_id IS NULL
UNION ALL
SELECT 'values differ', l.order_id
FROM legacy_order AS l JOIN target_order AS t ON t.order_id = l.order_id
WHERE (l.status IS NOT t.status OR l.total_cents IS NOT t.total_cents)
  AND l.last_lsn <= :applied
ORDER BY 1, 2
"""

def reconcile(label: str, applied: int) -> None:
    issues = db.execute(RECONCILE, {'applied': applied}).fetchall()
    later = db.execute('SELECT count(*) FROM legacy_order WHERE last_lsn > ?',
                       (applied,)).fetchone()[0]
    print(f'{label}: {len(issues)} differences, {later} orders left for the next run')
    for issue, order_id in issues:
        print(f'  {order_id}: {issue}')

reconcile('ignoring the CDC position', applied=10**9)
reconcile('up to applied position 1000', applied=1000)
```

It prints:

```
ignoring the CDC position: 5 differences, 0 orders left for the next run
  O-205: extra in target
  O-203: missing in target
  O-206: missing in target
  O-202: values differ
  O-204: values differ
up to applied position 1000: 3 differences, 2 orders left for the next run
  O-205: extra in target
  O-203: missing in target
  O-202: values differ
```

Without the boundary, two of the five differences are false: O-204 was packed and O-206 created after position 1000, so the target has not applied them yet. With the boundary, three real differences remain. O-205 is a smoke-test order that reached the target by another route, O-203 was lost, and O-202's total differs by one cent. The two deferred orders are compared in the next run, once the applier has passed their positions. SQLite's `IS NOT` compares values safely when either side is null; in PostgreSQL the same comparison is written `IS DISTINCT FROM`. In production the query also applies the recorded mapping version to the legacy values before comparing them.

Fence every writer, at the destination

A **write fence** stops an old authority from changing data after authority has moved. Orion's fence has two halves.

On the legacy side, the cutover begins by revoking insert, update and delete rights on the order tables from the application, partner, batch and support accounts and ending their open sessions. A probe write is

then attempted with each account and must fail, and only then is the final log position recorded. Blocking the web shop alone would leave the partner, the support script and the month-end job free to write.

On the destination side, the new store must accept writes from exactly one authority at a time: the CDC applier before cutover and the new order service after it. Orion enforces this with an **authority epoch**, a number that increases each time authority moves. Privileges come first. A dedicated role, `orion_fence`, owns the order tables, the authority table and the check; the writers, `orion_applier` and `orion_service`, receive only row privileges on the order tables:

```
-- Run by an administrator. orion_fence owns the tables and the fence;
-- writers get row privileges on the order tables and nothing on the fence.
CREATE ROLE orion_fence NOLOGIN;
CREATE ROLE orion_applier LOGIN;
CREATE ROLE orion_service LOGIN;
GRANT CREATE ON SCHEMA public TO orion_fence;
GRANT pg_create_subscription TO orion_applier;
ALTER ROLE orion_applier SET orion.writer_epoch = '7';   -- read by the apply worker
```

A trigger written in PL/pgSQL, PostgreSQL's procedural language, then checks the epoch on every row written. CDC writes order lines and status history as well as orders, so the triggers are attached to all three replicated tables:

```
-- Run as orion_fence, which owns the order tables created beforehand.
CREATE TABLE write_authority (
  domain text PRIMARY KEY,
  epoch  integer NOT NULL
);
INSERT INTO write_authority VALUES ('orders', 7);    -- 7 = replication from legacy
REVOKE ALL ON write_authority FROM PUBLIC;           -- writers cannot touch it

CREATE FUNCTION check_writer_epoch() RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER                    -- reads the fence as orion_fence, not as the writer
SET search_path = public, pg_temp  -- apply workers run with an empty search_path
AS $$
DECLARE
  active integer;
BEGIN
  SELECT epoch INTO active FROM write_authority
  WHERE domain = TG_ARGV[0]
  FOR SHARE;                        -- an epoch change waits for this transaction
  IF current_setting('orion.writer_epoch', true) IS DISTINCT FROM active::text THEN
    RAISE EXCEPTION 'writer epoch % rejected on %; active epoch is %',
      current_setting('orion.writer_epoch', true), TG_TABLE_NAME, active;
  END IF;
  IF TG_LEVEL = 'ROW' THEN
    RETURN COALESCE(NEW, OLD);     -- let the checked row through
  END IF;
  RETURN NULL;
END $$;
REVOKE ALL ON FUNCTION check_writer_epoch() FROM PUBLIC;

DO $$
DECLARE
  t text;
BEGIN
  FOREACH t IN ARRAY ARRAY['orders', 'order_lines', 'status_history'] LOOP
    -- Row triggers: logical replication apply workers fire row triggers only.
    EXECUTE format('CREATE TRIGGER %I BEFORE INSERT OR UPDATE OR DELETE ON %I
                    FOR EACH ROW EXECUTE FUNCTION check_writer_epoch(%L)',
                   t || '_epoch_fence', t, 'orders');
    EXECUTE format('CREATE TRIGGER %I BEFORE TRUNCATE ON %I
                    FOR EACH STATEMENT EXECUTE FUNCTION check_writer_epoch(%L)',
                   t || '_epoch_truncate', t, 'orders');
    -- ALWAYS: fire even when the applier sets session_replication_role = replica
    EXECUTE format('ALTER TABLE %I ENABLE ALWAYS TRIGGER %I', t, t || '_epoch_fence');
    EXECUTE format('ALTER TABLE %I ENABLE ALWAYS TRIGGER %I',
                   t, t || '_epoch_truncate');
    EXECUTE format('GRANT SELECT, INSERT, UPDATE, DELETE ON %I
                    TO orion_applier, orion_service', t);
  END LOOP;
END $$;
```

Each writer declares its epoch in its session. The new order service runs `SET orion.writer_epoch = '8'` when it connects. An applier that cannot run its own commands, such as PostgreSQL's logical replication apply worker, gets the setting from its role, as in the first listing; a worker reads it when it starts. The subscription is owned by `orion_applier` and created with `run_as_owner = true`, so changes are applied with the applier's own row privileges instead of the table owner's [37]. At cutover, an operator acting as

`orion_fence` runs `UPDATE write_authority SET epoch = 8 WHERE domain = 'orders'` and revokes the applier's row privileges. Because every write holds a share lock on the authority row, the update waits until in-flight applier transactions finish.

The check function is `SECURITY DEFINER`, so it reads and share-locks the authority row with `orion_fence`'s rights. A locking read such as `FOR SHARE` needs update rights on the table, so a check running with the writer's own rights would force Orion to give every writer, stale ones included, the power to change the epoch. Instead the writers hold no rights on `write_authority` at all, and `REVOKE ... FROM PUBLIC` keeps it that way. The pinned `search_path` puts `pg_temp` last, so a writer cannot plant a temporary table named `write_authority` for the definer's function to read, and only `orion_fence` may create objects in the schema.

Three more details come from how PostgreSQL's logical replication apply worker behaves:

- It runs with `session_replication_role` set to `replica`, so that ordinary triggers do not fire a second time on the copy [38]. A trigger enabled with `ENABLE ALWAYS` fires whatever the replication role [39].
- It fires row-level triggers but not statement-level ones, so the fence on inserts, updates and deletes must be `FOR EACH ROW` [38]. `TRUNCATE` has only a statement-level trigger, which is kept for the other writers.
- It runs with an empty `search_path`, a security measure introduced in 2020 [40]. In the test, every replicated write failed until the function pinned its own `search_path`, because `write_authority` could not be found.

Tested in PostgreSQL 18 with a real publication on a second instance and the subscription owned by `orion_applier`, the apply worker replicated inserts and updates to all three tables at epoch 7 and was stopped once the epoch became 8. An earlier version of the fence with statement-level triggers let the same stale change through. Both writer roles were refused when they tried to update `write_authority` or lock it with `FOR SHARE`. Further sessions on the new store showed the rest of the behaviour:

- before cutover, a session with epoch 7 wrote to all three tables, and the new service with epoch 8 was rejected on each;
- the epoch change waited for the applier's open transaction to commit;
- after cutover, epoch 7 was rejected and the new service wrote;
- a session that declared no epoch was always rejected, and so was a `TRUNCATE` from a stale writer;
- a stale writer that set `session_replication_role` to `replica` was still rejected; with the trigger merely enabled, the same write was accepted.

This is Kleppmann's fencing token, checked by the system that stores the data [11]. The epoch stops delayed or forgotten writers acting on old authority, but it cannot stop a misconfigured writer from declaring the new epoch. The applier and the new service therefore also use separate database roles, and the applier loses its write privilege at cutover. If an applier cannot declare an epoch at all, enforce the check in the applier's own configuration and rely on the revoked privilege. The partner's side of the fence is the epoch on dispatch commands agreed in the contract.

Recovery before and after customer writes

The point of no easy return comes when customers start writing to the new system, so Orion delays it. During the cutover, public routing stays on the legacy platform, which keeps serving order status, while the new service accepts only smoke-test orders for a test tenant through an internal route. If the go/no-go decision is "no", the reverse path takes six minutes:

1. as `orion_fence`, raise the target's epoch to 9, set `orion_applier`'s role setting to epoch 9 and restore its row privileges on the order tables;
2. restart the subscription, with `ALTER SUBSCRIPTION ... DISABLE` and then `ENABLE`, so that a new apply worker starts with the new setting;
3. restore the legacy accounts' write privileges, so the legacy platform is authoritative again;
4. let CDC resume from the final log position, and delete the test-tenant orders from the target in an operator session that declares epoch 9.

After public routing switches, customer orders exist only in the new store. Routing back would hide them and restoring a backup would delete them. Orion therefore runs reverse CDC for fourteen days after the switch, replicating every change from the new store back into the legacy database, while the two new order states stay disabled by a feature flag so that every order can be represented in the legacy schema. If a severe defect appears, Orion can fence the new store and return authority to the legacy platform

without losing an order. On day fifteen, reverse CDC stops, the new states are enabled, and recovery means fixing forward.

The way back has a price. About 300 orders a week need “awaiting customs”; for fourteen days the customs desk tracks them with the legacy “on hold” status and a note, at about ten extra minutes per order, or 100 hours in total. Orion chose fourteen days because the dual run found its last new defect on day 12. A shorter period would save desk time but leave part of the first fortnight in production without a tested way back.

> **Common mistake.** Promising “rollback” without saying what happens to orders created after the switch. Rolling back routing, restoring a database and returning authority with its data are three different operations with different losses.

Exercises

13.1 A team takes a snapshot of the legacy database at 01:00, at log position P0, and starts CDC at 01:20 from the current position P1. Both jobs report success. Which changes are missing from the target, and why does neither job notice? What would happen instead if CDC were started from a position before P0, and what property must the applier then have?

13.2 The initial load takes 8 hours, changes arrive at 17.4 per second and the applier processes 250 row changes per second in ordinary transactions. Calculate the queued changes after the load and the catch-up time.

13.3 Write a fence checklist for the five writers in Chapter 12’s inventory. For each, state the control and the evidence that it is stopped.

13.4 In the listing, the legacy platform deletes order O-201 at log position 1005, after the target’s applied position of 1000. What does the boundary-aware query now report, and why is the result false? Change the design so that deletions respect the boundary too.

13.5 Ten days after the public switch, while reverse CDC is running, a defect starts corrupting VAT on new orders. Compare returning authority to the legacy platform with fixing forward. State the information you need before deciding.

Key points

- A snapshot and a change stream must share one log position, and the catch-up time follows from the drain model.
- Lag maxima, not averages, set the cutover preconditions, and a large transaction delays every change behind it in the stream.
- A dual run with daily key-set and value reconciliation finds mapping and scope defects before customers do.
- The destination must enforce the fence: an authority epoch checked by the store rejects stale writers, alongside revoked privileges.
- Keeping customers on the old system until go/no-go, and replicating back for a fixed period afterwards, keeps a tested way back.

CHAPTER 14

Cutover rehearsal and modernisation review

Learning objectives

After this chapter you should be able to:

- Analyse rehearsal timings to find the latest safe go/no-go decision point.
- Design preconditions that remove the causes of slow cutover steps.
- Evaluate a change card that delays a partner dependency, and choose a response with explicit conditions.
- Define the evidence needed to retire the replaced responsibility and to claim the promised benefits.

The cutover sequence

Orion's cutover runs inside the 45-minute Sunday window in seven steps:

1. Fence the legacy writers: revoke rights, end sessions and confirm that probe writes fail.
2. Let CDC apply everything up to the final log position.
3. Run the final reconciliation.
4. Raise the target's authority epoch to 8 and open the internal route.
5. Run smoke tests through the internal route, with test-tenant orders and a test call to the partner.
6. Make the go/no-go decision.
7. On "go", switch public routing to the new service and verify it.

Customers can look up order status throughout, because public routing stays on the legacy platform until step 7.

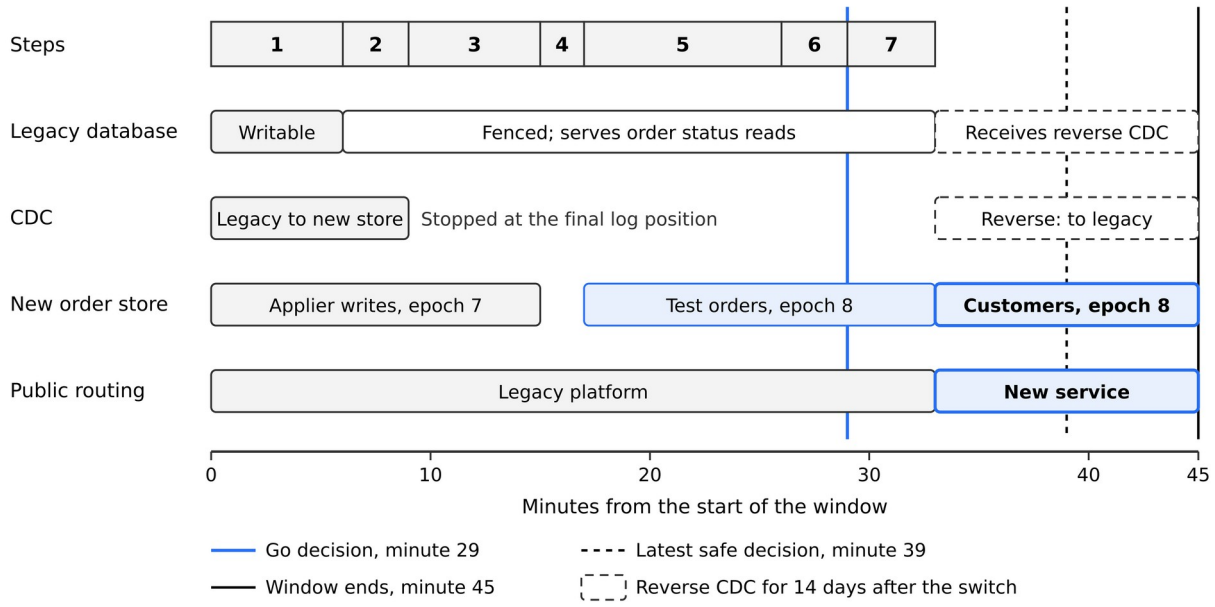


Figure 18 Orion's final cutover plan: public routing stays on the legacy platform until the go decision, and the latest safe decision falls at minute 39 of the 45-minute window. Numbers refer to the steps listed in the text.

Figure 18 shows the final plan that the rehearsal analysis below arrives at, including the replication that runs in reverse after the switch.

Three rehearsals

Orion rehearsed the cutover three times on a full-size copy of production. The durations, in minutes, were:

Step	Rehearsal 1	Rehearsal 2	Rehearsal 3	Worst observed
1. Fence legacy writers	6	4	4	6
2. Drain to final position	3	9	2	9
3. Final reconciliation	11	10	9	11
4. Epoch and internal route	2	2	1	2
5. Internal smoke tests	9	8	8	9
6. Go/no-go decision	3	3	3	3
7. Switch public routing	4	3	3	4
Total	38	39	30	44

Every rehearsal fitted the window, but the real question is whether a “no” can still be carried out in time. The latest safe decision point is the window minus the time needed to reverse: the rehearsed reverse path from Chapter 13 takes six minutes, so the decision must be made by minute 45 – 6 = 39. Three rehearsals then raise a question that a single timing cannot answer: how much do the steps vary, and why? A “go” needs at most four minutes to switch routing, which would allow a decision as late as minute 41, so the reverse path is the binding constraint.

If every step took its worst observed time, the decision would finish at minute 6 + 9 + 11 + 2 + 9 + 3 = 40, one minute too late. The cause is visible in the table: rehearsal 2 started a few hours after a month-end job, so step 2 had to apply its 600,000-row transaction, about 500 seconds at the applier’s large-transaction rate, plus the changes queued behind it: nine minutes in all. The lag precondition from Chapter 13, never within 24 hours of the month-end job, removes that case, and with a drain of at most three minutes the decision finishes at minute 34.

One more change buys margin. Running the full key-set comparison in the hour before the window, and reconciling only the changes since then during step 3, should cut that step to about six minutes; rehearsal 4 must confirm it. The revised plan reaches the decision at minute 29, ten minutes before the latest safe point, and completes the switch at minute 33. The decision rule is written down in advance: if smoke tests have not passed by minute 36, the answer is no-go, because the decision itself takes three minutes.

> **In practice.** Summing the worst observed time for every step is conservative when steps vary independently. But three rehearsals are a small sample, and steps that share a cause, such as month-end load, can be slow together. Remove shared causes with preconditions rather than hoping for slack.

Readiness beyond the timeline

A cutover can run perfectly to time and still fail. The release review also checks:

- the new service’s capacity at the weekday peak, not just at 02:00 on a Sunday;
- production service identities and permissions, since a load that works under an administrator account says nothing about the production account;
- operator access, monitoring and alerting for the new service;
- routing behaviour, for example lowering the DNS time-to-live a day in advance;
- support runbooks and the customer message for the 45-minute pause;
- the partner’s readiness, including the joint rehearsal.

Test data must look like production. For example, twelve million near-identical test rows can hide old character encodings, very long text fields and a few very large customer accounts. Rehearse failures at the points where the response differs: CDC stalls after the fence, reconciliation finds a mismatch, a smoke test fails on the partner call, or routing has switched but the new service rejects production identities. Before the switch, each leads to the six-minute reverse path; after it, to the reverse-CDC procedure or a fix forward.

A change card: the partner slips

Orion's plan counts weeks back from the hosting deadline, so week –10 means ten weeks before the hosting agreement ends. Under the partner contract from Chapter 12, the partner delivers its status API in week –14, after ten weeks of development. Two weeks of joint testing follow, ending with rehearsal 4, the joint cutover rehearsal, in week –12. The cutover is planned for the Sunday of week –10, which leaves nine more Sundays for a second attempt after a no-go. The current host will extend the agreement by three months for €120,000, but only with notice given by week –8.

In week –16, six weeks before the planned cutover, Orion receives this change card.

Field	Content
Card	CC-O1, partner API delay
New fact	The partner's status API will arrive six weeks late, in week –8
Source	Partner project manager, confirmed in writing
Assumptions affected	The partner status API is available for joint testing and rehearsal 4
Decisions affected	The cutover date; the partner integration in the plan for each phase
Question	Does Orion delay, cut over without the API, or extend hosting?

Three responses are credible:

Milestone	1. Delay the cutover	2. File fallback	3. Extend hosting
Partner status API	Week –8	Week –8, adopted after cutover	Week –8
Rehearsal 4	Week –6, with the API	Week –12, with a status file	Week –6, with the API
Cutover	Week –5	Week –10, as planned	Week –5 or later
Sundays left after a no-go	4	9	4, plus three months
Extension decision	Week –8, before any joint test	Week –8, after the cutover	Week –8, €120,000

1. **Delay the cutover until the API is ready.** Joint testing moves to weeks –8 to –6, and the cutover to week –5, after a week to fix what rehearsal 4 finds. Four Sundays remain for a second attempt, which is tight but workable. The weakness is the notice date: Orion must decide on the €120,000 extension in week –8, the week the API is due, before it has tested the API at all. A further slip of even two weeks would leave no room to recover.
2. **Cut over on time using the file fallback.** The partner sends a status file every 15 minutes, and Orion's adapter, estimated at 15 engineer-days, turns each row into a status API call using the row's event ID. Rehearsal 4 stays in week –12 and uses a full day's status file. Status updates can take up to about 20 minutes instead of 15. The partner's database account is revoked at the fence in any case. Because the cutover in week –10 comes before the notice date, a no-go still leaves time to buy the extension.
3. **Extend the hosting agreement.** €120,000 buys three months on the current host but no progress, and the migration still depends on the delayed API. This is a different option from the temporary rehosting of Chapter 12, which would move the legacy platform to new hosting and remains a fallback until its decision date at the end of month six, in week –13, three weeks after this card; the extension leaves the platform where it is.

Decision record ADR-O07 chooses option 2 under four conditions:

- rehearsal 4 processes a full day's status file through the adapter;
- the business owner agrees in writing to a temporary 20-minute status objective;
- Orion moves the partner to the API within four weeks of its delivery, by week –4;
- if the week –10 cutover is a no-go whose cause cannot be fixed within a week, Orion gives the extension notice by week –8.

The authority epoch on dispatch commands is unaffected, because dispatch flows from Orion to the partner while the fallback carries status the other way. The fallback term negotiated in Chapter 12 turns

the partner's delay into a bounded change with a known cost, about 15 engineer-days and five minutes of status delay.

Retire the old responsibility

The migration is complete only when the old responsibility is gone. Orion's retirement checklist includes:

- no reads or writes on the legacy order tables for 30 days, confirmed by the audit log;
- all legacy accounts disabled and scheduled jobs removed through controlled changes;
- the support team's auto-close script rebuilt as a scheduled function of the new service, with the support manager's approval of its 90-day rule;
- historical records archived according to the retention policy;
- hosting and licences cancelled only after all of the above.

Measure the benefits that justified the programme. If hosting cost falls but two people now reconcile data by hand each day, the cost has moved rather than disappeared. The technical cutover is a milestone; the business improvement is a separate outcome with its own measurements.

Compare the alternatives again

A good review may change the original recommendation, and it should record why. For Orion, direct replacement is now clearly ruled out, by the window and by the writers that only a long observation revealed. Temporary rehosting remained a live fallback until the end of month six, when the clean dual run removed the need for it, and phased migration with CDC is confirmed. Keep the earlier decision records and supersede them explicitly, so that a later reader can see which facts changed the ranking.

Studio assessment

Submit a migration dossier containing the writer inventory with confidence levels, the strategy comparison, the authority table for each phase, the CDC design with its lag preconditions, the mapping rules and dual-run results, the fence design, the rehearsal analysis with its latest decision point, the response to change card CC-O1, and the recovery and retirement plans. A strong dossier states what must be true before each step and how Orion stays operable if that condition fails.

Exercises

14.1 The rehearsed reverse path takes eight minutes instead of six. Calculate the latest safe decision point. If smoke tests then take 14 minutes under the revised plan, does the plan still fit?

14.2 Suppose each of steps 1 to 6 varies independently and is equally likely to take any of its three rehearsal times. Calculate the probability that all six take their worst observed times in the same cutover. What does the result say about the plan that sums the worst times, and in which two situations would that reasoning fail?

14.3 Draft a short decision notice for change card CC-O1, stating the option chosen, its conditions, the owner and the date of the next review.

14.4 Design a go/no-go gate of five checks for step 6. For each check, state the evidence and who confirms it.

14.5 A manager proposes skipping the internal smoke tests to save nine minutes and switching public routing straight after reconciliation. Evaluate the proposal.

Key points

- The latest safe decision point comes from the time needed to reverse, not from the time needed to finish.
- Rehearsal variance has causes, and preconditions that remove them are worth more than added slack.
- Keeping public routing on the old system until go/no-go makes a "no" cheap and fast.
- A fallback negotiated early can turn a partner delay into a bounded change.
- A migration ends with evidence-based retirement of the old responsibility and measured business benefits.

CHAPTER 15

Architecture review, communication and judgement

Learning objectives

After this chapter you should be able to:

- Design a review around one bounded decision, with the right participants and a short pre-read.
- Classify findings by severity and separate them from preferences.
- Communicate one decision to executives and to engineers without changing its substance.
- Evaluate whether a finding is closed, accepted as residual risk or deferred for investigation.

Review a bounded decision

A useful review names the decision, the architecture version and the evidence available. “Review the platform” is too broad for one meeting. “Decide whether Meridian’s first release can rely on manual review of CRM B handoffs during recalls” gives the participants a concrete responsibility. Invite the people who own each part of it: the sponsor, who owns the business promise; the technical lead; the support lead, who owns the manual review queue; and a security or risk owner where needed.

Send a short decision brief beforehand, such as the one at the end of Chapter 5, with detailed diagrams and experiment records as supporting material. The meeting’s time is for challenging assumptions, not for a long presentation.

A review ends in one of four outcomes: proceed, proceed with conditions, investigate further, or reject. Each condition needs an owner and checkable closure evidence. “Security to review” names an activity; “Cross-tenant download, search and export tests pass for release 1.4, and the policy owner approves any exception” can be checked.

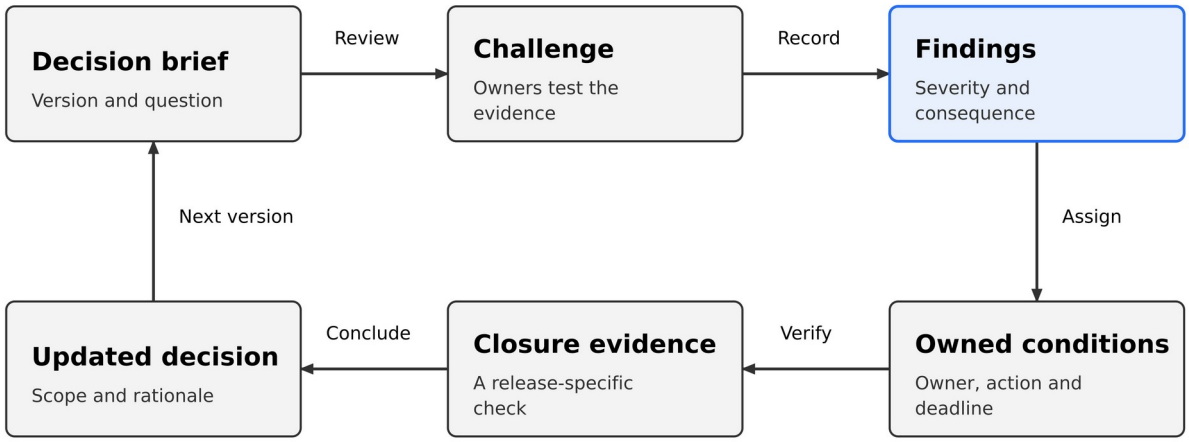


Figure 19 Review findings become owned conditions with closure evidence and feed a versioned decision.

Figure 19 shows the cycle from decision brief to updated decision.

Separate findings from preferences

A **finding** identifies the requirement affected, the defect or uncertainty, its consequence and the evidence for it. Severity follows the consequence:

Severity	Typical consequence	Effect on release
Critical	Accepted transactions lost, stock oversold or data disclosed to the wrong tenant	Blocks release
Major	An important claim, such as a recovery objective, is unsupported	Blocks release unless the accountable owner accepts it
Minor	A diagram label or document is ambiguous	Fixed in the normal course of work
Enhancement	An optional improvement	Recorded for later

A register makes the status of each finding visible. Part of the register from Meridian's quota review reads:

ID	Finding	Severity	Owner	Closure evidence	Status
F-11	Each worker replica runs its own rate limiter, so three replicas could send 24 calls per second against an 8-call quota	Major	Technical lead	Shared limiter; load test with three replicas stays at or below 8 calls per second	Closed
F-12	Manual review of CRM B handoffs during a recall needs about 2.4 agent hours per surge	Major	Business unit B manager	Staffing plan approved for declared recalls	Accepted residual risk
F-13	The vendor has not confirmed that a temporary uplift can be switched on within an hour	Major	Sponsor	Written confirmation from the vendor	Deferred investigation, due 30 April
F-14	The container view calls the worker a "sync service"	Minor	Architect	Diagram updated	Closed

Preferences need different handling. If a reviewer prefers another database, ask which requirement the change would improve and what evidence supports that. Equally, never downgrade a broken invariant because the team likes its current stack.

Treat risk matrices with care. Multiplying a likelihood score by an impact score can structure a discussion, but the product is not a measured probability. For example, risk X scores 3 × 4 = 12 and risk Y scores 2 × 5 = 10, so X ranks first; if a reasonable reviewer rates Y's likelihood as 3, Y scores 15 and the order flips. Record the reasoning behind each score and check whether plausible changes alter the priority.

Communicate the same decision at two levels

Executives need the business consequence, the options, the cost range, the uncertainty and the decision required of them. Engineers need ownership, contracts, failure states, configuration and test results. Both should receive two views of the same argument: an unconditional promise in the executive summary, with its conditions hidden in an appendix, is a misrepresentation.

For example, Meridian's executive message could read: "CRM B's new limit cannot meet the thirty-minute recall promise. We can buy capacity during recalls for about €1,200 a year, buy it permanently for €14,400, or accept 75 minutes. Customer requests are safe in every option." The technical appendix shows the backlog equation, the quota's scope and the shared rate limiter, and nothing in it contradicts the summary.

Diagrams should support what is said. Start with the business journey, then show the boundary that delivers the key guarantee; when discussing resilience, show the failure sequence.

Disagree without inventing certainty

When challenged, restate the concern and decide which of three kinds it is:

- **A new fact**, such as a vendor's new rate limit, may change what is feasible.

- **A different preference**, such as a stronger wish for low cost, may change the weights in a comparison.
- **A different interpretation** of a requirement may show that the design solves the wrong problem, as when Aster's team first read "confirmed" as "checkout received".

If you do not know the answer, say so and propose the smallest investigation that could settle it. Do not fill the gap with a confident claim about a product. Equally, avoid deferring every ordinary choice: a reversible, low-consequence choice can proceed under a recorded assumption, while a high-impact uncertainty blocks release until it is resolved.

> **In practice.** "I don't know yet; here is the test that would tell us, and it takes two days" is a strong answer in a review. It turns an argument into a plan.

Record learning after the review

Keep the finding register current. When a condition closes, record what was tested and which design version the test applies to, because a configuration change can invalidate earlier results. If a finding is accepted as residual risk, name the accountable owner and a review date rather than marking it "fixed". After each studio, compare your revised design with your first version: which assumptions changed, which decisions held, and was each investigation worth its cost?

Exercises

15.1 Rewrite "the architecture is scalable and secure" as a bounded review conclusion that states the scope, the evidence and two open conditions.

15.2 A senior reviewer insists on microservices without naming a requirement that they would improve. Draft a respectful response that brings the discussion back to evidence.

15.3 Classify these Orion findings as closed, accepted residual risk or deferred investigation, and state the evidence each status needs: the `ops_batch` script; the partner's API delay; the DNS time-to-live not yet lowered.

15.4 Write an executive summary of at most 80 words, and a matching engineering summary, for Aster's choice of bounded admission.

15.5 Risk P scores likelihood 4 and impact 3; risk Q scores likelihood 2 and impact 5. Rank them. Then show how one plausible change in a single score reverses the ranking, and explain what the review should record.

Key points

- A review should address one bounded decision, with the owners of each part present and a short pre-read.
- Findings state the requirement, defect, consequence and evidence, and their severity follows the consequence.
- Conditions need owners and checkable closure evidence; residual risk needs a named owner and a review date.
- Executive and engineering messages are two views of the same argument and must never contradict each other.
- Saying what is unknown, with a plan to find out, is stronger than an invented certainty.

CHAPTER 16

Portfolio assessment and professional transfer

Learning objectives

After this chapter you should be able to:

- Assemble a portfolio that shows reasoning, evidence and revision across the four studios.
- Apply a common rubric consistently to your own dossiers and to others'.
- Compare how one principle, such as durable intent, changes from one context to another.
- Plan supervised experience that closes a specific gap in your practice.

The final portfolio

Choose your two strongest studio dossiers for detailed presentation, and write shorter comparative reports for the other two. Keep the briefs, assumptions, first designs, decision records, experiment records, findings and revisions, because the portfolio should show how you reasoned and improved the design. Each detailed dossier contains:

- a one-page decision brief and a register of requirements and assumptions;
- connected context, container, data, sequence and deployment views;
- at least three significant decision records;
- a failure and recovery design, with security and data lifecycle reasoning;
- a cost model with every input stated, and an experiment report;
- a statement of what remains unverified.

The comparative reports explain which principles transfer and which constraints change the answer. Durable intent is a good example. Meridian records it as an outbox row committed with the submission, Aster as an order state with a payment operation ID, and Orion as the authority epoch in `write_authority`, together with the cutover gate record, which state durably who may write and when that changed. Beacon's publication identifier serves a different purpose, explaining a past result rather than driving future work. The comparison should therefore name what each studio's durable record protects, and who acts on it next, instead of reusing one diagram for all four.

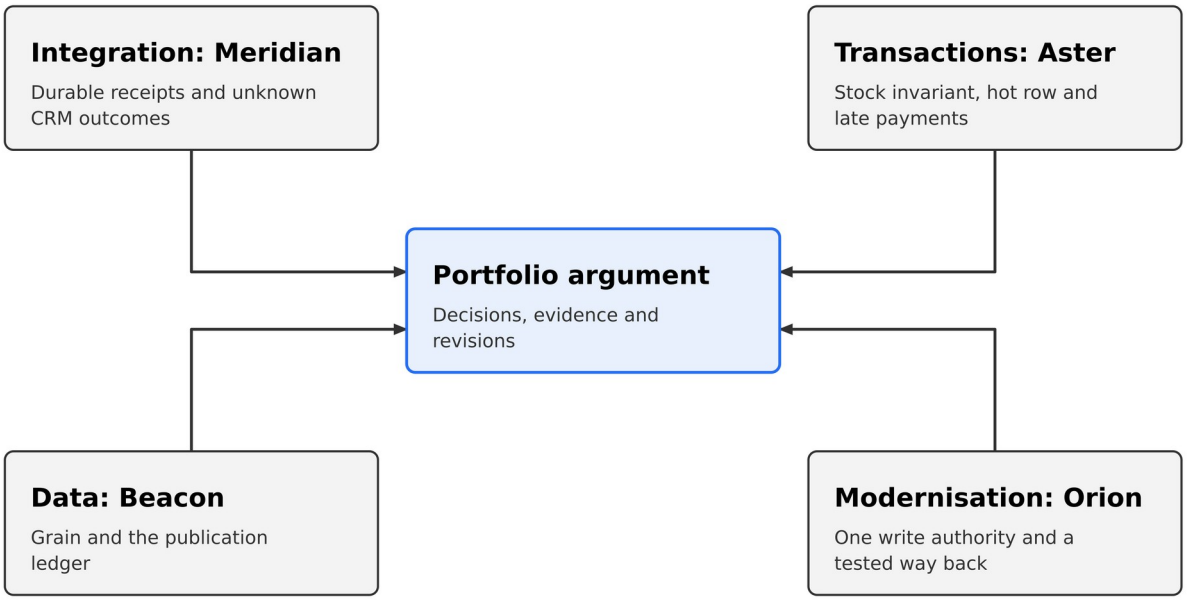


Figure 20 Each studio contributes a different kind of evidence to one portfolio argument.

Figure 20 summarises what each studio adds.

A common assessment rubric

Use the same rubric for every dossier. The points organise feedback and comparison; they do not certify professional competence. Award full credit to a well-argued alternative even when it differs from the worked design.

Dimension	Points	What the dossier must show
Problem and requirements	15	Clear outcomes, constraints, measurable scenarios and owners
Boundaries and invariants	20	Coherent authority, transactions, states and data contracts
Alternatives and judgement	15	At least two viable options and explicit tradeoffs
Failure, security and recovery	20	Threats, unknown outcomes, compensation and tested recovery claims
Evidence and research use	15	Reproducible methods, stated limits and accurate use of sources
Operation, economics and transition	10	Feasible support, explicit cost inputs and operable intermediate states
Communication and revision	5	A clear dossier and a traceable response to challenge

A strong submission scores at least 75 points and contains no unresolved critical flaw. A design that could knowingly oversell, lose accepted work or show one tenant's data to another must fix that defect, however well it is presented. When a constraint truly cannot be met, a clearly argued conclusion that it is infeasible can show strong judgement.

The oral defence

Prepare a ten-minute explanation followed by twenty minutes of questions. Open with the business problem and the promise, explain the two decisions that most shape the result, walk through one difficult failure sequence, and finish with the evidence and the conditions for proceeding.

During the questions, the examiner presents a change card, such as:

- a vendor halves its call quota;
- the payment provider shortens its idempotency window from 24 hours to one hour;
- a source starts correcting events that are 30 days old;
- a legacy writer reappears after cutover.

You have five minutes to name the assumptions affected and propose a response. For example, a one-hour payment window means Aster must resolve every unknown payment outcome within the hour or route it to people before the window closes, so the manual queue grows and the review triggers in the payment decision record fire. You need not redesign every component on the spot, but you must show which guarantees are now at risk. A careful answer that names an unresolved contract is stronger than a quick one that invents a contract. If no possible evidence could change your recommendation, it is probably a preference rather than an evaluated conclusion.

A research synthesis

Write a synthesis of about 1,500 words using at least four primary sources cited in this book. Compare the kind of knowledge each provides: a formal result, a design argument, a research method, a standard or operational guidance. For each source, explain one studio decision it informed and one claim it does not establish.

For example, Ongaro and Ousterhout's Raft algorithm keeps a replicated log consistent across servers [41]. Once a majority of servers has stored an entry, the entry is committed and survives the failure of the leader, and all servers apply the same commands in the same order. Transferring that result to Aster needs care, because Aster's PostgreSQL database does not run Raft. PostgreSQL's streaming replication is asynchronous by default, so a failover can lose transactions that the primary had already confirmed to clients, in proportion to the replication delay [42]. A reservation confirmed to a customer could then vanish. Aster gets a guarantee comparable to Raft's only if each commit waits for a standby. Three settings work together: `synchronous_standby_names` lists the standbys, for example `ANY 1 (s1, s2)` for a quorum of one; `synchronous_commit` keeps its default value `on`; and the failover procedure promotes a standby that has confirmed the latest commits. The guarantee has a price on the hot row. In a test on PostgreSQL 18, a committing transaction kept its row lock while it waited for the standby, so the standby round trip adds to the lock-hold time h. Raft also says nothing about payment compensation, which lies outside any

replicated log. Avoid long quotations, and do not let a bibliography stand in for your own transfer reasoning.

From course work to professional practice

Present the portfolio as course work on fictional cases, and explain the experiments you ran and the limits of their models. A reviewer trusts a portfolio more when it states plainly which results came from your own experiments and which came from worked examples.

Real architecture responsibility also means working with stakeholders under real operating constraints, which a book can prepare you for but not replace. Choose a next experience that closes a specific gap: take part in a design review, help maintain a set of decision records, trace a real incident across several dependencies, assist with a migration rehearsal, or review an integration contract with an experienced colleague. Record what you learned and how it changed one of your earlier assumptions.

The next book in this series moves from one solution to an organisation’s portfolio of systems, its capabilities, decision rights and transformation programme. Enterprise scope changes the scale and the stakeholders, while the need to make assumptions and evidence visible stays the same.

Exercises

16.1 Compare the role of durable state in two studios. Explain one shared principle and one important difference in who holds authority.

16.2 Apply the rubric to your first and your revised dossier for one studio. Identify the evidence behind the largest improvement, rather than only reporting a higher score.

16.3 Write a ninety-day development plan with one technical, one stakeholder and one operational objective. Give each an observable sign of progress.

16.4 Using Ongaro and Ousterhout’s description of Raft, explain what a Raft-replicated log guarantees about a committed entry after a leader failover. Then compare Aster’s PostgreSQL primary with asynchronous replication and with `synchronous_standby_names = 'ANY 1 (s1, s2)'`. For each, state what happens to a reservation confirmed just before the failover, what it costs on the hot row, and what is left to the application, such as a client whose reservation request timed out.

16.5 During your defence, the examiner states that, two days after Orion’s public switch, the audit log on the new order store shows 40 status updates from an unknown database account at 00:30. Within five minutes, list the assumptions and decisions in the Orion dossier that this puts at risk, and propose immediate and follow-up actions.

Key points

- A portfolio shows decisions, evidence and revision, not just finished diagrams.
- One rubric, applied consistently, lets different but well-argued designs earn equal credit.
- Principles such as durable intent transfer between contexts, while authority and external effects change.
- A good defence identifies which guarantees a new fact puts at risk, without inventing contracts.
- Course work prepares you to contribute to architecture practice; supervised experience and feedback build professional judgement.

Exercise answers and assessment guidance

Attempt each exercise before reading its answer. Calculations show their working and use the chapter's simplifying assumptions. Design and critique answers give one model answer and the criteria a marker should apply; a different answer earns equal credit when it meets the same criteria.

Chapter 1

1.1 One scenario observes acceptance: while one CRM is down for thirty minutes at the declared arrival rate, every valid submission receives a receipt that survives a restart of the portal, measured as zero lost receipts. A second observes recovery: after the CRM returns, the oldest pending handoff falls below one minute old within thirty minutes. Credit answers that give a stimulus, environment, response and threshold for each, and observe two different things.

1.2 P = 0.5 × 4 + 0.2 × 3 + 0.3 × 3 = 3.5; Q = 0.5 × 5 + 0.2 × 4 + 0.3 × 2 = 3.9; R = 0.5 × 2 + 0.2 × 3 + 0.3 × 5 = 3.1. Model paragraph: "Q scores highest but cannot export data in an open format, which is mandatory, so Q is ineligible. Of the eligible options, P (3.5) beats R (3.1), and we recommend P. Choosing Q would require the owner of the export requirement to withdraw it and accept the exit risk." Credit answers that exclude Q before ranking.

1.3 Context: the payment provider is an external authority. Container: the order coordinator owns the attempt. Sequence: the timeout follows a possible authorisation. Data: the provider operation ID and the state "payment unknown" are stored. Deployment: the reconciliation job runs with its own credentials. A contradiction a review could detect: the sequence retries with the same operation ID, but the data view has nowhere to store it.

1.4 "When we grow" cannot be observed, and the trigger names a solution rather than a condition. A measurable rewrite: "Revisit the single-service design if a second team needs to release order logic independently more than twice a month, or if order processing exceeds 70 percent CPU at peak for two consecutive months after optimisation."

1.5 (a) Constraint. (b) Preference. (c) Assumption, needing an owner, a test and a deadline. (d) Preference, unless the business sets a budget ceiling, which would make it a constraint.

Chapter 2

2.1 The net drain is 9 – 5 = 4 per second, so recovery takes 9,000 / 4 = 2,250 seconds, or 37.5 minutes. It fails the thirty-minute objective even before failures and slow calls are considered.

2.2 First, after 2,000 independent failure-free trials the 95 percent upper bound is 1 – 0.05^(1/2,000) ≈ 0.15 percent, fifteen times the 0.01 percent claimed; supporting 99.99 percent would need 29,956 clean trials. Second, the trials may share causes of failure, such as one build or one host, so they are less independent than the count suggests, and they may never have exercised the failures that matter, such as a CRM outage or a crash at a transaction boundary. Full credit needs the calculation and both defects.

2.3 Vary only the retry policy, for example fixed one-second retries against exponential backoff with jitter. Hold the build, data, CRM quota, offered workload and failure schedule constant. Observe distinct confirmed handoffs, unknown outcomes, oldest pending age, attempts and throttled calls per business unit. Inject a thirty-minute CRM outage and a worker crash after a CRM commit. Decision rule: both policies must lose nothing and create no duplicate in CRM A or C; among those, choose the one that clears the backlog sooner. Repeat runs and report the spread.

2.4 At 0.5 seconds, each cycle takes 2.5 seconds, so the rate is 60 / 2.5 = 24 per second; at 3 seconds, 60 / 5 = 12 per second. The test halves its own load as the system slows, so it cannot show behaviour under the intended 24 per second; an open workload is needed for that question.

2.5 W = L / λ = 1,875 / 3,000 per day = 0.625 days, or 15 hours on average. The average is well inside 24 hours, but the rule applies to every correction: with one nightly run, a correction that arrives just after a run starts waits about a day plus the run's duration, and longer if a run fails, so the worst case breaks the rule while the average looks safe. Little's law assumes a stable system over a long period, with arrivals and departures balanced; it gives the mean wait and says nothing about its spread or its maximum, which is what a deadline needs. Chapter 11 works through this case.

Chapter 3

3.1 CRM A keeps the external reference unique and returns the existing case for a repeat, so a retry is harmless. CRM B has no unique reference, so a repeat creates a second case, and its lagging index may

not show the first. A common adapter cannot create a guarantee that CRM B lacks; B needs a delayed search with bounded manual review, or a contract change.

3.2 Store the business unit with the submission at acceptance and use it for every delivery attempt; a change applies only to new submissions. Moving open requests needs a governed transfer that closes the case in the old CRM, records the new route and creates the case in the new CRM. Routing each retry by the customer's current business unit could deliver one request to two CRMs.

3.3 Logical isolation, with separate work selection, limits and alerts in one deployment, handles a slow CRM well. A separate deployment also contains process-level failures, such as an adapter that exhausts memory on very large responses. Neither helps when the shared database fails. Credit answers that tie the boundary to a named failure.

3.4 Timeouts: 36,000 × 0.008 = 288. Unresolved after the search: 288 × 0.25 = 72. Agent time: 72 × 5 = 360 minutes, or 6 hours per recall surge.

3.5 CRM C down, `retry_due`: "Received" for the first minute, then "Received, awaiting service team". CRM B in `manual_review`: "Received, awaiting service team", because the customer needs reassurance, not the internal state. Confirmed and resolved: the CRM's status, "Resolved". Credit answers that never show an error for a safely stored request and never show a CRM status before confirmation.

Chapter 4

4.1 With only `lease_owner = $2`, thread 1's confirm matches the row, because the handoff is `in_flight` and its owner is still w3. It records attempt 4's result while attempt 5 is still running, and attempt 5's confirm later matches nothing because the state is no longer `in_flight`. With `attempts = $3` added, thread 1 passes 4 while the row holds 5, so its update changes no row; it logs the late reply for comparison, and attempt 5 records its own result. CRM C ends with one case: both sends carry the submission ID as the idempotency key within 24 hours, so the second returns the first response. The token protects the portal's record, and CRM C's contract protects the CRM. Credit answers that separate the two protections.

4.2 Keep the record in `manual_review` with both candidate case IDs and the evidence for each. The agent compares them with the submission and applies one guarded update that moves the record to `confirmed` with the chosen case only if it is still in `manual_review`; a duplicate is closed through CRM B's own process. The system must not claim that the second case never existed or that the request failed. A late reply is compared with the recorded case ID and logged.

4.3 A model ADR chooses polling: one delivery path per receipt, modest volumes and a team that runs PostgreSQL. The broker alternative offers independent consumers and buffering but adds a relay, a second durability boundary and more operation. Consequences: work selection and fairness are built in SQL, and replay is limited to the outbox. Trigger: a second consumer needs replay, polling exceeds 10 percent of database CPU at peak, or a business unit's oldest pending handoff exceeds its target at the agreed peak. Credit a measurable trigger.

4.4 Either CRM B committed the case and its index has not caught up, or the create never committed. A search five minutes after the timeout cannot tell them apart, because the index has no stated maximum lag. Under ADR-M03 the handoff stays in `retry_due` with `next_action = 'lookup'` and `next_attempt_at` fifteen minutes after the timeout; if that lookup still finds no single match, the handoff moves to `manual_review`. A new create is never allowed after an unknown outcome with CRM B.

4.5 Make the count and the claim atomic: the claim transaction first locks a per-business-unit row, for example `SELECT ... FROM unit_limit WHERE business_unit = 'B' FOR UPDATE`, or takes an advisory lock, then counts B's `in_flight` handoffs and claims at most 8 minus that count. A second worker waits for the first to commit and then counts again. Without the lock, both could see seven in flight and both claim one.

Chapter 5

5.1 The backlog is 5 × 2,700 = 13,500 handoffs. Clearing it in 1,800 seconds while 5 arrive each second needs 5 + 13,500 / 1,800 = 12.5 successful creates per second. With 15 percent failures that is 12.5 / 0.85 ≈ 14.7 attempts per second, and with 2 calls of polling about 16.7, so the quota must be at least 17. With 16 calls and polling on, creates get 14 calls, 11.9 succeed, the net drain is 6.9 per second and recovery takes about 13,500 / 6.9 ≈ 1,957 seconds, or 33 minutes, which misses. With polling paused, 13.6 succeed, the net drain is 8.6 and recovery takes about 1,570 seconds, or 26 minutes, which meets it. A longer outage therefore makes the polling pause part of the promise, not an optimisation.

5.2 Attempts include retries and failed calls; the total across business units can hide CRM B's growing backlog; and completions may be counted at the worker rather than as confirmed CRM cases, so duplicates look like progress. Check successful completions against arrivals per business unit, the oldest pending age and the manual queue.

5.3 Request a temporary uplift for declared recalls, run one account-wide limiter and pause CRM B status polling during recovery. Every request stays durable; customers of business unit B see "Received, awaiting service team" for up to about 16 minutes after an outage, and support needs a script explaining this. Until the uplift is confirmed, publish a 75-minute objective for B with the sponsor's approval. Discarding or failing accepted requests is never valid.

5.4 Creates get 12 – 2 = 10 calls per second, of which 9 succeed; the net drain is 9 – 5 = 4 per second, so recovery takes 9,000 / 4 = 2,250 seconds, or 37.5 minutes, which fails. Pausing polling would give 10.8 successful creates per second and about 1,552 seconds, or 26 minutes, which meets it.

5.5 The receipt shows that the portal stored the request, and a confirmed handoff shows that the CRM holds it; neither shows that an agent can act on it. Following Saltzer, Reed and Clark, that check can be completed only at the endpoint, the business unit's working queue, for example by a daily reconciliation of submissions against open CRM cases. The outbox and retries are valuable performance enhancements, not the end-to-end guarantee.

Chapter 6

6.1 After reserving three: A = 1,997, R = 3, C = 0. After confirming two: A = 1,997, R = 1, C = 2. After releasing one: A = 1,998, R = 0, C = 2. The sum is 2,000 at every step and nothing is negative.

6.2 The promoted product still receives 350 attempts per second on its one allocation row, and every one of them meets the same row lock however the other products are spread. Useful responses act on that row: shorter lock-hold time, bounded admission or allocation buckets.

6.3 The ceiling is 1 / 0.0025 = 400 per second. The offered 350 per second gives a utilisation of 350 × 0.0025 = 0.875, so the row keeps up on average, but waiting grows sharply near saturation and any burst above 400 builds a queue. If every attempt succeeds, the units last 2,000 / 350 ≈ 5.7 seconds.

6.4 Utilisation is 200 × 0.0015 = 0.3, so the row is locked 30 percent of the time. With random arrivals, about 30 percent of attempts find it locked; under REPEATABLE READ each of those waits and then fails with SQLSTATE 40001 once the holder commits, even when units remain. If every failure retries until it succeeds, the attempt rate is about 200 / (1 – 0.3) ≈ 286 per second, and each failed attempt has held a connection while waiting. Under READ COMMITTED the second transaction waits, re-evaluates `available >= 1` against the newest row version and either takes a unit or changes nothing, so it can never oversell and needs no retry.

6.5 The event carries the order ID, an event ID and version, the seller scope, the committed reservation IDs, the verified authorisation reference and the confirmation time. It means that invariant I4 holds; a checkout request entering a queue never produces it. Consumers deduplicate by event ID, and a later cancellation is a separate event.

6.6 Step 1 finds `available` at zero, so the update matches no row, the insert adds nothing and the statement returns no row. Step 2 looks up the `reservation_id`, finds the active reservation for the same order and product, and returns it with its expiry time, which is the answer the first request would have received. If the service checked the sold-out flag first, the customer who holds the unit would be told "sold out", would probably leave, and the unit would stay reserved until it expired five minutes later. The flag may only answer requests whose identifier the service has never stored.

Chapter 7

Check your understanding (Chapter 7). The stock proof rests on the guarded transitions: the conditional update that never takes a unit below zero, the single guarded transition that lets only one of expiry and confirmation consume a reservation, and, for buckets or regions, the conservation equation with its in-transit term. The workflow argument rests on the durable order states, each transition's operation ID, the payment-unknown and cancellation-pending states with owners, and the rule that a void is never treated as done until the provider confirms it.

7.1 Yes. The guard compares `expires_at` with `now()`, which is 12:04:58, the start of the transaction, so the confirmation wins three seconds after expiry; a test in PostgreSQL 18 showed exactly this. I1 and I3 still hold, because the state guard and the row lock still let only one of expiry and confirmation consume the reservation. The broken rule is the five-minute expiry policy, stretched by however long the transaction

stays open, so a unit that should have returned to stock goes to a late payer. Run the confirm as its own short transaction after the checks, or compare with `statement_timestamp()`, the start of the current statement. Credit answers that identify the transaction start time and keep the single guarded transition.

7.2 Holding until the outcome is known prevents late authorisations without stock but lets slow or abandoned checkouts lock scarce units; control it with a maximum hold and per-customer limits. A fixed deadline frees units predictably but needs fresh reservations or voids for late results; control it with a reconciliation job and an owned cancellation-pending queue.

7.3 The ceiling is 8 / 0.004 = 2,000 per second. With units in only 2 of 8 buckets, a random first pick misses with probability 6 / 8 = 75 percent. The service must try the remaining buckets, or check a total maintained in the same transactions, before answering “sold out”.

7.4 West available + East available + reserved + committed + in transit = 2,000. West asks East about transfer T. If East credited it, West marks T complete. If East has no record, East records T as refused, so it can never be credited later, and only then does West return the units. Reclaiming without East’s answer could let both regions sell the same units. A region restored from backup must replay the transfer history before selling.

7.5 Reservation-first keeps a unit for the paying customer but holds scarce stock; a late callback needs a fresh reservation or a void, and a timed-out void leaves the order in owned cancellation pending. Payment-first avoids holding stock for non-payers but can authorise payments with no stock left, making voids routine and needing the same pending state. Credit answers that cover both failures for both orderings and choose by the event’s policy.

Chapter 8

8.1 A duplicate callback can receive a correct HTTP 200 while its effect is applied twice. The test must show that stock did not change again, no second charge or capture was made, the order made no extra transition, and the event ID was recorded.

8.2 p95 latency measures only the time to an answer, while the policies admit different numbers of customers and leave different numbers of payments unresolved and units unsold. Further measurements: distinct customers admitted and confirmed, time from checkout to a final outcome, what rejected customers did next, and how many unresolved payments completed after the cutoff.

8.3 Reserve one unit, let the provider authorise while delaying its reply, let the reservation expire, then deliver the authorisation. Expect no confirmation against the expired reservation, and either a fresh reservation or a void with a durable void request ID and the order in cancellation pending. Then make the void time out and expect the order to stay visibly pending. The design fails if the order is confirmed against released stock, stock goes negative, a second charge is made, or the order is marked cancelled before the void is confirmed.

8.4 The history is invalid. In the sequential specification, r1 takes one unit, the repeat of r1 returns r1’s reservation without taking another, and r2 then takes the second unit, so r2 should have succeeded. The final state also breaks I1: A + R + C = 0 + 1 + 0 = 1, not 2, so a unit has leaked. A likely defect is that the decrement and the insert run in separate transactions: the repeat of r1 decremented and committed, then its insert failed with SQLSTATE 23505, and nothing put the unit back. The fix is one statement or one transaction that rolls back on 23505, followed by the step 2 lookup. A test that checked only the replies, all of which look plausible, would miss the leak; comparing reservation rows with the counter catches it.

8.5 The end-to-end mean is 40 + 60 + 150 = 250 milliseconds. Each stage takes 20 milliseconds with probability 0.97, at least 0.95, so each stage’s p95 is 20 milliseconds and their sum is 40. Both stages are fast together with probability 0.97 × 0.97 = 0.9409, below 0.95, so the end-to-end p95 is 20 + 800 = 820 milliseconds.

8.6 The late authorisation needs a stock condition, which the rule table cannot express, so `run()` checks it: when the key is `('expired', 'pending', 'authorise')` and `available >= 1`, the transition becomes `('active', 'authorised', -1)`, a fresh reservation; otherwise the existing void rule applies. To test the void branch inside a one-unit model, add an event in which another customer takes the returned unit, `('expired', 'pending', 'other_reserves')` with a stock change of –1, count it in a variable `other`, and change the assertion to `available + held + other == 1`. Two schedules then run: reserve, expire, authorise, confirm gives `committed/authorised, available=0`; reserve, expire, other_reserves, authorise, void_confirmed gives `expired/voided, available=0`. Credit answers that keep the one-unit invariant true in both schedules.

8.7 Utilisation is (350 + 300) × 0.0014 = 0.91, too close to 1: waits would grow long and any burst would overload the row. Two buckets give 0.455 each, still above 0.4; three give about 0.30 each, so three buckets are needed. At the end of the sale, units remain in fewer buckets, so a random first pick misses more often, up to two times in three when only one bucket has units; the service must try the other buckets before answering “sold out”, and arrival order across buckets is lost.

8.8 If the reply to a block request is lost, Aster has already debited 100 units into an in-transit record. A retry with a new transfer ID looks like a new request, so Aster debits a second block, and the partner may credit both, or credit one and leave 100 units stuck in transit. Either way, the conservation equation no longer matches what was sold. Contract term: “Each block request carries a transfer ID chosen by the partner; after a timeout the partner retries or queries with the same ID, and Aster returns the original result for an ID it already holds.” Test: drop the reply to a block transfer in the joint test, let the partner retry, and check that the transfer table holds one transfer and that Aster’s in-transit units plus the partner’s credited units equal 100.

Chapter 9

9.1 A closure-time measure counts only closed requests, so a division that leaves difficult requests open reports faster closures while customers wait longer. Publish the count and age of open requests alongside it, and define the population explicitly.

9.2 Total hours are 10 × 2 + 90 × 6 = 560 across 100 requests, a combined mean of 5.6 hours. Averaging the two means gives 4 hours and treats 10 requests as if they were as numerous as 90.

9.3 Keep the source system, source event ID, source version, event time, processing time and a digest of canonical content. Same ID, version and digest: a duplicate. A higher version of the same ID: a correction. Same ID and version with a different digest: a conflict for quarantine.

9.4 The event belongs to the 10 May window. It reaches Beacon on 13 May, after the publications of 11, 12 and 13 May, so the run that starts at 01:00 on 14 May includes it and restates 10 May at 07:00 on 14 May, within the seven-day window. In the accumulating and retracting refinement mode, the new value replaces the old and the publication records which one it supersedes. The dashboard marks 10 May as restated, shows the previous value and gives the reason, a late record from the late-arriving source.

9.5 The correct total is 700 × 3 + 300 × 8 = 4,500 hours over 1,000 requests, a mean of 4.5 hours. After the join, each large customer’s completion appears twice, so there are 1,300 rows and 2,100 + 2 × 2,400 = 6,900 hours, a mean of about 5.31 hours: the total is inflated by half and the mean drifts towards the duplicated group. Tests: the number of rows after the join must equal the number of completions, and the dimension’s business key must be unique among its current rows. Counting distinct request keys in the result is a useful third check.

Chapter 10

10.1 Four hours are 14,400 seconds, so the minimum average rate is 10,000,000 / 14,400 ≈ 694.44 records per second. It ignores transformation cost, skew, joins, retries, checks and publication, so a capacity plan needs measured rates on representative data and time reserved for reruns.

10.2 In a fresh ledger: P1, day 2, mean (2 + 6) / 2 = 4.0 hours over 2 requests; P2, mean (2 + 6 + 13) / 3 = 7.0 hours over 3, superseding P1 because r6 arrived late; P3, mean (5 + 6 + 13) / 3 = 8.0 hours over 3, superseding P2 because r4 was corrected. The latest mean is twice the first, yet nothing about day 2’s service changed: one slow closure arrived late and one start time was corrected. Only the ledger’s inputs and reasons show this, so the dashboard should display them with any restated value.

10.3 Key events by source and event ID and keep the highest version. A repeat of the same version and content has no effect; an older version never replaces a newer one; the same version with different content goes to quarantine for the source owner. Affected publications are restated only through the versioned correction process, and arrival order never decides.

10.4 For the monthly snapshot, batch fits naturally with a formal close at a fixed extract boundary. For the intraday view, only streaming can help, and only for the three streaming sources, so the view must say that extract-based sources are a day old. A hybrid is a strong answer.

10.5 A withdrawal is a new version of the same request with a withdrawn flag, for example (‘r3’, 2, 1, None), which `latest()` keeps as the current version and `publish()` excludes from the mean while listing it among the inputs. After r3 is withdrawn, day 1 is computed from r1 (4 hours) and r2 version 2 (16 hours): P4, mean 10.0 hours over 2 requests, supersedes P3 with the reason “withdrawal of r3”. The ledger must

show the fall from 3 requests to 2 and the reason, so that a reader does not mistake the higher mean for slower service.

Chapter 11

Check your understanding (Chapter 11). Evidence: the metric version and population used, source completeness for the period, the case mix by product and complexity, staffing and workload per agent, and the same measure for comparable divisions over time. Alternative explanations to rule out: a changed metric definition, a late or missing source, more difficult requests, more reopened cases, a policy or process change, and too few agents rather than slower ones. Even then, “less efficient” is a judgement about people that the data alone cannot support; a publishable sentence reports the measured difference and its likely causes.

11.1 (a) A key-set comparison finds both the duplicates and the missing records; a row count misses them, because they cancel. (b) Grouped totals by division find the misassignment; a key-set comparison misses it, because every key is present. (c) A digest of canonical content, or a value comparison, finds the changed amounts; grouped totals may miss them if the rounding differences cancel within a group. (d) A relationship check finds the orphan closures; counts and key sets on the closures table alone miss them. All checks must use the same extraction boundary.

11.2 Before republishing, reapply every lifecycle decision recorded since the backup from the controlled list, and verify that the removed record is absent from the raw zone and every rebuilt table, index, cache and export. A restore that ignores later deletion decisions is incomplete even when the database restore succeeds.

11.3 Check the metric version, source completeness, provisional data, case mix, reopened requests, late events, policy changes and staffing levels, and look for evidence that separates the explanations, such as the same slowdown in other divisions. A correlation with a division does not show that its staff caused it; a generated narrative should report supported observations with links and pass human review.

11.4 Model answer for Alternative B: own division, tenant in scope (allow); own division, tenant out of scope (deny); another division (deny); group analyst, approved aggregate (allow); group analyst, raw detail (deny); revoked analyst, anything (deny). Run every case through the dashboard, notebook, direct query and export, showing that a missing path is truly inaccessible rather than untested.

11.5 A curated view can keep its columns and meaning while one source moves from extracts to a change stream, so dashboards and notebooks need no change, which is the protection Codd’s data independence describes. It does not decide the grain of a fact, the metric definition or the correction policy.

11.6 The longest gap follows the 19:00 run: a correction arriving at 19:01 joins the nightly run at 01:00 and appears at 07:00, about 12 hours later. With one failed run the worst case is a correction arriving at 13:01 whose 19:00 run fails: it joins the nightly run and appears at 07:00, about 18 hours later. With two consecutive failures, a correction arriving at 07:01 misses 13:00 and 19:00 and appears at 07:00 the next day, just under 24 hours later, with no margin. The schedule can absorb one failed run safely, against three for the four-hourly schedule in ADR-B02, whose worst case after three failures is about 22 hours.

11.7 The request mixes two products. The regulator’s rule names the daily service figures, while the monthly snapshot exists so that finance can reproduce a closed month exactly. Restating a closed month would break that purpose and gain nothing the regulator asked for. A good response keeps the close, records late corrections as adjustments in the next month’s close with a reference to the daily restatement, and shows finance the reconciliation between the two. Credit answers that distinguish the products and keep both requirements visible.

Chapter 12

12.1 Monthly jobs, rare recovery procedures, occasional partner updates and support scripts may not run in one week, as the finance job shows. Add scheduler entries, database account lists, configuration and owner interviews, and observe across a month end.

12.2 Rehosting is favoured when the deadline is firm and phased migration cannot show credible progress by the decision date, provided the old platform can run securely on new hosting. Phased migration is favoured when a stable seam exists, CDC works on the source and the intermediate states can be operated. Compare the total cost of coexistence, migration and any later second move.

12.3 Assign authority at order creation, record it with the order and keep it for the order’s life: orders created before a cohort’s switch stay with the legacy platform until they close, or move through a governed

transfer. Never route updates by an attribute that can change, and cover partner, batch and support writers as well as the web shop.

12.4 Find the account's owner and the code or job that uses it, read what it changes, and ask the support manager whether the behaviour is still needed. If it is, rebuild it as a scheduled function of the new service and add it to the inventory, fence plan and dual-run comparison. If not, retire it by an approved change before cutover and disable the account.

12.5 Without the epoch term, a delayed dispatch command from the legacy platform could be accepted after cutover, risking a second dispatch. Orion can compensate by disabling the legacy platform's partner credentials at the fence, giving every dispatch command a unique ID that the partner deduplicates, and reconciling dispatches daily. These controls are weaker, because they rely on Orion finding every path to the partner.

Chapter 13

13.1 Changes committed between P0 and P1, from 01:00 to 01:20, are in neither input: the snapshot was taken before them and CDC starts after them. Each job checks only its own work, so both report success. Starting CDC before P0 would apply some changes that the snapshot already contains, so the applier must be idempotent, for example by applying each change only if its position is later than the one recorded for the row. The safe design starts CDC exactly at the snapshot's position.

13.2 Queued changes: 8 × 3,600 × 17.4 ≈ 501,000. The net drain is 250 − 17.4 = 232.6 per second, so catch-up takes about 501,000 / 232.6 ≈ 2,150 seconds, or 36 minutes, at the average rate; a weekday peak would take longer.

13.3 For the application, marketplace import, partner account, finance job and `ops_batch`: revoke write rights, end sessions, disable any scheduler entry, and attempt a probe write with each account, which must fail. The partner also confirms that it uses the new route. Record the final log position only after all five probes fail.

13.4 O-201 disappears from `legacy_order` but remains in the target, so the query reports it as "extra in target". The result is false: the target has simply not yet applied the deletion at position 1005. Keep deletions visible on the legacy side, for example as a tombstone row with the position of the deletion, or read deleted keys from the change log, and exclude any key whose deletion lies after the applied position from the "extra" check until the next run.

13.5 Returning authority is possible because reverse CDC keeps the legacy database current and the new states are disabled; it contains the defect quickly but needs a fence on the new store and a correction of VAT values already written. Fixing forward keeps authority in place and is faster for a small, well-understood fix. Needed information: the number of affected orders, whether invoices have gone out, the time to fix, and whether reverse CDC is current.

Chapter 14

14.1 The latest safe decision becomes 45 − 8 = 37 minutes. With 14-minute smoke tests, the revised plan decides at 6 + 3 + 6 + 2 + 14 + 3 = 34, which fits with three minutes to spare, and the rule changes to "smoke tests must pass by minute 34".

14.2 The worst time appears once in three rehearsals for steps 1, 2, 3 and 5, twice for step 4 and in all three for step 6, so the probability is $(1/3)^4 \times 2/3 \times 1 = 2/243$, under 1 percent. Summing the worst times is therefore very conservative when steps vary independently. The reasoning fails when steps share a cause, such as month-end load slowing both the drain and the reconciliation, and when three rehearsals have not revealed the true worst case of a step.

14.3 "ADR-O07, week −16. The partner's status API will arrive six weeks late, in week −8. We will cut over as planned in week −10, using the partner's 15-minute status file, converted to API calls by Orion's adapter. Conditions: rehearsal 4 in week −12 processes a full day's file; the business owner approves a temporary 20-minute status objective; the partner moves to the API by week −4; after a no-go that cannot be fixed within a week, we give the hosting extension notice by week −8. Owner: migration lead. Next review: after rehearsal 4." Credit a notice that names the option, conditions, owner and review date.

14.4 Clean final reconciliation (report, confirmed by the data owner); every legacy writer fenced (failed probes, database lead); epoch 8 active with the applier rejected (failed applier write, database lead); smoke tests passed, including the partner call (test log, test lead); reverse path ready (operators and scripts, operations lead). The cutover manager collects the confirmations and the business owner decides.

14.5 The proposal sends the first customer writes to the new service before any end-to-end test in production. Any fault in identities, routing, configuration or the partner call would reach customers, and the six-minute reverse path would be lost because real orders would exist only in the new store. Saving nine minutes does not justify that when the plan already has ten minutes of margin.

Chapter 15

15.1 “For release 1.4, at the tested peak of 20 submissions per second, the receipt path meets Q1, Q3 and Q6: the peak load test returned 95 percent of receipts within 0.6 seconds (X-02), local-commit and outage tests lost no receipt (X-03 to X-05), and the tenant-scope tests disclosed nothing (X-06). Open conditions: the vendor confirms CRM B’s quota in writing (sponsor, by 30 April), and a timed database restore is rehearsed (operations lead, by 15 May).” Credit scope, version, evidence and owned conditions.

15.2 “Which requirement do you expect microservices to improve: independent release, scaling one part, or isolating a failure? If we pick one, I will compare both designs against that scenario, including the cost of the extra network boundary. If the split wins, we revise the decision record; if not, we record the trigger that would reopen it.” Credit a respectful answer that asks for the requirement.

15.3 The `ops_batch` script is closed once rebuilt or retired and a probe write with its account fails. The partner delay is an accepted residual risk under ADR-O07, with an owner and review date. The DNS time-to-live is a deferred investigation until it is lowered and verified a day before cutover.

15.4 Executive: “Demand will far exceed our 2,000 units. We will admit up to 50 checkouts at a time for the product and ask others to retry, so that admitted customers get an answer within seconds and nothing is oversold. In testing this sold 1,988 units, compared with 1,942 when everyone was admitted.” Engineering: a 50-attempt admission gate per hot product, the reservation statement at READ COMMITTED, a sold-out flag, and the experiment results with their limits.

15.5 P = 4 × 3 = 12 and Q = 2 × 5 = 10, so P ranks first. Rating Q’s likelihood at 3 gives 15 and reverses the order, as would rating P’s likelihood at 3, giving 9. The review should record the reasoning behind each score and treat the risks as close.

Chapter 16

16.1 Meridian and Aster both record durable intent with an operation ID so that a crash cannot erase a commitment. Meridian’s receipt says nothing about the CRM case, which the CRM owns once confirmed, while Aster’s confirmed order depends on two authorities: its own stock and the payment provider.

16.2 Apply the same rubric to both versions and cite specific improvements, such as a failure sequence that exposed a race or a repeated experiment that confirmed a recovery assumption. Do not award points for extra pages. State the remaining weaknesses and whether the revision changes the release decision.

16.3 Technical, month one: build and fault-test the outbox and claim query, evidenced by code and an experiment record. Stakeholder, month two: run a review of one decision, evidenced by a revised decision record. Operational, month three: rehearse a recovery with a colleague, evidenced by a timed report. State which experiences were simulated.

16.4 In Raft, an entry is committed once a majority of servers has stored it; any future leader holds every committed entry, and all servers apply the same entries in the same order, so a committed entry survives a leader failover. With PostgreSQL’s default asynchronous replication, a failover can lose commits that the primary had already confirmed, so a customer may hold a reservation that the new primary does not have, and the stock count is one unit too high. With `ANY 1 (s1, s2)` and `synchronous_commit = on`, every confirmed commit is on disk on at least one standby, so promoting the standby with the latest commits keeps it. The price is a standby round trip inside every commit, which lengthens the lock-hold time and lowers the hot row’s ceiling, and commits stall if no listed standby responds. In both cases the application still handles unknown outcomes: a client whose request timed out retries with the same `reservation_id`, and the step 2 lookup returns the reservation if it survived; if it did not, the retry is a new attempt. Payment compensation lies outside the log. Credit answers that make the PostgreSQL guarantee conditional on synchronous commit and the failover choice.

16.5 At risk: the claim that every legacy and support writer was fenced and its account disabled; the retirement checklist; the authority epoch, if the account can declare the current epoch; and the assumption that the writer inventory was complete. Immediate actions: identify the account, its owner and the statements it ran, revoke its privileges, and check whether the 40 updates conflicted with the new service’s orders, correcting them through the normal business process. Follow-up: add the source to the writer inventory with its evidence, decide whether its function must be rebuilt in the new service, and add a

daily check for writes by unexpected roles. Credit answers that connect the event to the fence and the inventory without inventing facts about the account.

Glossary

Accumulating and retracting mode. A refinement mode (an accumulation mode in the Dataflow Model) for emitting a later result for a window in which the new value replaces the earlier one and the earlier value is explicitly withdrawn; Beacon's restatements work this way.

Admission control. A rule that limits how much work enters a system, for example at most 50 reservations in flight for one product, so that admitted work finishes and the rest receives a clear answer.

Architecture decision record (ADR). A short document that records one significant decision with its context, alternatives, consequences, evidence and review triggers, under a stable identifier such as ADR-M05.

Architecture dossier. The connected set of documents that explains a design and the reasons for it: brief, quality scenarios, views, decision records, experiments, findings and a decision brief.

Assumption. A statement believed to be true but not yet verified; it needs an owner, a verification task and a deadline.

Authority epoch. A number that increases each time write authority over a domain moves; the data store rejects writers whose declared epoch is not the current one.

Bucket. One of several rows that together hold a product's allocation, so that reservations spread their locks across rows and the throughput ceiling rises to about k / h for k buckets.

Change card. A one-page statement of a new fact that arrives part-way through a studio, with the assumptions and decisions it affects and the question it raises.

Change data capture (CDC). Reading committed changes from a database's transaction log and applying them to another store, so that every committed writer is seen.

Closed workload. A load test in which a fixed population of virtual users each waits for a response before sending again, so the offered load falls as the system slows.

Compensation. A business action that semantically undoes an earlier step of a saga, such as releasing a reservation or voiding a payment authorisation; it can itself fail or stay pending.

Constraint. A requirement that decides whether an option is eligible at all; it is applied before preferences are scored.

Cutover. The planned moment when write authority moves from an old system to a new one, usually inside a maintenance window.

Decision brief. A one-page summary for the person who must decide: what changed, what stays the same, the options with cost and effect, a recommendation and its conditions.

Dossier. See architecture dossier.

Dual run. A period in which the old system stays authoritative while the new one receives the same data or requests, and daily comparisons check that both agree.

Event time. The time at which the source says a business event happened, as opposed to when it was processed or published.

Fencing token. A number that increases each time a lease or authority is granted; the system that receives a write rejects any write carrying an older number.

Grain. The exact meaning of one row in a fact table, such as one row per request and reporting day.

Handoff. The delivery of a stored submission to the business unit's CRM, tracked in one of five states: `pending`, `in_flight`, `confirmed`, `retry_due` or `manual_review`.

Hot row. A single database row that a large share of transactions must lock, such as the stock row of a promoted product.

Idempotency key. A client-chosen identifier sent with a request so that the server can recognise a repeat and return the original result instead of acting twice.

In-transit record. A durable record of units or work that one party has debited and another has not yet credited, identified by a transfer ID so that the transfer completes exactly once.

Invariant. A condition that must hold after every transaction, such as A + R + C = 2,000 for Aster's event stock.

Isolation level. The degree to which concurrent transactions see each other's changes; PostgreSQL offers READ COMMITTED, REPEATABLE READ and SERIALIZABLE.

Latest safe decision point. The last moment in a cutover window at which a “no” can still be carried out in time, equal to the end of the window minus the rehearsed time to reverse.

Lease. A time-limited claim on a piece of work, recorded with its owner and expiry, after which another worker may take the work.

Linearizability. A correctness condition under which every operation appears to take effect at one instant between its start and its end, in an order that the object’s sequential specification allows.

Little’s law. For a stable system over the long run, the average number of items in the system equals the arrival rate multiplied by the average time each item spends there: L = λW.

Lock-hold time. The time a transaction holds a row lock, from the update until the commit completes; written h, it limits the row to about 1 / h updates per second.

Metric contract. A definition of a measure that fixes its population, start and end events, time zone, exclusions, owner and version.

Open workload. A load test in which requests arrive on a schedule whether or not earlier requests have finished, as real customers do during a surge.

Outbox. A table written in the same transaction as a business change, recording the intent to send a message; a worker or relay delivers it later.

p50, p95, p99. The 50th, 95th and 99th percentiles of a set of measurements; p50 is the median, and p95 is the value that 95 percent of observations do not exceed.

Percentile. The value that a stated percentage of observations do not exceed; it says nothing about the observations above it.

Preference. A statement that ranks eligible options, such as low cost or fast delivery, without excluding any of them.

Processing time. The time at which a pipeline received or processed a record, as opposed to the event time the source reports.

Publication ledger. An append-only record of every published result, with its identifier, value, the exact inputs it used and the publication it supersedes.

Publication time. The time at which a particular result became visible to its readers.

Quality gate. A check that must pass before a result is published; a gate that can never fail protects nothing.

Quality scenario. A requirement stated as a stimulus, an environment, a required response and a measure, so that it can be tested.

Receipt. The portal’s confirmation that it has durably stored a submission and taken responsibility for its handoff.

Reconciliation. A comparison of two stores, by key sets, versions, relationships, totals or content digests, within a declared boundary, to find records that are missing, extra or different.

Refinement mode. How a later result for a window relates to an earlier one: discarding, accumulating, or accumulating and retracting; the Dataflow Model calls it the accumulation mode.

Replication lag. The time between a change committing in the source and the same change being applied in the target.

Reservation. A record that holds one unit of stock for an order until it is committed, released or expired; its identifier makes a repeated request harmless.

Restatement. A new publication of a past period’s result that replaces an earlier one because late, corrected or withdrawn data changed it.

Retry storm. A surge of load caused by failed requests being retried, which adds more contention and causes further failures.

Review trigger. An observable condition, recorded in a decision record, that should reopen the decision when it occurs.

Saga. A long-lived transaction split into a sequence of smaller transactions, each paired with a compensating action that semantically undoes it if the sequence cannot finish.

Serialization failure. An error, SQLSTATE 40001, that PostgreSQL raises under REPEATABLE READ or SERIALIZABLE when a transaction would conflict with a concurrent one; the whole transaction must be retried.

Strangler fig. A migration approach in which a new system grows around the old one behind a routing seam until the old one can be removed.

Synchronous replication. Replication in which a commit waits until one or more standbys confirm that they have stored it, so that a failover does not lose confirmed transactions.

Tenant. A customer organisation whose data is kept separate from other customers’ data in a shared system.

Trigger (Dataflow Model). The rule that decides when to emit a result for a window, for example at 07:00 each morning for the previous day, and at each later run while late data for that day keeps arriving.

Utilisation. The fraction of time a resource is busy, equal to its arrival rate multiplied by the time each request occupies it; waiting grows sharply as utilisation approaches 1.

Void. The cancellation of a payment authorisation before money is captured, as opposed to a refund of money already captured.

Watermark. A system’s estimate of how complete its input is up to a given event time.

Window. A grouping of events by event time, such as one business day, for which a result is computed.

Write fence. A control that stops an old authority from changing data after authority has moved, enforced where the data is stored.

Zero-failure bound. The upper limit on a failure probability that a run of n independent failure-free trials supports; at 95 percent confidence it is 1 − 0.05^(1/n).

Experiments, templates and research tasks

This chapter collects extensions to the experiments, the dossier templates and the research tasks for the four studios. Each experiment builds on a listing printed in its chapter. Before changing a listing, write down what you expect; afterwards, record what happened, why, and what the model leaves out.

Experiments

- **Drain time and failure bounds (Chapter 2).** Add a parameter for the share of attempts that fail and reproduce every recovery time in Chapter 5, including the 16-minute recovery with status polling paused above 1,000 pending handoffs. Compute the zero-failure bound for 300, 3,000 and 30,000 trials and compare it with the rule of thumb 3 / n. List the measurements that would replace each constant-rate assumption.
- **The outbox and its workers (Chapter 4).** With PostgreSQL, create the outbox schema and a few hundred submissions. Run the claim query in two sessions before committing either, and confirm that they receive different rows. Expire the lease of a CRM B handoff, run the sweep, and show that no worker can claim it for fifteen minutes and that its next action is a lookup. Then show that a confirm carrying the old attempt number changes nothing, even when it comes from a worker with the same owner ID. Implement the eight-handoff limit from exercise 4.5 and test it with two simultaneous claims.
- **The hot row (Chapter 6).** Create the stock and reservation tables with one product and two units. In two sessions, show that the second reservation waits and then succeeds under READ COMMITTED, and fails with SQLSTATE 40001 under REPEATABLE READ. Reproduce the five rows of the retry table in Chapter 6, including the repeat of the winning request after the product has sold out. Time a reservation sent as one statement and as three round trips, and estimate h and 1 / h for each.
- **Event orderings (Chapter 8).** Write the invariant for an order of two units, then extend the state model to two units. Add the fresh-reservation path from exercise 8.6 and a void that times out.
- **Publication ledger (Chapter 10).** Add the withdrawal record from exercise 10.5. Then publish a second measure, the number of open requests, from the same inputs, and decide whether a change in either measure creates a new publication.
- **Correction schedules (Chapter 11).** Write a function that returns the worst-case delay from a record's arrival to its publication for a list of runs, each with a start time and a duration, when k consecutive runs fail. Reproduce the CC-B1 table, then find the cheapest schedule that survives two failed runs within 24 hours.
- **The access matrix (Chapter 11).** Add an `export` action allowed only to the publisher, and an analyst with a delegation for one tenant of another division. Update the hand-written matrix before changing the policy, then introduce a deliberate defect and confirm that the matrix catches it.
- **Reconciliation and fencing (Chapter 13).** Add an order-lines table and a check for orders whose line totals differ, a mapping-version column, and the deletion tombstones from exercise 13.4. With PostgreSQL, create the epoch fence on small `orders`, `order_lines` and `status_history` tables and reproduce its behaviours with three sessions, including one that sets `session_replication_role` to `replica`, first with `ENABLE ALWAYS` and then without it. Then publish the three tables from a second PostgreSQL instance, subscribe as `orion_applier`, and show that the apply worker passes the fence at epoch 7, is stopped at epoch 8, and slips through if the insert, update and delete triggers are made statement-level.

Dossier templates

Replace each prompt with case-specific reasoning, and keep identifiers stable so that reviewers can follow the links between documents. A template filled with generic language explains nothing; each field should make a specific claim about your case.

Decision brief

- **Decision ID, version and date:**
- **Decision needed, from whom and by when:**
- **What changed or what problem exists, in business terms:**
- **What stays the same in every option:**
- **Options, each with cost, effect on customers and main risk:**
- **Recommendation and the reason for it:**
- **Conditions, open questions and their owners:**

- **Links to the supporting scenarios, decision records and experiments:**

Requirement and assumption register

ID	Type	Statement and scope	Owner	Verification task or evidence	Consequence if false
R-01	Requirement				
A-01	Assumption				

Quality scenario

- **ID:**
- **Stimulus and its source:**
- **Environment, such as normal operation, peak load or a named failure:**
- **Required response:**
- **Measure and threshold:**
- **Owner of the promise:**

Architecture decision record

- **ID, status and date:**
- **Context and the scenarios it serves:**
- **Decision:**
- **Mechanism and the authority that enforces it:**
- **Alternatives considered and why each was not chosen:**
- **Consequences, including residual risks:**
- **Evidence identifiers:**
- **Review triggers, each an observable condition:**
- **Supersedes, or is superseded by:**

Experiment record

- **Question, phrased so that it can fail:**
- **Claim under test and the decision it informs:**
- **Model or build, configuration and data version:**
- **Independent variables and controlled conditions:**
- **Workload model, open or closed, and offered rate:**
- **Failures injected and their timing:**
- **Business-level observations:**
- **Decision rule, written before the run:**
- **Result, variation and threats to validity:**
- **Design consequence:**
- **How to reproduce the experiment:**

Change card

- **Card ID and date:**
- **New fact and its source:**
- **Assumptions and decisions affected:**
- **Question for the team:**
- **Response chosen, and the decision record that records it:**

Review finding

- **ID and the requirement affected:**
- **Architecture version reviewed:**
- **Defect or uncertainty, and its consequence:**
- **Severity and the reason for it:**
- **Evidence:**
- **Correction, or the residual risk accepted:**

- **Owner and deadline:**
- **Closure evidence:**
- **Status: open, closed, accepted residual risk or deferred investigation:**

Migration gate

- **Transition, with the current and target write authority:**
- **Scope of the data and the writers covered:**
- **Source position and mapping version:**
- **Writers fenced, and the evidence that each is stopped:**
- **Reconciliation results:**
- **Access, capacity and operating checks:**
- **Forward duration, reverse duration and latest decision point:**
- **Recovery before customer writes:**
- **Recovery after customer writes:**
- **Decision owner and communication plan:**
- **Retirement evidence:**

Research tasks

Write one critique of 400 to 600 words for each studio. Identify the source's question, method, assumptions, supported conclusion and limits, then explain one way it transfers to the studio and one way it does not. Suggested sources and questions are:

- **Integration:** Saltzer, Reed and Clark on end-to-end arguments [18], or Helland on idempotence [12]. Where should Meridian check that a request reached the service team?
- **Transactions:** Garcia-Molina and Salem on sagas [22], or Helland on life beyond distributed transactions [21]. Which of Aster's guarantees need one transactional scope, and which can rely on messages?
- **Data:** Akidau and colleagues on the Dataflow Model [26], or Codd on the relational model [33]. How should Beacon trade completeness against the 07:00 deadline?
- **Modernisation:** Fowler on the strangler fig application [34], or Kleppmann on fencing [11]. Which of Orion's writers can a routing seam control, and which need a fence?

Chapter 16's research synthesis then compares the kinds of knowledge these sources provide.

References

References are numbered in order of first citation. Online sources were accessed in October 2026. Product documentation changes; check it again before implementation.

[1] S. Brown, “The C4 model for visualising software architecture,” *c4model.com*, n.d. Accessed: Oct. 11, 2026. [Online]. Available: https://c4model.com/

[2] M. Nygard, “Documenting architecture decisions,” *Cognitect blog*, Nov. 15, 2011. Accessed: Oct. 11, 2026. [Online]. Available: https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions

[3] R. Kazman, M. Klein, and P. Clements, “ATAM: Method for architecture evaluation,” Software Engineering Institute, Carnegie Mellon University, Pittsburgh, PA, USA, Tech. Rep. CMU/SEI-2000-TR-004, Aug. 2000. Accessed: Oct. 11, 2026. [Online]. Available: https://www.sei.cmu.edu/library/atam-method-for-architecture-evaluation/

[4] B. Schroeder, A. Wierman, and M. Harchol-Balter, “Open versus closed: A cautionary tale,” in *Proc. 3rd USENIX Symp. Networked Systems Design and Implementation (NSDI ‘06)*, San Jose, CA, USA, May 2006, pp. 239-252. Accessed: Oct. 11, 2026. [Online]. Available: https://www.usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale

[5] J. D. C. Little, “A proof for the queuing formula: L = λW,” *Operations Research*, vol. 9, no. 3, pp. 383-387, 1961. doi: 10.1287/opre.9.3.383

[6] D. L. Parnas, “On the criteria to be used in decomposing systems into modules,” *Communications of the ACM*, vol. 15, no. 12, pp. 1053-1058, Dec. 1972. doi: 10.1145/361598.361623

[7] Amazon Web Services, “Transactional outbox pattern,” *AWS Prescriptive Guidance: Cloud design patterns, architectures, and implementations*. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

[8] J. Jena and S. Dalal, “The Idempotency-Key HTTP header field,” IETF HTTPAPI Working Group, expired Internet-Draft draft-ietf-httpapi-idempotency-key-header-07, Oct. 15, 2025, work in progress. Accessed: Oct. 11, 2026. [Online]. Available: https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/07/

[9] R. Fielding, M. Nottingham, and J. Reschke, Eds., “HTTP semantics,” IETF, RFC 9110, Jun. 2022, doi: 10.17487/RFC9110. Accessed: Oct. 11, 2026. [Online]. Available: https://www.rfc-editor.org/info/rfc9110

[10] PostgreSQL Global Development Group, “SELECT: The locking clause,” *PostgreSQL 18 Documentation*. Accessed: Oct. 11, 2026. [Online]. Available: https://www.postgresql.org/docs/18/sql-select.html

[11] M. Kleppmann, “How to do distributed locking,” Feb. 8, 2016. Accessed: Oct. 11, 2026. [Online]. Available: https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html

[12] P. Helland, “Idempotence is not a medical condition,” *ACM Queue*, vol. 10, no. 4, pp. 30-46, Apr. 2012. doi: 10.1145/2181796.2187821

[13] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Communications of the ACM*, vol. 21, no. 7, pp. 558-565, Jul. 1978. doi: 10.1145/359545.359563

[14] RabbitMQ, “Consumer acknowledgements and publisher confirms,” *RabbitMQ Documentation*. Accessed: Oct. 11, 2026. [Online]. Available: https://www.rabbitmq.com/docs/confirms

[15] Microsoft, “Compare AWS and Azure compute services,” *Azure Architecture Center*, Jun. 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/aws-professional/compute

[16] Microsoft, “Compare AWS and Azure database services,” *Azure Architecture Center*, Oct. 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/aws-professional/databases

[17] Microsoft, “Comparing AWS and Azure messaging services,” *Azure Architecture Center*, Sep. 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/aws-professional/messaging

[18] J. H. Saltzer, D. P. Reed, and D. D. Clark, “End-to-end arguments in system design,” *ACM Transactions on Computer Systems*, vol. 2, no. 4, pp. 277-288, Nov. 1984, doi: 10.1145/357401.357402. Accessed: Oct. 11, 2026. [Online]. Available: https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf

[19] PostgreSQL Global Development Group, “Transaction isolation,” *PostgreSQL 18 Documentation*, sec. 13.2. Accessed: Oct. 11, 2026. [Online]. Available: https://www.postgresql.org/docs/18/transaction-iso.html

[20] PostgreSQL Global Development Group, “Serialization failure handling,” *PostgreSQL 18 Documentation*, sec. 13.5. Accessed: Oct. 11, 2026. [Online]. Available: https://www.postgresql.org/docs/18/mvcc-serialization-failure-handling.html

[21] P. Helland, “Life beyond distributed transactions: An apostate’s opinion,” in *Proc. 3rd Biennial Conf. Innovative Data Systems Research (CIDR)*, Asilomar, CA, USA, Jan. 2007, pp. 132-141. Accessed: Oct. 11, 2026. [Online]. Available: https://www.cidrdb.org/cidr2007/papers/cidr07p15.pdf

[22] H. Garcia-Molina and K. Salem, “Sagas,” in *Proc. 1987 ACM SIGMOD Int. Conf. Management of Data*, San Francisco, CA, USA, May 1987, pp. 249-259. doi: 10.1145/38713.38742

[23] M. P. Herlihy and J. M. Wing, “Linearizability: A correctness condition for concurrent objects,” *ACM Transactions on Programming Languages and Systems*, vol. 12, no. 3, pp. 463-492, Jul. 1990. doi: 10.1145/78969.78972

[24] J. Dean and L. A. Barroso, “The tail at scale,” *Communications of the ACM*, vol. 56, no. 2, pp. 74-80, Feb. 2013. doi: 10.1145/2408776.2408794

[25] R. Kimball, “Fact tables,” *Kimball Group*, Nov. 5, 2008. Accessed: Oct. 11, 2026. [Online]. Available: https://www.kimballgroup.com/2008/11/fact-tables/

[26] T. Akidau, R. Bradshaw, C. Chambers, S. Chernyak, R. J. Fernández-Moctezuma, R. Lax, S. McVeety, D. Mills, F. Perry, E. Schmidt, and S. Whittle, “The Dataflow Model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing,” *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 1792-1803, Aug. 2015, doi: 10.14778/2824032.2824076. Accessed: Oct. 11, 2026. [Online]. Available: https://www.vldb.org/pvldb/vol8/p1792-Akidau.pdf

[27] Apache Software Foundation, “Design,” *Apache Kafka 4.3 Documentation*. Accessed: Oct. 11, 2026. [Online]. Available: https://kafka.apache.org/43/design/design/

[28] Microsoft, “Compare AWS and Azure analytics services,” *Azure Architecture Center*, Sep. 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/aws-professional/analytics

[29] European Parliament and Council of the European Union, “Regulation (EU) 2016/679 (General Data Protection Regulation),” *Official Journal of the European Union*, L 119, pp. 1-88, May 4, 2016. Accessed: Oct. 11, 2026. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2016/679/oj

[30] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero trust architecture,” NIST SP 800-207, Aug. 2020. doi: 10.6028/NIST.SP.800-207

[31] National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile,” NIST AI 600-1, Jul. 2024. doi: 10.6028/NIST.AI.600-1

[32] E. Tabassi, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” NIST AI 100-1, National Institute of Standards and Technology, Jan. 2023. doi: 10.6028/NIST.AI.100-1

[33] E. F. Codd, “A relational model of data for large shared data banks,” *Communications of the ACM*, vol. 13, no. 6, pp. 377-387, Jun. 1970. doi: 10.1145/362384.362685

[34] M. Fowler, “Original strangler fig application,” *martinfowler.com*, Jun. 29, 2004. Accessed: Oct. 11, 2026. [Online]. Available: https://martinfowler.com/bliki/OriginalStranglerFigApplication.html

[35] M. Fowler, “Strangler fig,” *martinfowler.com*, Aug. 22, 2024. Accessed: Oct. 11, 2026. [Online]. Available: https://martinfowler.com/bliki/StranglerFigApplication.html

[36] Microsoft, “Strangler Fig pattern,” *Azure Architecture Center*. Accessed: Oct. 11, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/patterns/strangler-fig

[37] PostgreSQL Global Development Group, “Logical replication: Security,” *PostgreSQL 18 Documentation*. Accessed: Oct. 11, 2026. [Online]. Available: https://www.postgresql.org/docs/18/logical-replication-security.html

[38] PostgreSQL Global Development Group, “Logical replication: Architecture,” *PostgreSQL 18 Documentation*, sec. 29.9. Accessed: Oct. 11, 2026. [Online]. Available: https://www.postgresql.org/docs/18/logical-replication-architecture.html

[39] PostgreSQL Global Development Group, “ALTER TABLE,” *PostgreSQL 18 Documentation*. Accessed: Oct. 11, 2026. [Online]. Available: https://www.postgresql.org/docs/18/sql-altertable.html

[40] N. Misch, “Empty search_path in logical replication apply worker and walsender,” *pgsql-committers* mailing list, Aug. 10, 2020, commit 11da970, security fix for CVE-2020-14349. Accessed: Oct. 11, 2026. [Online]. Available: https://www.postgresql.org/message-id/E1k5AbJ-0004t8-6p%40gemulon.postgresql.org

[41] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *Proc. 2014 USENIX Annual Technical Conference (USENIX ATC 14)*, Philadelphia, PA, USA, Jun. 2014, pp. 305-319. Accessed: Oct. 11, 2026. [Online]. Available: https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro

[42] PostgreSQL Global Development Group, “Log-shipping standby servers: Synchronous replication,” *PostgreSQL 18 Documentation*, sec. 26.2.8. Accessed: Oct. 11, 2026. [Online]. Available: https://www.postgresql.org/docs/18/warm-standby.html#SYNCHRONOUS-REPLICATION

Index

Page numbers point to definitions and main discussions in the chapters.

A

admission control 41
architecture decision record (ADR) 6, 8
assumption 7–8, 12, 75–76, 78
authority epoch 62, 66, 68–69, 71, 76

B

bucket 36–38, 42–44

C

change card 24, 42, 56, 71–72, 77
change data capture (CDC) 60–62
closed workload 10
compensation 35, 38, 44, 77
constant-rate queue model 9–10
constraint 6–8, 19, 77–78
container view 7, 16, 34, 74
context view 7
controlled conditions 9, 12
cutover 66–67, 69–72

D

data view 7
decision brief 28, 43, 52, 57, 73, 76
dependent variables 9
deployment view 7–8, 76
direct replacement 59–60, 72
dossier 8, 26, 72, 76–78
dual run 62–63, 68, 72

E

event time 45–47

F

fencing token 18, 20–21, 23, 67

G

grain 45–47, 57

H

handoff 14, 16, 19–20, 25, 27
hot row 36, 38, 77–78

I

idempotency key 13, 18, 22–23
independent variables 9, 12
in-transit record 37, 42
invariant 30, 34, 38–39, 43–44
isolation level 30, 32

L

latest safe decision point 59, 70, 72
lease 18–21, 23, 27
linearizability 39
Little's law 11–12
lock-hold time 33–34, 42–44, 77

M

metric contract 45, 47, 57

O

outbox 13, 15–16, 19, 22–23

P

percentile 9, 39, 41, 44
phased migration 59–61, 72
preference 7–8, 73–75, 77
processing time 45–47
publication ledger 48, 50–51, 57
publication time 45–47

Q

quality gate 51
quality scenario 8–9, 16, 24, 27

R

READ COMMITTED 30, 32–34, 42–43
receipt 8, 13, 15, 18, 24, 27
reconciliation 52, 63, 65, 69–70, 72
refinement mode 46–47
REPEATABLE READ 30, 32, 34
replication lag 9, 59, 62
reservation 31–32, 34–35, 40, 42
restatement 47–48, 50, 56–58
retry storm 33
review trigger 8, 28, 43, 48, 57, 77

S

saga 30, 35
sequence or state view 7
SERIALIZABLE 30, 32–34
serialization failure 32
strangler fig 59–60

T

temporary rehosting 59–61, 71–72
tenant 13, 18–19, 27, 55, 67
transactional outbox 13
trigger (Dataflow Model) 45–48, 57

U

utilisation 32–34, 42–44

V

void 36, 38, 40, 44

W

watermark 45–47
window 45–48, 51, 56
write fence 59, 62, 65